

Requirements-Based Test Generation: A Comprehensive Survey

ZHENZHEN YANG, School of Computer Science and Engineering, Macau University of Science and Technology, China and School of Artificial Intelligence, Zhejiang Polytechnic University of Mechanical and Electrical Engineering, China

RUBING HUANG, School of Computer Science and Engineering, Macau University of Science and Technology, China and Macau University of Science and Technology Zhuhai MUST Science and Technology Research Institute, China

CHENHUI CUI, School of Computer Science and Engineering, Macau University of Science and Technology, China

NAN NIU, School of Computing, University of North Florida, USA

DAVE TOWEY, School of Computer Science, University of Nottingham Ningbo China, China

As an important way of assuring software quality, software testing generates and executes test cases to identify software failures. Many strategies have been proposed to guide test-case generation, such as source-code-based approaches and methods based on bug reports. *Requirements-based test generation* (RBTG) constructs test cases based on specified requirements, aligning with user needs and expectations, without requiring access to the source code. Since its introduction in 1994, there have been many contributions to the development of RBTG, including various approaches, implementations, tools, assessment and evaluation methods, and applications. This paper provides a comprehensive survey on RBTG, categorizing requirements types, classifying approaches, investigating types of test cases, summarizing available tools, and analyzing experimental evaluations. This paper also summarizes the domains and industrial applications of RBTG, and discusses some open research challenges and potential future work.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; **Requirements analysis**; • **General and reference** → **Surveys and overviews**.

Additional Key Words and Phrases: Software testing, test generation, software requirements, survey

1 INTRODUCTION

Software development is a complex process driven by rapidly evolving technologies and continuously changing requirements. It is often influenced by shifts in business processes. A key part of the software development process is software testing, which helps ensure its quality. Testing involves verifying that the software satisfies the

Authors' Contact Information: Zhenzhen Yang, 3240002362@student.must.edu.mo, School of Computer Science and Engineering, Macau University of Science and Technology, Taipa, Macau, China and School of Artificial Intelligence, Zhejiang Polytechnic University of Mechanical and Electrical Engineering, Hangzhou, Zhejiang, China; Rubing Huang, rbhuang@must.edu.mo, School of Computer Science and Engineering, Macau University of Science and Technology, Taipa, Macau, China and Macau University of Science and Technology Zhuhai MUST Science and Technology Research Institute, Zhuhai, Guangdong Province, China; Chenhui Cui, 3230002105@student.must.edu.mo, School of Computer Science and Engineering, Macau University of Science and Technology, Taipa, Macau, China; Nan Niu, nan.niu@uc.edu, School of Computing, University of North Florida, Jacksonville, FL, USA; Dave Towey, dave.towey@nottingham.edu.cn, School of Computer Science, University of Nottingham Ningbo China, Ningbo, Zhejiang, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s).

ACM 1557-7392/2025/10-ART

<https://doi.org/10.1145/3771727>

specified requirements, and can account for over half of the total development costs [195]. There are three main stages in software testing: test-case generation, execution, and evaluation. Among these, test-case generation may represent the most intellectually demanding process. Manual test-case generation can be time-consuming and prone to human error.

Automated test-case generation has been widely regarded as an effective approach to reducing effort and cost [140]. Test cases can be derived from requirements specifications, design artifacts [320], source code [117], or historical test artifacts (such as bug reports) [218]. While code-based test-case generation can effectively identify errors at the implementation level, it may not address issues arising from misunderstandings or mistakes in the analysis or design phases [315]. Faulty code can also mislead the test-generation process. Automating test-case generation from bug reports involves reproducing reported issues following the steps provided by users [92, 218]. This can be valuable for managing and analyzing a large number of bug reports, thus reducing the manual effort required. However, there may be challenges due to the informal nature of user-reported bugs, and these reports may be mainly from the software maintenance phase. *Requirements-based test generation* (RBTG), which relies on requirements specifications or models (before any code is written), may provide a more robust solution. By generating test cases early in the development lifecycle, RBTG can detect inconsistencies and ambiguities in requirements, reducing the risk of propagating errors into later stages of development. This early intervention can significantly lower development costs by addressing defects earlier.

Due to its simplicity and no specialized training being needed to understand it, *natural language* (NL) has often been used for software requirements' documentation [80]. However, NL can be imprecise and ambiguous, potentially making it unsuitable for automated test-case generation. Over the years, significant research has been dedicated to addressing the challenge of RBTG [230]. To address the limitations of NL, semi-formal and formal representations have also been proposed [97]. RBTG can be traced back to Peng Lu in 1994 [178]. Mustafa et al. [195] presented a review of RBTG from 2000 to 2018, reporting that most approaches used *Unified Modeling Language* (UML) for RBTG, with a primary focus on functional testing. Farooq and Tahreem [91], Jin and Lano [140], and Ahmad et al. [4] also examined UML-based methodologies for test-case generation. Recent studies have also explored the application of *natural language processing* (NLP) techniques to the testing of NL-based requirements [42] [103]. Wang et al. [307] provided a comprehensive analysis of *large language models* (LLMs) in software testing, while Fontes and Gay [99] investigated the integration of *machine learning* (ML) techniques for automated test generation. Despite these advances, there is still a lack of a comprehensive, up-to-date survey that systematically covers the entire RBTG workflow. Previous studies have often focused only on specific perspectives, such as UML-based modeling or NLP-based techniques, missing a comprehensive view across the full spectrum of requirements types, generation strategies, and evaluation methods. This paper aims to address this gap.

This paper presents a comprehensive survey of RBTG, covering 267 papers published up to 2024. It examines the entire RBTG process, including: (1) a summary and analysis of the selected 267 papers; (2) a classification of the RBTG input requirements; (3) a description, classification, and synthesis of RBTG approaches; (4) a description of the types of test cases produced; (5) a summary of RBTG tools; (6) an analysis of RBTG evaluation methods; (7) an overview of the RBTG application domains; and (8) a discussion of open research challenges and potential future directions for RBTG. To the best of our knowledge, this is the first large-scale and comprehensive survey on RBTG.

The rest of this paper is structured as follows. Section 2 discusses the background and previous surveys. Section 3 explains the methodology adopted for our literature review. Section 4 explores the distribution of RBTG studies. Section 5 examines the types of RBTG input requirements. Section 6 reviews the approaches used for RBTG. Section 7 categorizes the types of test cases generated. Section 8 presents the tools developed for RBTG. Section 9 provides a detailed analysis of RBTG evaluation methods. Section 10 examines the application domains covered in RBTG studies. Section 11 identifies key RBTG challenges, and proposes future research directions. Finally, Section 12 concludes the paper.

2 BACKGROUND AND RELATED WORK

This section introduces the definition and classification of requirements, and explains their importance for defining system objectives. We also outline the types of tests, explaining how they help to ensure that the system performs as intended. Previous survey studies on test-case generation are also reviewed, summarizing their key outcomes and methodologies.

2.1 Requirements

Software requirements serve as the blueprint for the development of a software application, presenting the core goals and objectives that guide the development team [294]. The requirements specify the intended features and functionality, covering aspects such as product perspective, user characteristics, operating environment, design constraints, user documentation, assumptions, and dependencies [46]. The quality of the requirements can significantly impact the overall quality of both the software testing and development [195].

In typical industrial environments, users may communicate the needs of a software application by stating the requirements in NL, using formats like requirements specifications and user stories. Pure NL is easy to understand, making it accessible to both developers and testers. However, requirements described in NL may contain ambiguities or inconsistencies, and may be incomplete. This will impact the quality of software testing. To minimize such problems, requirements can be translated into more structured and formal representations, improving clarity and consistency during the architectural design phase, where requirements are analyzed.

Fischbach et al. [97] noted that requirements could be specified into three different classes: informal, semi-formal, and formal techniques. Informal descriptions do not follow a particular syntax or semantic structure; semi-formal requirements are expressed in a predefined and fixed syntax that is not formally defined; and formal notations are based on mathematical notation, and have precise syntax and semantics. However, it is difficult to fully represent the diverse forms of requirements within RBTG using just this classification. Furthermore, these broad categories lack the precision needed to align with the trends in modern requirements engineering.

Bruel et al. [46] categorized requirements into five groups based on their primary specification style: NL, semi-formal, automata/graphs, mathematical, and seamless (programming language-based). Semi-formal specifications structure the expression of requirements by establishing a defined language that is more precise than NL, yet does not rely on mathematical formalisms or programming-language constructs. Examples of this kind of requirements include: Reqtify [282], KAOS [296], and *systems modeling language* (SysML) [60]. Automata/graphs-based requirements are modeled through state machines [258], transition systems [52], or graph-theoretical representations, such as UML *activity diagrams* (ADs) [143] and state diagrams [66]. Mathematical notation relies on formalisms based on set theory [141, 234] or universal algebra [79]. Seamless requirements are closely integrated with implementation artifacts through programming language constructs, enabling direct traceability to the code [196]. In the context of an RBTG review study, the generation of test cases is independent of source code: The “seamless” category, therefore, is problematic, as studies involving source code are typically excluded during the literature search. Nevertheless, within the field of test-case generation methods, model-based approaches are extensively employed, with a wide variety of model types. The “automata/graphs” category outlined in this paper does not comprehensively cover all model types, nor does it account for scenarios involving the combined use of multiple requirements types. To address these limitations, this paper proposes a revised classification framework that builds on the existing categories while aligning more closely with the characteristics of RBTG methods.

Table 1 summarizes five categories of requirements’ representations used in RBTG: The categories are based on the main expressions observed in the surveyed studies. In practice, requirements artifacts often span multiple categories. The classification reflects the primary representation used during test-case generation, not the semantics or content of the requirements. For example, although use cases may involve both textual and diagrammatic

Table 1. Requirements Classification for RBTG

Category	Core Characteristics	Typical Formats
Natural language	Unstructured or simplified textual descriptions in human language. Often used for communication with non-technical stakeholders.	Free form natural language [25], textual use-case description [309], user stories with acceptance criteria [98]
Semi-formal	Combines natural language with structured elements such as templates, tables, or simplified graphical representations	Ontology-based requirements specification [28, 293], SysML [60, 324]
Model-based	Abstract representations of software behavior or functionality, using graphical models, automata, or trees.	UML diagrams [173, 233], Automata [129, 172]
Formal mathematical	Rigorous mathematical or logical notations, used for precise requirements' definitions and formal verification.	<i>Structured Object-Oriented Formal Language</i> (SOFL) [50, 234], Z formal specification [124, 141]
Hybrid	Uses multiple representation categories, leveraging their respective strengths.	Textual specifications combined with a use-case diagram, textual use cases, HTML screen mockups, and glossaries [70]

representations, we classify them based on their main usage in each study, either as an NL artifact or as part of a model-based approach. The five categories of requirements are:

- *Natural language (NL) requirements*: NL requirements are written in English or other natural languages, either in free text or using controlled language. Common forms include plain text descriptions, textual use cases, and user stories. Such representations are widely used in practice, due to their readability and accessibility to both technical and non-technical stakeholders. *Textual use cases* describe interactions between an actor and the system in a structured narrative, typically including elements such as preconditions, the main flow, and alternative flows [139]. Although textual use cases originate from the UML standard, they are often expressed in free text, and are thus considered NL artifacts in automated test generation. *User stories*, widely adopted in agile development [67], follow a simple template (e.g., “As a (role), I want (function) so that (business value)”) and are often accompanied by NL acceptance criteria [302].
- *Semi-formal requirements*: Combining NL with some formal structure (such as tables in text documents, or diagrams from modeling tools), these approaches provide enhanced clarity and accessibility for users with minimal technical training. However, they cannot be directly processed using formal verification methods. SysML, an extension of UML dedicated to systems engineering, expresses requirements in textual specifications, including non-functional requirements [46]. Ontology-based specifications, which support semantic interpretation and machine reasoning, encode user requirements in a machine-processable format, and allow requirements to be classified as actions, qualities, or softgoals [293]. *Expressive decision tables* (EDTs), a tabular notation based on regular expressions, organize NL requirements into structured rows that represent system reactions [299]. Although not fully formal, semi-formal requirements often act as an effective bridge between NL and formal models in automated test-case generation workflows.
- *Model-based requirements*: These requirements describe system behavior or functionality using abstract models, such as UML diagrams, state machines, or automata. Visual representations help bridge communication between technical and non-technical stakeholders without requiring a deep understanding of formal methods. It is also possible to extend the UML models with constraints written in the *object constraint language* (OCL) [9], specifying invariants, preconditions, or postconditions. Although OCL is a formal language, it is typically only used to complement UML [10]. We classify OCL-enhanced UML models under the model-based category, reflecting their role in enriching visual representations for test-case generation.
- *Formal mathematical requirements*: These requirements are specified using rigorous mathematical notations (such as those based on set theory or algebra), allowing precise definitions suitable for formal

verification. However, these are typically only accessible to technical audiences with a strong foundation in mathematics, making them less appropriate for non-technical stakeholders. Z [141], for example, is one of the most widely used formal specification languages: Its syntax is based on mathematical notation, and its semantics are grounded in classical mathematics.

- *Hybrid requirements*: Hybrid requirements involve two or more categories, combining the strengths of each to provide a more comprehensive and detailed representation. This approach ensures both breadth and accuracy in capturing requirements information, making it adaptable to diverse project needs. For instance, a combined textual specification may include a use-case diagram, textual use cases, HTML screen mockups, and a glossary of key terms [70].

2.2 Types of Tests

There are different types of tests, such as *abstract test cases* (ATCs), *concrete test cases* (CTCs), and *test scenarios* (TSs). A test case specifies a particular condition to be evaluated, and includes a set of test inputs, execution conditions, and expected results developed to verify a specific aspect of system behavior. A test case may consist of a title, precondition, test steps, and expected result. The definitions used in this paper conform to standard terminology [34].

- *Abstract test cases* are high-level test cases without concrete (implementation level) values for input data and expected results. Logical operators are used; instances of the actual values are not yet defined and/or available (e.g., given an abstract value “ n is a positive integer”). It is typically in a human-readable description.
- *Concrete test cases* are low-level test cases with concrete (implementation level) values for input data and expected results. Logical operators from high-level test cases are replaced by actual values that correspond to the objectives of the logical operators (e.g., a concrete value for n is 100).
- *Test scenarios* provide a high-level description of a functionality or business process to be tested. They specify a sequence of actions or events that reflect the intended interaction flow and behavioral aspects of the *system under test* (SUT), helping to determine what should be tested and why. Unlike abstract test cases (which emphasize logical conditions) and concrete test cases (which specify exact input and output values), test scenarios are more narrative and process-focused: They act as a bridge between system requirements and executable test cases, and have been treated as ATCs in several studies [232].

In software testing, CTCs can either be executed manually (for example, by test engineers) or automatically, using testing frameworks such as the Selenium framework [302] or JUnit [295]. However, some of the reviewed studies did not actually generate test cases: This paper considered those test artifacts lacking concrete input values or expected results as ATCs.

2.3 Previous Survey Studies

Previous work has explored the challenge of generating test cases directly from software requirements, addressing the problem using a wide range of methods and technologies. This paper provides a comprehensive review of these efforts, synthesizing the existing body of knowledge and categorizing the diverse strategies employed in RBTG. Table 2 provides a comparison of prior survey studies against this paper.

Boukhelif et al. [42] presented an overview of diverse NLP techniques used to address various linguistic challenges. They provided an analysis of the distribution of software testing contexts across the reviewed research papers, noting a wide range of testing methodologies and objectives. They noted the critical importance of both machine- and deep-learning techniques in NLP for enhancing the capabilities and performance of language-centric applications. However, their study did not specify which techniques were best suited for particular testing

Table 2. Comparison of Previous Survey Studies

Year	Studies	# Studies	Time Span	Survey Scope	SLR	RQ1	RQ2	RQ3	RQ4	RQ5	RQ6	RQ7	RQ8
2011	Escalona et al. [88]	24	1988-2009	Test-case generation from functional requirements described in informal language	✗	✗	✓	✓	✗	✓	✗	✗	✗
2017	Ahsan et al. [5]	16	2005-2016	NLP-based test-case generation from natural language requirements	✓	✓	✗	✓	✗	✓	✗	✗	✗
2018	Gupta et al. [110]	79	1988-2017	Regression test-case generation for web applications	✗	✗	✗	✓	✗	✓	✗	✗	✗
2019	Ahmad et al. [4]	41	2004-2016	Model-based testing using UML activity diagrams	✗	✓	✗	✓	✗	✓	✗	✗	✗
2021	Mustafa et al. [195]	29	2000-2018	Functional test generation from requirements expressed in various notations	✓	✓	✓	✓	✗	✓	✗	✗	✓
2021	Jin and Lano [140]	62	1999-2019	Test-case generation from UML-based software models	✓	✓	✓	✗	✗	✗	✗	✗	✓
2022	Farooq and Tahreem [91]	30	2010-2021	Test-case generation from functional requirements represented in UML diagrams	✓	✗	✓	✓	✗	✗	✗	✗	✓
2023	Potuzak and Lipka [219]	67	2000-2022	Combined generation of test cases and corresponding test data	✗	✓	✓	✓	✗	✓	✓	✗	✗
2023	Verma et al. [300]	125	2011-2021	Tools and techniques for automated test-case generation	✓	✓	✗	✓	✗	✓	✗	✗	✗
2024	Boukhelif et al. [42]	24	2013-2023	NLP-based test-case generation from NL requirements	✓	✓	✗	✓	✗	✓	✗	✓	✓
2024	Wang et al. [307]	102	2020-2023	Use of LLMs to support testing across the software development lifecycle	✗	✓	✓	✗	✗	✗	✗	✗	✓
2025	This survey	267	1994-2024	Comprehensive analysis of the entire RBTG workflow, covering requirement types, generation approaches, test case types, supporting tools, evaluation methods, and application domains.	✓	✓	✓	✓	✓	✓	✓	✓	✓

methodologies. Nor did they explain how these integrated NLP and machine/deep learning approaches were applied to advance software testing practices.

Mustafa et al. [195] focused on functional testing, presenting input requirements and the approaches used for requirements transformation. However, they did not establish a clear connection between the requirements types and the corresponding methods, leaving a gap in how these elements are aligned. Similarly, Farooq and Tahreem [91] explored the generation of functional test cases, with particular attention on the use of UML diagrams: They identified several gaps and challenges in RBTG, offering insights into areas requiring further investigation.

Ahmad et al. [4] provided a comprehensive overview of model-based testing using UML ADs. They noted that only a limited number of their surveyed approaches had been validated against realistic, industrial case studies, with most focusing on very restricted application domains. They also reported that UML ADs were not commonly employed for non-functional testing, and that there was a lack of holistic or comprehensive approaches for model-based testing using UML ADs.

As shown in Table 2, although several surveys have explored RBTG, most of them concentrate only on specific aspects, such as UML-based modeling or NLP-based techniques. These surveys often limit their scope to particular requirements types or generation strategies, lacking a comprehensive overview of the end-to-end RBTG workflow. Moreover, they seldom offer a systematic synthesis across relevant topics, such as requirements representation, generation approaches, intermediate model transformation, test-case types, and evaluation. In recent years, there has been significant progress both in terms of technical innovation and practical adoption. The emergence of

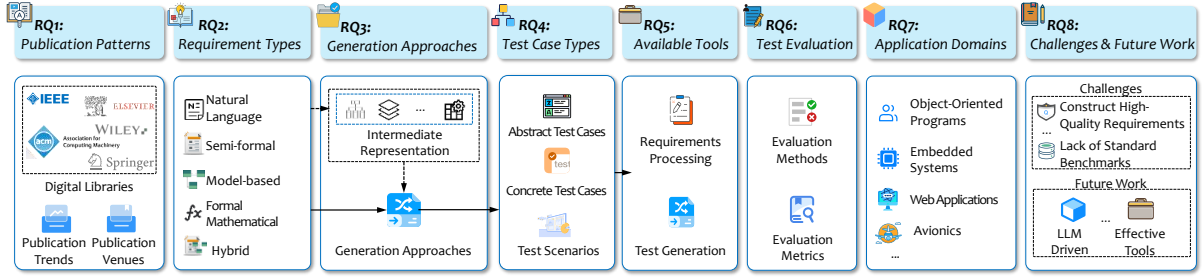


Fig. 1. Structure of this survey paper.

LLM-driven generation, knowledge-assisted reasoning, and graph-based modeling has fundamentally changed how requirements are processed, and how test cases are derived [104, 169]. Increasing industrial interest in automating quality assurance has raised new practical challenges and evaluation dimensions that previous surveys have not adequately covered.

This survey was motivated by the need to fill these gaps through a comprehensive and fine-grained synthesis of the entire RBTG workflow. It draws from recent advances across both academic and industrial contexts. As shown in Table 2, compared with previous studies, this paper systematically classifies studies across a full spectrum of dimensions – including requirements types, generation approaches, test-case forms, tools, evaluation techniques, application domains, challenges, and future work directions. To facilitate deeper understanding and support future research, we provide a structured taxonomy and a carefully selected dataset of 267 studies published between 1994 and 2024. The key offerings of this review include:

- A comprehensive review of the full RBTG workflow, from requirements to test evaluation.
- A refined classification of requirements representations into five major categories.
- A detailed taxonomy of test-generation approaches and intermediate models.
- An analysis of test-case formats, tools, evaluation techniques, and industrial domains.
- A discussion of open challenges and future research directions for RBTG.
- A reproducible dataset and classification index to support future studies.

3 RESEARCH METHODOLOGY

We conducted a comprehensive investigation of RBTG using the SLR methodology proposed by Kitchenham et al. [161]. As part of the SLR, we adopted the snowballing technique, which starts with an initial set of papers and expands by examining their references to identify additional relevant studies [311]. The research methodology consisted of several key steps: defining research questions; identifying relevant studies; applying the snowballing technique to expand the search; implementing exclusion criteria; and systematically extracting the required data. This structured approach ensures a rigorous and comprehensive analysis of the literature.

3.1 Research Questions

The research objectives for this survey were based on the complete RBTG process (Figure 1), and involved analyzing and categorizing information at each stage. The study also aimed to identify research challenges and areas requiring future work. To achieve these objectives, we defined the following research questions (RQs):

- *RQ1: What are the publication patterns of RBTG studies?*
- *RQ2: What types of requirements have been used in RBTG?*
- *RQ3: What are the approaches that support RBTG?*

Table 3. Inclusion and Exclusion criteria

Inclusion criteria	
IC1	Studies propose or improve an approach, technique, or tool/framework explicitly aimed at automated test-case generation.
IC2	Studies use input artifacts derived from requirements, structured specifications, or architectural models, provided that these artifacts are explicitly used to represent or formalize software requirements, rather than to describe system architecture or software design.
Exclusion criteria	
EC1	Studies without a full text.
EC2	Studies not written in English.
EC3	Books, book chapters, or technical reports (most of which may have been published as articles)
EC4	Masters or Ph.D. theses.
EC5	Survey studies.
EC6	Studies less than five pages in length.

- *RQ4: What types of test-case representations have been explored in RBTG?*
- *RQ5: What tools have been developed to support RBTG?*
- *RQ6: How have RBTG approaches been evaluated?*
- *RQ7: In what domains and industrial applications has RBTG been applied?*
- *RQ8: What are the remaining challenges and future work for RBTG?*

3.2 Literature Search and Selection

The search strategy was optimized to identify relevant information, ensuring a thorough and effective investigation of the research questions. This process involved four steps: selecting appropriate digital libraries; identifying additional search sources; determining relevant search strings; and performing the snowballing technique. Following previous review studies [4, 140, 195, 300], we selected five online source repositories to perform the SLR:

- ACM Digital Library
- Elsevier Science Direct
- IEEE Xplore Digital Library
- Springer Online Library
- Wiley Online Library

These selected repositories provide a wide range of scholarly resources highly relevant to the research topic.

After identifying the relevant literature databases, a search string was applied to each. The search strings were developed iteratively, being refined based on preliminary results. For each repository, customized search strings were constructed to optimize the advanced search process. The overall strategy for constructing the strings was: (1) The **Title** should include *test case OR test suite OR test scenario*, (2) The **Title** should also include *generate OR create OR requirement*, (3) The **Abstract** should include *requirement OR specification OR user stories*. These three criteria were combined using the **AND** operator to construct the search strings.

To ensure the relevance of the included studies, we applied two clear inclusion criteria, as listed in Table 3. According to the V-Model [17], software development activities span multiple phases, including requirements analysis, architectural design, subsystem design, detailed design, and implementation. In this survey, we consider artifacts produced during the requirements analysis and architectural design phases as requirements-level artifacts, since they are intended to specify what the system should do. Our focus is specifically on approaches that generate test cases from requirements-level artifacts. Requirements representations are diverse [33]. Model-based test-generation methods were included only if their underlying models were explicitly derived from requirements,

Table 4. Overview of Search Result and Study Selection

Source	Initial Search Results	After De-duplication & Title/Abstract Review	Post-Snowballing	Final After Exclusions
ACM	76	30	49	31
ELSEVIER	46	12	19	14
IEEE	314	127	182	113
Springer	34	6	57	51
Wiley	23	4	6	3
Other	–	–	55	55
Total	493	179	368	267

without implementation details. For example, UML models (such as use-case diagrams, activity diagrams, and state machines) were included when used to visually and intuitively represent functional requirements [216]. *Finite state machines* (FSMs) [174] were included when explicitly used to model high-level system behavior based on requirements. B [44] formal specifications were included when the models were annotated with requirements' identifiers, or were explicitly described as formalizations of requirements specifications. *Business process models* (BPMs) [151] were included if they were used to capture business rules or stakeholder workflows derived from requirements.

The search covered all studies published up to December 31, 2024, with the earliest identified work dating back to 1994. Table 4 shows the initial search results: These were then de-duplicated and screened based on titles and abstracts, as reported in the third column. To further enhance our literature search, we employed Snowballing techniques [311] to enhance the literature search: The references of selected papers were reviewed, and additional relevant studies were identified (and downloaded from Google Scholar). This iterative process was continued until no new relevant papers were identified. The final results are listed in the "Post-Snowballing" column. The final set of relevant papers was determined by applying the exclusion criteria in Table 3. As part of the exclusion process, books were excluded because they often consolidate previously published work. The approaches discussed in books typically originate from peer-reviewed conference or journal papers. Some classic textbooks, such as by Ammann and Offutt [17], Binder [33], and Marick [184], discuss the derivation of test cases from requirements: Although not part of our systematic review dataset, these works have strongly influenced both academic research and practical testing approaches.

Table 4 also shows the distribution of selected studies across the various digital libraries: 45% from IEEE Xplore; 18% from SpringerLink; 12% from ACM Digital Library; 5% from Elsevier; 1% from Wiley; and 19% from other databases (such as ProQuest, Academia.edu, and SCIRP.org), identified through snowballing techniques. This diverse distribution highlights the collaborative and extensive nature of academic research in the field.

Although it may not have been possible to find every relevant paper through our search, we are confident that we have included most of the significant publications. This survey provides a comprehensive overview of the current trends and state-of-the-art in RBTG.

3.3 Data Extraction

The 267 selected studies were thoroughly reviewed and inspected, with data extracted according to the research questions: Table 5 summarizes the extracted data. To minimize the risk of missing information and reduce potential errors, we performed reviews both during the data-extraction process, and after its completion. Appendix A lists all reviewed studies along with their key classification attributes. The full dataset with detailed attributes is available in the online repository (<https://github.com/aimeenorwind/SLR-on-RBTG> and <https://zenodo.org/records/17283189>).

Table 5. Data Collection for Research Questions

RQs	Type of Data Extracted
RQ1	Core information for each paper (publication year, publisher, type of paper).
RQ2	Type of requirements, descriptions, classifications.
RQ3	Type of techniques, degree of automation, intermediate representations, descriptions.
RQ4	Type of generated output artifacts and test execution environment.
RQ5	Tool name, function, language, types, and descriptions.
RQ6	Evaluation methods, metrics.
RQ7	Application domains, programs in case studies.
RQ8	Details of remaining challenges, potential future work.

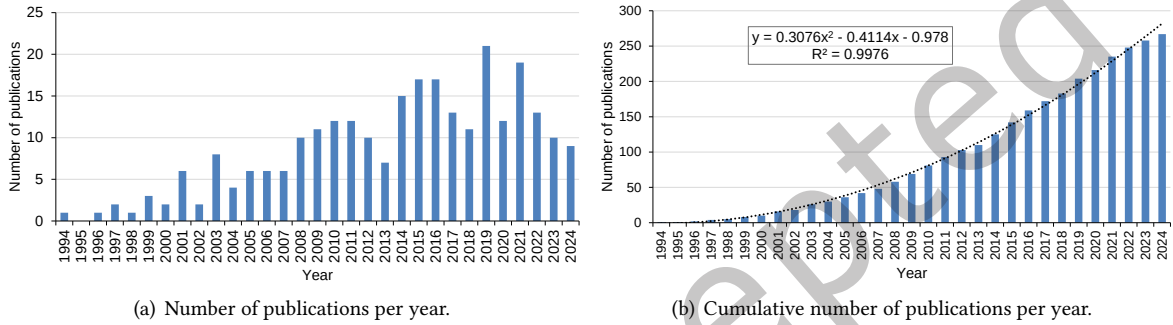


Fig. 2. Overview of the surveyed 267 RBTG studies' publication.

4 ANSWER TO RQ1: WHAT ARE THE PUBLICATION PATTERNS OF RBTG STUDIES?

267 studies were identified for critical evaluation regarding RBTG, with the extracted data summarized in Table 5. Core information was extracted for each study, including publication year, publisher, and type of paper. This section provides an overview of the surveyed studies, focusing on their publication trends and venues.

4.1 Publication Trends

Figure 2 provides a comprehensive overview of the yearly RBTG publication trends of the 267 primary studies, up to the end of 2024: Figure 2(a) shows the number of publications each year; and Figure 2(b) presents the cumulative number of publications over time. The earliest reviewed publication was from 1994. This field has attracted significant attention and interest from researchers since 2001. Since 2008, there have been at least ten published studies each year, with notable peaks of over 20 papers in 2019 and 2021. Figure 2(b) shows the polynomial trendline, with a high determination coefficient ($R^2 = 0.9976$). The overall trend indicates a sustained interest in RBTG. Although publication counts have slightly declined over the last three years, the annual output remains steady (around ten papers): This suggests that the field has reached a stable plateau, which we interpret as a sign of growing maturity and consolidation. The continued research activity indicates that RBTG is both well-established, and an active area of investigation.

4.2 Distribution of Publication Venues

Figure 3 shows the distribution of publications in our dataset, with Figure 3(a) presenting the overall venue distribution, and Figure 3(b) illustrating the distribution per year. As shown in Figure 3(a), 162 (61%) of the 267 primary studies were published in conference proceedings, 89 (33%) in journals, and 16 (6%) in workshops.

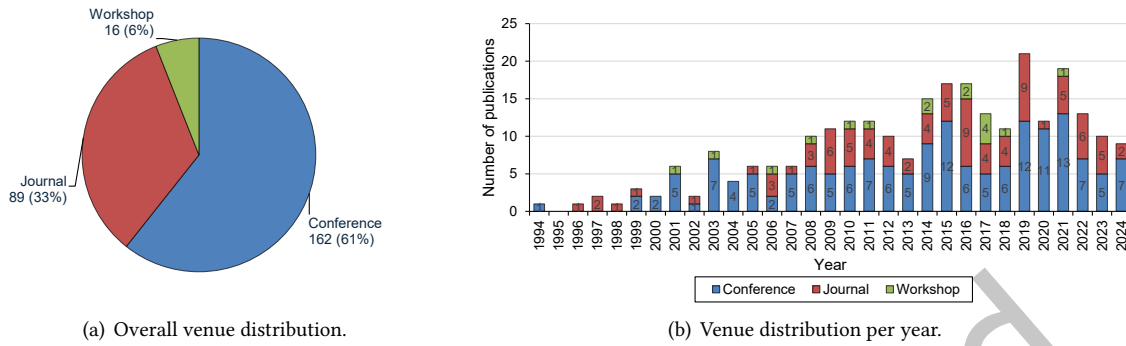


Fig. 3. Venue distribution for RBTG studies.

Figure 3(b) further reveals that conference publications have consistently outnumbered journal articles each year. This suggests that RBTG research has primarily been disseminated through conferences, possibly due to their shorter review cycles and greater responsiveness to emerging trends. Workshop papers only represent a minor proportion, and seem to have no clear pattern over time.

To determine whether or not this conference-dominant trend is unique to RBTG, we examined related surveys in other areas. A survey on software testing automation using UML activity diagrams [4] identified that 28 of the 41 studies (68%) were published in conference proceedings. Similarly, a recent survey on software testing with LLMs [307] reported that 51 of the 102 included studies were from conferences, compared to only eight journal articles, and 43 preprints (e.g., from arXiv). These patterns indicate that a preference for conference venues is common in fast-moving areas of software-testing research. Our findings align with this broader trend.

5 ANSWER TO RQ2: WHAT TYPES OF REQUIREMENTS HAVE BEEN USED IN RBTG?

Requirements play a crucial role in RBTG, as their content and representations directly influence generation strategies and approaches, ultimately affecting the quality of the resulting test cases. In real-world projects, requirements are often written in NL, which is easy to understand, but inherently ambiguous. To address this, NL requirements are often transformed into formal or semi-formal specifications. This section categorizes the collected papers based on requirements types, identifying the most commonly used representations, and analyzing their evolution over time.

As explained in Section 2.1, this paper uses five requirements categories: NL; semi-formal; model-based; formal mathematical; and hybrid. Figure 4(a) presents the distribution of the types of requirements, showing that 53% (141 papers) focused on model-based representations; 28% (75 papers) involved NL; and 19% (51 papers) used other representations. Figure 4(b) shows the requirements' types over time: Although model-based requirements have been extensively studied over the years, NL types began to attract attention in 2011, and have surpassed model-based approaches since 2019.

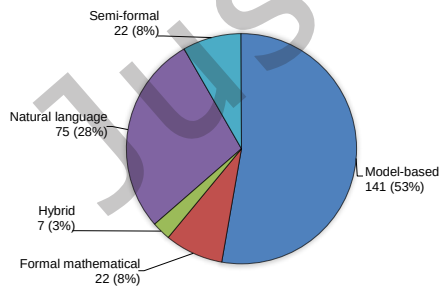
5.1 Natural Language Requirements

Users usually communicate the needs of a software application in NL. This has meant that generating test cases directly from NL requirements has become a key objective for researchers. Among the surveyed studies, the NL-based requirements can be categorized into three types: unstructured; templated; and *controlled natural language* (CNL) requirements. Unstructured NL refers to free-text expressions with no constraints on the requirements'

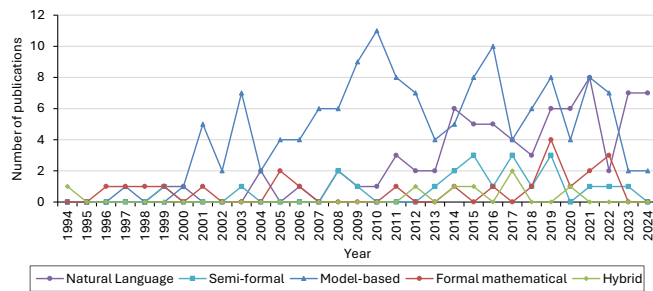
formulation: This is the most commonly used approach in practice. Templated requirements include use-case descriptions, user stories (which are commonly employed in agile development), and scenarios. A use case is a sequence of interactions between a system and a user (understood as an instance of an actor) [231]. A scenario depicts a portion of the application’s behavior at a specific time and place, capturing the situational context [246]. Use cases can be seen as distinct models of scenarios. CNL requirements use restricted grammar and predefined vocabulary to reduce textual complexity and ambiguity.

Figure 5 presents the distribution of different NL types used in the surveyed studies. Unstructured natural language [27, 187] represents the main form of input requirements, used by 27 studies (36%). The use-case description [181, 270] is also widely adopted, appearing in 24 studies (32%). Other notable NL types include user stories (seven papers), scenarios (four papers), CNL, like SysReq (six papers), and ITLingo RSL (three papers). SysReq is a CNL for system requirements, defined by a phrase-structure context-free grammar and a lexicon encompassing the application domain vocabulary [56, 265]. ITLingo RSL (Requirements Specification Language) is a controlled and integrated NL, designed to assist in the systematic, rigorous, and consistent production of requirements specifications [191, 211].

Unstructured NL requirements are the most common way to capture customer needs. In traditional testing processes, testers design test cases based on such requirements, ... but only after thorough reviews to identify ambiguities and reduce mistakes. The automatic generation of test cases directly from NL requirements needs for them to be complete, unambiguous, and clearly expressed. However, NL naturally suffers from ambiguity, inconsistency, and incompleteness. Although NLP and ML techniques have been combined to directly transform requirements into test cases [27, 291], usually an intermediary step is included — converting the requirements into structured models before generating the test cases. These approaches are explored in Section 6. Textual use cases follow standardized templates (e.g., Cockburn’s format [15]), and capture essential behavioral information through the main, alternative, and exception flows [139]. Many organizations use use cases to validate and refine business processes with clients, thereby improving requirement clarity and reducing ambiguity. Accordingly, textual use cases have become a popular and effective form of input for automated test-case generation. User stories, popular in agile development [67], follow a simple template (e.g., “As a [role], I want [function] so that [value]”). In practice, effective communication among team members often enriches these stories with additional context, and can help to resolve ambiguities. However, when used as input for automated test-case generation, their inherent lack of detail and potential ambiguity can result in incomplete or incorrect test cases. CNLs restrict grammar and vocabulary to reduce ambiguity and better support automation. However, writing CNL-based



(a) Overall requirements types.



(b) Requirements types per year.

Fig. 4. Types of requirements.

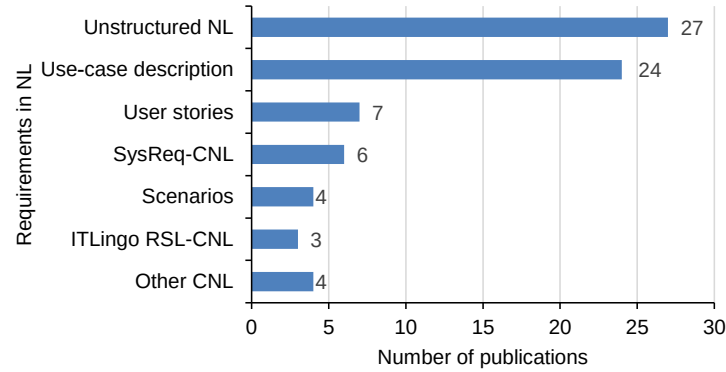


Fig. 5. Distribution of types of natural language requirements.

requirements often requires specialized training, and such formats are usually tailored to specific systems, limiting their general applicability.

5.2 Semi-formal Requirements

Semi-formal requirements combine NL with structured templates, tables, or simplified graphical representations. Examples include ontology-based specifications [28], SysML [60], and courteous logic representation [261]. Within this category, the requirements ontology (five papers) and SysML (three papers) were the most frequently used.

An ontology represents concepts and relationships within a domain [28, 293], with inference rules describing strategies for deriving test cases [284]. As a semi-formal specification, it can represent both concrete and abstract concepts, and is particularly useful in the early stages of requirements engineering [28]: It provides a semantic foundation over which requirements can be precisely defined, linked, and interpreted, typically in the form of structured sentences. Ontology-based representations are frequently used in safety-critical domains, such as industrial cyber-physical systems [269]. SysML models the SUT along with its interactive external environment [324], providing diagrams that enable the textual depiction of requirements (including non-functional requirements) [1, 60]. It reuses seven of UML's fourteen diagrams, and introduces two new ones, the requirements diagram and the parametric diagram [8]. Compared to UML, SysML has more flexible and expressive semantics, reducing UML's inherent software bias. The built-in requirements diagram facilitates the representation and traceability of NL requirements across the development lifecycle, making SysML particularly well-suited for modeling complex systems. However, SysML modeling is a human-driven activity that requires substantial domain expertise, making it prone to errors, and limiting its widespread adoption. Semantic specifications [193] that are formulated in the controlled English *semantic application design language* (SADL) map directly into the *web ontology language* (OWL) [71], ensuring readability and comprehension for both humans and machines. Courteous logic-based requirements representations [260, 262], *expressive decision tables* (EDTs) [298, 299], and *domain specific languages* (DSLs) [49, 208] have also been used, with each appearing in two studies.

Despite their potential to streamline test generation, only 22 studies (8%) used semi-formal requirements: These were mostly applied to embedded systems (six papers) and safety-critical domains, like avionics, automotive and industrial control systems (eight papers), while the remaining eight involved unspecified domains (six papers) or individual cases in telecommunications and numerical programs.

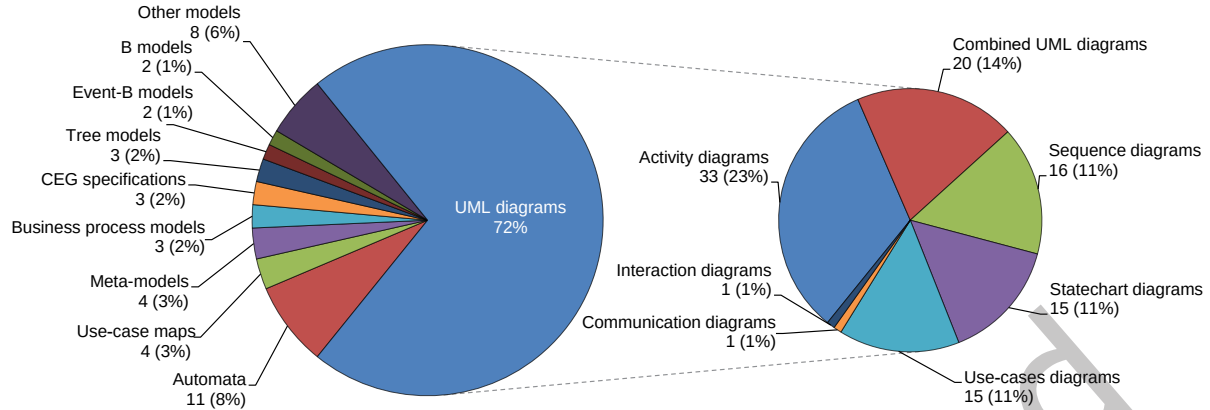


Fig. 6. Distribution of model-based types.

5.3 Model-based Requirements

Requirements models provide an intuitive and visual representation of system requirements. This section examines the models used in the surveyed studies. Figure 6 presents the distribution of requirements models across the studies. The majority of papers (72%) used UML diagrams, highlighting the significant focus on UML in the literature. Automata models, such as *finite state machines* (FSMs) and *extended finite state machines* (EFSMs), were used in 8% of the papers. Other models included: use-case maps (3%), meta-models (3%), business process models (2%), specifications (2%), tree models (2%), B models (1%), and event-B models (1%). There were also several other uncommon models, each appearing only once.

5.3.1 UML Diagrams. The UML has been widely used as a system-level specification language. It is a visual modeling language that comprises various diagrams, each representing different aspects of a system's structure and behavior. Figure 6 shows that UML ADs [90, 173, 190] (23%) are the most commonly used model: As a behavioral model, ADs provide detailed implementation information, making them well-suited for test derivation. However, their effectiveness decreases when dealing with large and complex systems. Combined UML diagrams (14%), sequence diagrams (11%), statechart diagrams (11%), and use-case diagrams (11%) have also been widely used. Combined UML diagrams combine the strengths of multiple models, enabling the generation of more reliable test cases — however, they also incur a time overhead [216]. Examples of combined UML diagrams include activity and statechart diagrams [233, 281], sequence and use-case diagrams [244, 280], and sequence and statechart diagrams [157, 271]. Both sequence [154, 236] and statechart diagrams [118, 220] are also behavioral models, capturing system interactions and state transitions to support test generation. In contrast, use-case diagrams [120, 202] describe system functionality from a user's perspective, providing a high-level view of interactions without detailing execution flow or state changes. Communication diagrams [240] convey both the messages exchanged between objects and their execution order, providing insight into both data flow and control flow. Similarly, interaction diagrams [198] visually depict the sequence of messages exchanged among collaborating objects across different scenarios of a use case.

5.3.2 Automata. Automata [129, 172] are abstract mathematical machines used to model the behavior of systems, usually through states and transitions between them. Four types of automata were used in the surveyed papers: EFSM [87, 222]; FSM [174, 224]; safety SysML state machine (S2MSM) [305]; and Mealy machines [217]. EFSM

Table 6. Automata Models

Automata Model	Tuple Structure	Mathematical Definition	Description
Finite state machine (FSM)	5-tuple	$M = (S, s_0, I, O, T)$	S is nonempty finite set of states, $s_0 \in S$ is the initial state, I and O denote nonempty finite sets of inputs or outputs respectively, T is the finite set of transitions
Extended finite state machine (EFSM)	6-tuple	$M = (S, s_0, V, I, O, T)$	V is the finite set of (internal) context variables
Safety sysML state machine (S2MSM)	7-tuple	$M = (S, T, E, G, P, A, V)$	E is a finite set of events, G is a finite set of guards, P is a finite set of the priority, A is a finite set of actions, V is a finite set of variables
Mealy machine	6-tuple	$M = (S, s_0, I, O, \delta, \lambda)$	$\delta : S \times I \rightarrow S$ and $\lambda : S \times I \rightarrow O$ are total functions which denote the deterministic transition and output relations

was the most commonly used, appearing in five papers, followed by FSM, which appeared in four. Both S2MSM and Mealy machines were found in one paper each. Table 6 compares the different automata models.

An FSM is a finite labeled transition system where each transition is associated with an input/output pair, consisting of an input action and a corresponding output action [133, 174]. An EFSM system model is an extension of FSMs that includes additional features, like variables and conditions on transitions [65, 297]. S2MSM has additional verification and refinement procedures to check the safety of the generated model using a formal verification tool: S2MSM can both precisely model functional requirements and capture safety concerns, in a unified model [305]. A Mealy machine is an FSM where the outputs are determined by both the current state and the current input. It enables compact modeling of reactive systems by associating outputs directly with transitions, allowing for precise representation of input–output behaviors [217]. Due to their logical structure, these automata are well-suited for modeling the structural and behavioral aspects of software applications, particularly in testing. However, their effectiveness is limited when handling more complex data dependencies and complex systems.

5.3.3 Use-Case Map. A use-case map (UCM) is a visual notation for modeling scenarios/workflows derived from requirements [150]. It is similar to a UML AD, but imposes fewer constraints, emphasizing the causal relationships between workflow steps without specifying detailed message exchanges or data [41]. Four of the surveyed papers used UCMs as input [18, 41, 43, 150].

5.3.4 Meta-Model. Meta-models standardize the information used in test-case generation: They formally specify a concept by defining its attributes and its relationships with other concepts [114]. Gutiérrez et al. [114, 115] used a functional requirements meta-model as the basis for testing: The core concept of this meta-model was the functional requirements' element, defined by (1) *step* elements representing discrete actions (e.g., data requests or computations); and (2) execution-order elements specifying their sequence. Chen et al. [61] proposed a test-requirement meta-model that defined the core concepts along with their relationships and dependencies. Granda et al. [106] applied transformation rules to convert meta-models into test cases.

5.3.5 Other Types of Models. In addition to the models already discussed, several others were also used. *Business process models* (BPMs) [113, 151, 315] represent a series of related activities producing a specific service or product. A *Cause-effect graph* (CEG) [142, 162, 214] is a black-box testing technique that describes relationships between events using logical relations (and events are labeled as either causes or effects). A *Behavior tree* (BT) model [171, 235] is a directed tree with various node types and branching, aiding in modeling complex behaviors. Additional models include: Event-B models [183, 252]; B models [44, 167]; scenario-based state/event trees [290];

EDTs [3]; *variable relation models* (VRMs) [309, 323]; and *multiple condition-control flow graphs* (MCCFGs) [273]. These models all provide different approaches for representing requirements.

5.3.6 Discussion. Model-based requirements aim to provide a structured, and often visual, representation of system functionality, interactions, or domain concepts. However, their actual use varies significantly across studies and practical settings, especially for complex software systems.

UML is the leading standard for system modeling in both industry and academia. Its relatively low learning curve and broad tool support contribute to its popularity. Despite often being incomplete or ambiguous, UML diagrams provide valuable guidance for test design and serve as a basis for automated test-case generation. UML supports multiple perspectives of a system, encompassing both structural and behavioral aspects. However, in practice, most studies use only a subset of UML's full specification: Activity diagrams are the most commonly adopted, due to their intuitive representation of workflows, business processes, and interaction scenarios. For large-scale systems, however, these diagrams can become overly complex and difficult to maintain. Combined UML models have been shown to detect more faults than individual UML diagrams [155], and are therefore commonly adopted in practice. However, the integration of multiple UML diagrams requires substantial effort and domain knowledge. Ensuring consistency and synchronization across models can be demanding, and the added overheads may outweigh the benefits, especially in fast-paced or resource-constrained development environments.

Automata-based models are the second most commonly used (after UML diagrams), with FSMs and EFSMs being the most frequently applied. FSMs are well-suited for modeling the control logic of systems with simple behavior and no data dependencies. However, the representation of systems involving both control and data aspects requires that FSMs be extended to EFSMs: EFSMs offer greater expressive power, but introduce challenges in test generation through infeasible paths and state explosion.

UCMs provide a graphical, scenario-based notation that emphasizes causal relationships and path-based flows, describing how a system behaves under specific conditions. They support early requirements analysis, and offer a high-level representation of system functionality. However, although UCMs can capture causal flows, they often lack the detailed behavioral information required for automated test-case generation (such as variable inputs, and boundary conditions). As a result, they may not provide sufficient precision for testing activities. Tools like jUCMNav allow engineers to create, manage, and transform UCM models. However, they typically involve manual mapping from use cases to UCM scenarios [150]. Despite their conceptual advantages, UCMs are primarily used in academic research or prototype systems in specific domains: They have had limited adoption in large-scale industrial projects.

Other modeling techniques, such as CEG and BPMs, require specialized modeling expertise, which can be a barrier for many test engineers. Additionally, the diverse requirements across different project domains make it difficult to establish standardized or broadly applicable models.

In summary, model-based requirements can offer valuable structure and abstraction for test-case generation, but their practical application varies significantly depending on the modeling technique. UML is the most widely used, due to its accessibility and tooling support. However, many studies only employ a subset of the UML diagrams (typically activity diagrams), due to complexity and maintenance concerns. Combining multiple UML diagrams can enhance fault detection, but introduces considerable modeling overhead. Automata-based models, especially FSM and EFSM, are the second most common. They enable precise behavioral modeling, but are limited by scalability issues. UCMs support high-level scenario representation, but often lack the detail needed for automated testing, and have rarely been adopted in industrial contexts. Other techniques, like CEGs and BPMs, are hindered by steep learning curves and limited generalizability. Overall, the depth of modeling and ease of integration into existing workflows remain critical factors influencing the adoption and effectiveness of model-based requirements in test-case generation.

5.4 Formal Mathematical Requirements

Formal mathematical specifications use precise logical formulas and constraint-specification languages to express system requirements, typically using mathematical notations [46]. A *structured object-oriented formal language* (SOFL) [50, 234, 251] integrates graphical and formal notations, defining functions through pre- and post-conditions. Z formal specification [57, 124, 141, 274] structures specifications into modular units called schemas using formal mathematical notation. *Linear temporal logic* (LTL) [21, 22, 310] describes the temporal evolution of system behaviors with mathematical formulas: It has been widely used in formal verification, especially for model checking. Other formal specifications, such as *software cost reduction* (SCR) specifications [102], *functional scenario form* (FSF) [176] and *featured transition systems* (FTSs) [77], have also been used. Although these specifications offer advantages (like automatic generation of test cases), they also face disadvantages in terms of meeting coverage requirements efficiently.

Formal mathematical requirements are particularly well-suited for embedded systems and numerical software, where precision, consistency, and rigorous verification are essential. They are generally accessible to technical teams with a solid mathematical background. However, their use is limited in agile or business-driven environments due to the steep learning curve and poor readability for non-technical stakeholders. As a result, such notations are rarely adopted in projects that demand frequent requirement changes, fast iterations, or broad stakeholder engagement.

5.5 Hybrid Requirements

The hybrid category includes requirements' specifications that combine two or more types of representations. A combined textual specification, for example, may consist of a use-case diagram, textual use cases, HTML screen mockups, and a glossary defining key terminology [70]. Other combinations include UML diagrams integrated with business vocabulary and business rules [89] or object constraint language [10], and visual requirement specifications or prototypes [268].

Hybrid requirements are used relatively infrequently (3%) and typically involve familiar models such as UML diagrams integrated with business rules or visual requirement prototypes. These combinations help compensate for missing information in individual models, leading to more accurate and comprehensive test-case generation. They are particularly effective for web-based systems and object-oriented applications. However, maintaining consistency across different representations increases overheads, and in projects with frequent requirement changes, the effort required to keep artifacts synchronized can become a significant burden.

6 ANSWER TO RQ3: WHAT ARE THE APPROACHES THAT SUPPORT RBTG?

RBTG generation approaches take requirements as input and produce test cases as output. Different requirements' representations use different strategies to generate test cases. Of the 267 surveyed studies, 39% generated test cases directly from the requirements, with the other 61% first transforming the requirements into intermediate representations (before generating test cases). Figure 7 shows that the most common forms of requirements (such as models and NL) use intermediate representations as the basis for test-case generation. Accordingly, this section examines these two main strategies: direct test-case generation and test-case generation through intermediate representations.

6.1 Test-Case Generation without Intermediate Representations

Table 7 summarizes the direct generation approaches identified in the surveyed papers. These approaches are categorized based on the types of input requirements: The form of the requirements naturally shapes the generation strategy. Within each category, the approaches are arranged in the order of their frequency of occurrence in the reviewed studies. The table also reports the *degree of automation* for each approach: *Fully automated* refers

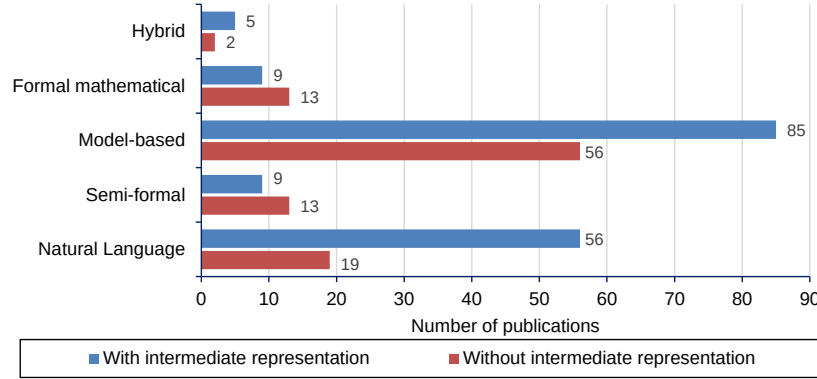


Fig. 7. Test-case generation approaches with/without intermediate representations.

Table 7. Approaches without Intermediate Representations

Requirements	Approach	No. of Studies	Degree of Automation		
			Fully	Semi	Not Specified
Natural language	Natural language processing (NLP)	7	6	1	0
	Machine learning (ML)	2	0	2	0
	Model-based approach	2	0	2	0
	Hybrid approach	2	0	2	0
	Other specific approach	6	0	5	1
Semi-formal	Ontology-based approach	4	0	4	0
	Model-based approach	3	1	2	0
	Semantic-head-driven approach	2	0	2	0
	ASSERT™	2	1	1	0
	Row-guided random algorithm with fuzzing (RGraF)	2	1	0	1
Model-based	Model-based approach	31	17	9	5
	Heuristics-based approach	5	3	0	2
	Transformation-based approach	3	0	3	0
	Hybrid approach	2	1	0	1
	Other specific approach	15	7	2	6
Formal mathematical	Model-based approach	6	3	2	1
	Heuristic-based approach	2	1	1	0
	Specific approach	5	4	0	1
Hybrid	Other specific approach	2	1	1	1

to methods that, according to the authors, require no human intervention; *Semi-automated* methods involve some manual effort, such as preprocessing or reviewing and adjusting generated outputs; and *Not specified* means that the original study either did not clarify the degree of automation or presented only a conceptual framework without implementation. This classification reflects both the maturity and the practical applicability of the identified approaches.

6.1.1 Approaches Based on Natural Language Requirements. In the context of NL requirements, NLP remains the most widely adopted: This includes techniques such as syntax testing [137], which analyzes and captures the sequence of patterns or syntax of variables from the data dictionary; syntactic similarity [186], which determines

the case conditions and actions; and keyword extraction [23]. Recently, LLMs have gained significant attention and are increasingly becoming a preferred approach for test-case generation [27, 188, 259, 316]. As shown in Table 7, among the approaches dealing with NL requirements, only those based on NLP, particularly LLMs, support fully automated test-case generation. NLP uses advances in language understanding and reasoning to automatically process NL requirements. They can also easily be integrated into real-world applications. However, NLP techniques also have limitations: Traditional syntax-testing techniques, for example, focus only on verifying whether or not the inputs match the correct syntax or format, without analyzing value ranges or boundary conditions [137]. LLMs overcome some of these limitations by supporting complex language understanding and inference: They can generate test cases from NL requirements without human intervention. However, LLMs also face challenges: Most LLMs have token limits, which constrain the length of context that they can process; hallucinations, including incorrect or fabricated outputs, affect the accuracy of the results; and data privacy can be a concern, especially when dealing with sensitive or confidential information [316]. In spite of their potential, the use of LLMs in software testing remains underexplored.

ML has also been commonly used: Ueda et al. [292], for example, enhanced the accuracy of automatic test-case generation by training data selection. Similarly, Kikuma et al. [158] trained ML on tagged data, and then applied the model to new requirements specifications to generate test cases. However, ML approaches typically involve manual preprocessing. Requirements must be annotated with tags and structured into training datasets, which is both labor-intensive and time-consuming. Furthermore, the quality of the generated test cases depends on the relevance and representativeness of the training data for the target requirements.

Model-based approaches have also been used to derive test cases from CNL requirements, making use of the characteristic CNL constructs. However, these approaches may also require substantial manual effort [180, 212], such as specifying requirements, specifying test cases, and refining test cases. Hybrid approaches integrate multiple methodologies: Fernandes et al. [94], for example, integrated equivalence class partitioning and boundary value analysis, while Elghondakly et al. [85] combined text mining with symbolic execution methodologies. Specific tailored approaches have also been proposed: Dwarakanath et al. [80], for example, introduced Litmus, which uses a syntactic parser (called the Link Grammar parser) alongside pattern-matching rules. These rules were designed to evaluate testability, identify test intents, and systematically generate both positive and negative test cases — a positive test case verifies the system’s expected behavior under normal conditions, and a negative test case checks the system’s response to exceptional or invalid inputs. These approaches also require manual intervention.

In summary, NL-based approaches are highly scalable and domain-independent. However, their precision and reproducibility may suffer due to the ambiguity and variability in natural languages. NLP approaches, especially those using LLMs, show promise for full automation, but most methods still require significant human involvement.

6.1.2 Approaches Based on Semi-Formal Requirements. “Semi-formal” approaches primarily depend on the structure and form of the input requirements. These approaches include those based on ontologies, such as the approaches by Ul Haq and Qamar [293] and Feldmann et al. [93], which integrate a knowledge-based system (ontology) with a learning-based testing algorithm to automate the test-case generation. Similarly, Tarasov et al. [284] used inference rules to query the ontology, enabling the verification of conditions and the retrieval of the necessary data for test-case construction. However, these approaches are specifically designed for mechatronic or embedded systems and still involve manual effort — selecting test cases from different ontologies and updating relevant parameters, for example. Furthermore, the proposed methods are mainly at a conceptual level, without practical implementation or experimental validation: Several critical aspects remain unexplored, such as how to handle different combinations of test data within a single test case.

Model-based approaches are also included in this category, where test cases are derived from SysML models [1, 324]. Abbors et al. [1] proposed a method to support requirements traceability throughout the model-based testing process: Their approach allows test cases to be traced back to the requirement models from which they were generated. While effective for small-scale systems with well-defined requirements and standardized toolchains, the method supports only traceability from requirements to transitions, and overlooks action-level logic. It also depends heavily on commercial tools such as MagicDraw and Qtronic, which limit its applicability to modern complex systems that require more flexible and tool-agnostic traceability solutions.

The semantic-head-driven generation approach [262] produced functional test cases using knowledge stored in a courteous logic [165] representation of requirements — this is an expressive subclass of standard logical representations that incorporates procedural attachments for prioritized conflict resolution. However, this approach relies on manually defined predicates. It also lacks sufficient support for modeling complex relationships, such as state dependencies and temporal constraints, making it less applicable to systems with dynamic behavior. Furthermore, the generated test cases often contain invalid combinations, leading to bloated and inefficient test suites.

Hybrid approaches have also been proposed, such as *ASSERTTM* [71, 193]: This approach uses a suite of tools designed to capture requirements; provides explainable and automated formal analyses to detect and resolve errors during the requirements authoring process; and automatically generates a complete set of test cases. However, this approach also incurs a high cost when transforming the input requirements, and relies heavily on manual translation by domain experts.

RGRaF [298, 299] applied a dependency-driven random algorithm with fuzzing to generate test cases, targeting time-boundary conditions. However, its applicability is limited to the automotive domain.

Although each of the above approaches reduces ambiguity, most are limited to specific domains and require manual effort. None is sufficient on its own to address the challenges of modern complex systems, and many either lack industrial validation or fail to scale to scenarios with high variability and real-world complexity.

6.1.3 Approaches Based on Model-Based Requirements. In the “Model-based” category, naturally, model-based approaches represent the main strategy, with the core methodology centered on constructing detailed models derived from requirements. Once the model is established, test cases can be automatically generated using traversal algorithms, such as *depth first search* (DFS) [173], pre-order traversal algorithms [318], and category-partition methods [115]. Yonathan et al. [318] extracted the sequence diagram in PlantUML format, organized the parsed information into a tree structure, and then used a pre-order traversal algorithm to explore each node. However, the branch handling is relatively simplistic — it only supports basic *alt* and *opt* constructs, and does not account for nested conditions. As a result, the generated paths may miss certain combinations of scenarios.

Several heuristics-based approaches have also been applied for model-based requirements: Genetic algorithms [133, 263], ant-colony algorithms [3], and metaheuristic techniques [152] have all been used to generate test cases. A genetic algorithm uses iterative stochastic optimization through selection, crossover, and mutation to evolve test-case populations, converting objectives into quantifiable fitness functions, and evaluating generated test data. An ant colony algorithm, inspired by the way ants use pheromones to find routes between colonies and food, can solve computational problems and identify optimal paths within graphs. Metaheuristics, as problem-independent optimization strategies, are black-box techniques capable of finding global optima without relying on greedy search methods. Heuristics-based approaches can achieve full automation in academic settings, and are valuable for exploratory research. However, their industrial applicability has been limited, due to the high computational costs and the risk of exponential growth in coverage calculations.

Transformation-based approaches [41, 130] represent specific implementations of model-based strategies, focusing on unique traversal mechanisms. Boucher and Mussbacher [41] used boundary value analysis to input data, and Myers’ test selection heuristics to generate a set of test cases for the specified workflow. However,

the method requires test engineers to manually augment the existing UCM model with additional information necessary for acceptance testing. Moreover, when dealing with complex workflows involving loops or nested branches, the number of scenarios can grow exponentially, leading to a path-explosion problem.

Hybrid approaches combine multiple methods, such as equivalence and *classification tree methods* (CCTM) [135], to improve testing efficiency and coverage. Intana and Sriraksa [135] verified and validated differences in requirements' specifications by analyzing two versions of the requirements. This enabled the generation of new test cases that improve coverage and reduce redundancy. However, it focuses solely on patterns of variable requirement changes, without addressing modifications in business logic. Additionally, the introduction of new variables can still lead to a significant increase in the number of test cases, and the estimated test-case reuse remains relatively low.

Model-based approaches, particularly those using graph-based representations, have been widely adopted in the reviewed studies. These techniques are effective in generating comprehensive test suites with quantifiable coverage, and are suitable for safety-critical and embedded systems. However, their effectiveness depends on the quality of the models and the complexity of the SUT. Manual modeling remains a significant bottleneck, particularly in large-scale or frequently evolving systems. Model-based testing is prone to state explosion, which, despite efforts at mitigation, remains a challenge for highly complex systems. Test-case generation for concurrent and distributed systems is another challenge, with a gap between academia and industry.

6.1.4 Approaches Based on Formal Requirements. Model-based and heuristic-based techniques are common in "formal mathematical" approaches. Kadakolmath and Ramu [141] introduced a ProZ model-checking-based approach that uses a breadth-first search strategy to generate test cases. Aniculaesei et al. [22] used the NuSMV model checker to derive test cases from LTL requirements. Liu and Nakajima [176] proposed a vibration method to heuristically generate test cases, and Wang et al. [308] used a genetic algorithm to identify the best mutant test condition, which was then used to generate test data. Table 7 shows that most approaches based on formal mathematical requirements support fully automated test-case generation. However, constructing such specifications requires specialized expertise. Most of these approaches can produce concrete test cases, but face state-space explosion. Furthermore, these methods are typically validated only in specific domains, particularly in safety-critical systems.

6.1.5 Discussion. Among the approaches that generate test cases directly from requirements, each category offers distinct advantages and faces specific limitations. Model-based techniques stand out for their versatility, and have been the most widely adopted. They support a range of formats, and enable structured generation of test suites with measurable coverage, especially in safety-critical and embedded domains.

NLP approaches and heuristics-based methods, particularly those leveraging genetic algorithms, have also been frequently used. NLP approaches have gained notable scalability, and the potential to handle unstructured NL inputs. However, such approaches may lack formal guarantees or traceability, unless combined with structured representations.

No single approach outperforms all others across all evaluation metrics. NL and semi-formal methods offer accessibility and ease of use; model-based and formal methods provide precision and rigor; and hybrid approaches combine the strengths of multiple paradigms: Some of the most effective solutions arose from synergistic combinations. LLM-based or NLP techniques, for instance, can be used during the pre-processing stage to extract structured artifacts from free-text requirements, which can then be used as inputs for model-based or search-based test generators, (functioning similarly to the generator approaches that utilize intermediate models, as discussed in Section 6.2). These multi-strategy pipelines are better suited to accommodate varying domain characteristics and system complexities, resulting in more robust and adaptable test-generation processes. The optimal combination of approaches typically depends on factors such as the formality of the requirements, the criticality of the SUT, and domain-specific constraints.

6.2 Test-Case Generation with Intermediate Representations

Many test-case generation approaches use an intermediate representation as part of the process: This helps with the extraction and formalization of key information from requirements, improving both the accuracy and effectiveness. As Figure 7 shows, 61% of the surveyed studies use an intermediate model as part of the transformation process.

6.2.1 Intermediate Representations. This section explores the different types of intermediate representations commonly used in the reviewed studies.

Table 8 groups the intermediate formats into five categories: graphical models; state/process models; logical specifications; Petri nets; and tabular representations. Graphical models include diagrams and directed graphs that represent system states and transitions. State/process models are formal models that describe system behavior with mathematical notations. Logical specifications use specification-based expressions to define requirements. Petri nets, with their unique notation and formal semantics, are treated as a separate category. Finally, tabular representations arrange requirements' logic into structured tables for enhanced clarity and interpretability. As Table 8 shows, graphical models were the most frequently utilized intermediate format, followed by state/process models. Logical specifications, Petri nets, and tabular formats are comparatively less common.

ADs [120, 246, 248] or activity graphs [60, 143, 278] are the most commonly used graphical model: This aligns with the findings in Section 5.3. The second most popular models were combined directed graphs (10 papers):

Table 8. Intermediate Representations

Intermediate Representation Categories	Intermediate Representations	No. of Studies	Total
Graphical models	Activity diagram (AD) or activity graph (AG)	19	90
	Combined directed graphs	10	
	Control flow graph (CCFG)	9	
	Sequence diagram (SD) or sequence graph (SG)	8	
	Trees	7	
	State start diagram or statechart diagram graph (SCDG)	6	
	State transition diagram (STD)	4	
	Cause-effect-graph (CEG)	3	
	Use case test models (UCTMs)	3	
	Flow dependency graph (FDG)	2	
State/Process models	Other diagrams or other directed graph	19	47
	Finite state machine (FSM)	11	
	Communicating sequential processes (CSP) test models	6	
	Extended finite state machine model (EFSM)	4	
	Meta-model	2	
	Symbolic model	2	
	Knowledge graph	2	
Logical specifications	Other specific models	20	20
	Functional scenarios form (FSF)	3	
	Requirement interfaces	2	
	Requirement specification ontology	2	
	Software cost reduction (SCR) specifications	2	
Petri nets	Other formalization	11	4
	Coloured petri nets (CPN)	2	
Tabular formats	Petri-net models	2	3
	Activity dependency table (ADT)	1	
	Decision table	2	

These integrate multiple representations, such as *control-flow graphs* (CFG) and *data-flow graphs* (DFG) [83, 87], or statechart diagram graphs and sequence-diagram graphs [166]. The third most popular were control flow graphs (nine papers) [29, 165, 281], which provide a static representation of models by capturing all possible control flow paths. Other graphical formats include sequence diagram/graph [78, 81, 213], trees [72, 111, 287], SCDG [100, 220], STD [59], CEG [96], UCTMs [304], and FDG [146]. Additionally, 19 other studies used various other specialized diagrams or directed graphs tailored to specific contexts [156, 168, 288, 305, 317].

FSMs were the most widely adopted state/process models, with 11 studies exploring their use [9, 32, 70]. *Communicating sequential processes* (CSP) models [203–205] were also frequently used for formalizing system behaviors. Other intermediate models include EFSM [139, 229], meta-models [11, 12], symbolic models [255], and knowledge graphs [197, 301]. Various other specialized models were also explored [31, 101].

Other intermediate representations that were less common included logical specifications [55, 56, 58, 67, 234, 250], Petri nets [227, 247, 266], and tabular representations [206]. However, even though less frequently used, these formats offer unique advantages, and may support specialized test-generation approaches.

In summary, the transformation of raw requirements into intermediate representations has often employed specialized algorithms. Diagrams and directed graphs are the most popular choice, highlighting their effectiveness in test-case generation. However, alternative intermediate formats (such as formal models and formal specifications) may advance test-generation methodologies and enhance their applicability.

6.2.2 Test-Case Generation from Intermediate Representations. Methods for generating test cases based on intermediate models align closely with the “model-based” category in Table 7. Model-based approaches [155, 199] dominate this field, followed by heuristics-based methods [264]. These studies highlight that, in most research, requirements are first transformed into specific models, which then support the efficient extraction of the test cases: Various algorithms and techniques are then applied to generate the test cases, such as traversal algorithms [2, 73, 205, 256], category-partition method [121, 127, 279], vibration method [175, 234], and other specialized techniques [13, 75, 215].

7 ANSWER TO RQ4: WHAT TYPES OF TEST-CASE REPRESENTATIONS HAVE BEEN EXPLORED IN RBTG?

Test cases can be generated using different approaches and representations, which then structure the outputs into various forms. This section examines on the types of test cases produced. Through an analysis of the reviewed papers, we identified three primary categories: Abstract test cases (ATCs); Concrete test cases (CTCs); and Test scenarios (TSs). (Detailed definitions of these categories were provided in Section 2.2.)

The distribution of each type of test output is shown in Figure 8(a). ATCs make up the majority of outputs, accounting for 61% of the reviewed studies. CTCs make up 33%, and can be executed either manually or automatically. Finally, TSs represent only 6% of the test outputs. In this paper, studies that specified the manual conversion of ATCs into CTCs were categorized as ATCs rather than CTCs: This helps to distinguish between directly executable test outputs and those requiring intermediate manual processing. As shown in Figure 8(b), ATCs account for more than 60% of the generated outputs for NL, semi-formal, and model-based requirements; ATCs represent 57% of outputs from hybrid requirements. Requirements expressed through formal mathematics, in contrast, result in less than 32% ATCs: This is because mathematical notations and applications in fields such as signal processing or control systems often define precise input-output relationships, making them more suitable for the direct generation of CTCs, rather than ATCs.

ATCs are high-level descriptions of testing requirements that are human-readable. They cannot be directly executed: They require additional modifications, such as incorporating concrete test data and defining test oracles [48], to transform them into executable test cases. The reviewed papers identified several common representation types for ATCs, including: *textual test cases*, *test paths*, *tabular test cases*, *test cases in other formats*.

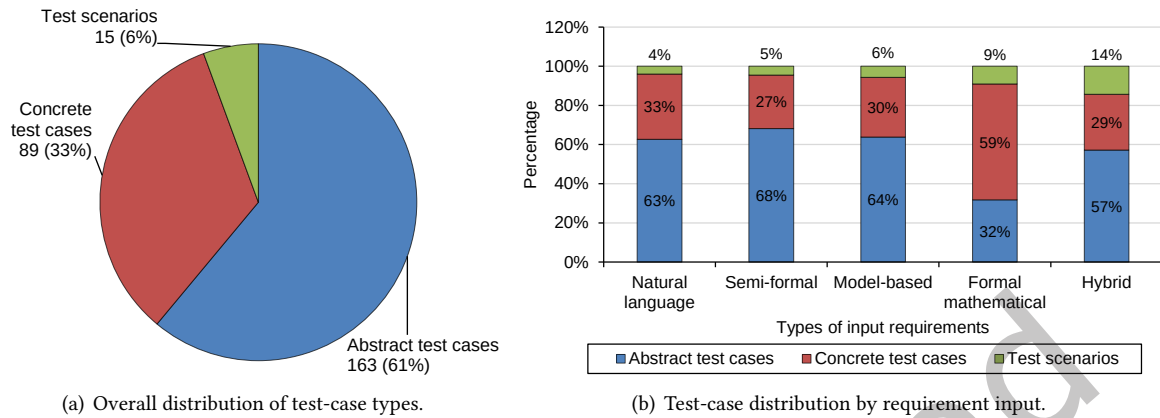


Fig. 8. Distribution of different test types.

- **Textual test cases:** These closely resemble manually written test cases, and are easy for humans to understand and analyze. Textual ATCs are often derived from NL or semi-formal input requirements. Yin et al. [316] used LLMs to generate textual ATCs from NL requirements. Fujita and Ueda [101], Ueda and Tsukada [292], and Fujita and Ueda [101] used ML techniques to generate textual test cases. Crapo and Moitra [71] and Sharma and Biswas [262] proposed approaches to obtain test cases from semi-formal requirements.
- **Test paths:** Test paths are derived by traversing specific graphs or models using specialized algorithms. Gupta et al. [111], for example, generated test paths from root to leaf nodes. Hamza and Hammad [119], Kamonsantiroj et al. [143] and Kim et al. [159] extracted test paths from activity graphs. Lafi et al. [165] and Alrawashed et al. [15] derived test paths from control flow graphs.
- **Tabular test cases:** These use decision tables with test inputs and outputs being represented as true/false values. Aoyama et al. [24] and Silva et al. [266] derived test cases by defining input steps based on the true or false conditions specified in the decision tables. Almohammad et al. [14], Olajubu et al. [208], and Kalaei and Rafe [142] also used decision tables for test-case representation. The tabular form is particularly useful for industrial control and embedded systems, where inputs and outputs are often digital signals.
- **Test cases in other formats:** There are also several other less common representations. Gnesi et al. [105], for example, generated test cases in a language combining elements of process algebra that offered a formal description of system behavior. Chinnaswamy et al. [67] used regular expressions, and Kadakolmath and Ramu [141] constructed test cases using specified predicates, storing them in XML format.

CTCs are low-level test cases with concrete (implementation-level) values as input data and expected results. CTCs are more closely aligned than ATCs with actual execution. They can be executed (manually or automatically using test scripts) in a specific environment.

- **Manual execution:** Manually-executed CTCs often augment ATCs with detailed test data. The data can be generated through manual input, randomization, or specialized methods tailored to specific applications. Gupta and Mahapatra [109] and Wang et al. [303] used semi-automatic techniques to generate textual CTCs that were then manually executed. Sun et al. [277], Jena et al. [138], and Zhang et al. [321] developed

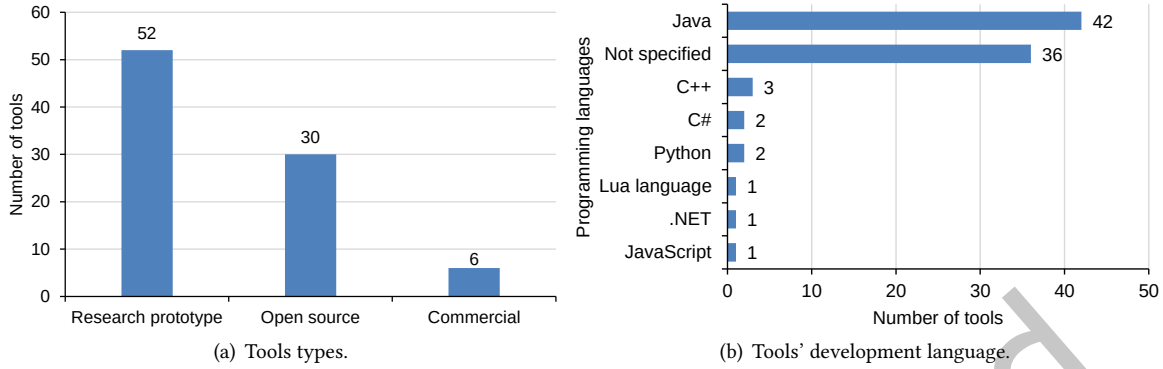


Fig. 9. Distribution of RBTG tools

test-case generation approaches by first randomly filling values along test paths without considering input-variable constraints, and then selecting cases that met the guard conditions.

- Automated execution: Automated CTC execution is primarily driven by test scripts, which can be generated using advanced tools and frameworks. Fernandes and Maciel [94] used AutomTest, a tool developed in Python 3, to produce executable CTCs. Aoyama et al. [25] and Khan et al. [153] converted ATCs into PROMELA test programs for their specific test execution environments. Granda et al. [107] generated test scripts based on relations between requirements, GUI elements, and Sikulix code in the Eclipse editor. Maciel et al. [180], Clerissi et al. [70], and Amalia et al. [16] used Selenium WebDriver to create executable test scripts, and Pradhan et al. [220] implemented test cases within the TestNG framework of J-Unit.

TSs are documents specifying a sequence of actions for the execution of a test. Arora et al. [27] noted that test scenarios simulate diverse user interactions and system behaviors. These scenarios can be represented in various formats, such as step sequences in UML activity diagrams, sequence diagrams, or NL descriptions. Some and Cheng [272] generated test scenarios based on path sequences, each comprising a test-scenario identifier, a setup section, a sequence of test steps, and an expected outcome.

In summary, test cases can be derived as ATCs in textual form or as textual test paths generated through graph traversals. However, these test cases often lack sufficient detail and completeness, and are unready for direct execution. CTCs can be generated semi-automatically with human intervention, bridging some of these gaps. Nevertheless, achieving a fully automated pipeline for both test-case generation and execution remains essential to significantly enhance testing efficiency and minimize manual effort. This transition from ATCs to CTCs poses substantial challenges and is an area of ongoing research.

8 ANSWER TO RQ5: WHAT TOOLS HAVE BEEN DEVELOPED TO SUPPORT RBTG?

Across the reviewed literature, we identified 88 tools designed specifically to support RBTG tasks, collectively referred to as RBTG tools. Table 9 classifies these RBTG tools into two groups, according to their operational phases: *requirements processing*, and *test generation*. (If a tool supports both requirements processing and test-case generation, it was classified under the test generation category.) The table includes the tools' names (e.g., jUCMNav), relevant reference (e.g., [6]), and implementation language (e.g., Java): Interested readers can find the original studies by consulting the references.

Table 9. RBTG Tools

Type	Tools	Counts
Requirements Processing		
Open Source	SequenceDiagram2ESG [81] (Java) , spaCy [108] (Python), ASSERT™ tool chain [193] (Not specified), Text2Test [31] (Not specified), USLTG [131] (Java), jUCMNav [43, 149] (Java), SystemCockpit [6] (Not specified), MoMuT::REQs [6] (Not specified), ROO [136] (Java), ArgoUML [276] (Not specified), ATOM3 [125, 126] (Python), Sahi [39] (Not specified), Semiformalizer [24] (Not specified)	13
Research Prototype	GUI-Test tool [107] (Java), DODT [6] (Not specified), RETNA [35] (Java), XML-based tool for State/Event Tree [290] (Java), TestMEReq [194] (Not specified), CNLParser [54] (Not specified), IMR-Generato [54] (Not specified), CAT [178] (Not specified), VDM [79] (Not specified)	9
Commercial	IBM Engineering Requirements Management DOORS [193] (Java), IBM Rational Rose [144, 146] (Java), Magic Draw UML [243, 267] (Java)	3
Test Generation		
Open Source	UMTG toolset [304] (Java), Test Generator for Cause Effect Graphs (TOUCH) [162] (Java), TESTSD2EFMS [229] (Java), AutomTest [94] (Java), TCGen [216] (Java), SPECMATE [98] (Java), Test-o-Matic [185] (Java), REBATE (REquirements Based Automatic Testing Engine) [269] (Java), AndromDA [84] (Java), FDR [203] (C++), TaRGeT [203] (Java), Use Case Acceptance Tester (UCAT) [82] (Java), SMT solver Z3 [7] (C++), EclEmma [183] (Java), T-VEC tool [56] (Not specified), TestGen [116] (Java), JWebUnit [116] (Java)	17
Research Prototype	MC/DC-TG-RT [24] (Java), ReqOntoTesGen [136] (Java), Tsgenerator [314] (Java), SYN-Test [137] (JavaScript), REQCAP [74] (Not specified), SAGA [74] (Not specified), Virtual Test Engineer (VTE) [19] (Java), ConcurTester [277] (Java), ACT (Abstract to Concrete Tests) [48] (Java) , MISTA [192] (Java), UBTCTG [209] (Java), NAT2TEST [53] (Java), BT Analyser [171] (Not specified), C&L (Scenarios & Lexicons) [246] (Lua language), Litmus [80] (.NET), SOLIMVA [76] (Not specified), STATEST [59] (Not specified), ATCGT [63] (Not specified), Comprehensive Test (ComTest) [281] (Java), UTCG [280] (Java), TSGen [276] (Java), ASSIST [245] (Java), GTSC [242] (Not specified), TAD [206] (C#), TDE/UML [123] (Not specified), UML behavioural test case generator (UTG) [237–239] (Java), GenTCase [134] (C++), Condado [241] (Not specified), UC-SCSystem [202] (Java), TSGAD [312] (Not specified), LEIRIOS Test Generator (LTG) [44] (Not specified), ScenTED-DTCD tool [225] (Not specified), UMLTGF [173] (Not specified), AGATHA [179] (Not specified), EXTRACT [319] (Not specified), UCTSystem [201] (Java), UMLTest [207] (Not specified), Objectteering CASE [200] (Not specified), PerformCharts [241] (Not specified), UsageTester [228] (Not specified), TCaseUML [275] (Not specified), Requirements Model Code Synthesis (RMCS) [112] (Not specified), RT-Tester [54] (Not specified)	43
Commercial	Spec explorer [315] (C#), Conformiq Creator [132] (Java), Conformiq's Qtronic [1] (Java)	3

In the requirements-processing category, open source tools are ranked highly (13 tools), followed by research-prototype tools (nine) and commercial tools (three). In the test-generation category, research-prototype tools lead significantly (43), with open-source (17) and commercial (three) tools being less common.

Overall, as shown in Figure 9(a), research-prototype tools were the most common RBTG tools, followed by the open source, and then commercial tools. Regarding the programming languages used in the implementation of these tools, as shown in Figure 9(b), Java stands out as the most widely used, accounting for 42 tools. 36 tools did not have their implementation language specified. The other languages (C++, C#, Python, Lua, .NET, and JavaScript), were used in significantly fewer cases.

9 ANSWER TO RQ6: HOW HAVE RBTG APPROACHES BEEN EVALUATED?

9.1 Overall Evaluation in RBTG

Chen and Babar [62] described nine types of evaluation methods for software product lines. Four of these were analyzed statistically based on the papers reviewed in this study. To ensure clarity, we renamed the evaluation types as *Example*, *Empirical Case Study*, *Real-World Case Study*, and *Discussion*.

- **Example.** In the example category, an example is used to demonstrate how the proposed method works. The example is primarily illustrative and not intended to validate or evaluate the approach. For instance, Ueda et al. [291] provided an example covering the entire process from requirements specification through data processing to test-case generation and evaluation. Yonathan et al. [318] used the login feature as an example to illustrate both the process and the results. Hong et al. [128] did the same using a coffee-vending machine scenario.
- **Empirical Case Study.** This explores a phenomenon within its real-life context. While based on real scenarios, these studies are typically conducted in controlled academic settings, and are explicitly designed as case studies. Gupta et al. [111] demonstrated their technique using a case study of ATM cash-withdrawal processes.
- **Real-World Case Study.** In this category, approaches are applied to real-world systems, often in collaboration with companies or public organizations. These studies emphasize practical applicability, and evidence of success in operational environments. Arora et al. [27] collaborated with Austrian Post, an international logistics and service provider, to generate test scenarios from NL requirements. Intana et al. [136] demonstrated the effectiveness and accuracy of their approach and tool through two case studies involving real-world systems.
- **Discussion.** Discussions provide some qualitative, textual, and opinion-based evaluation, without concrete examples or empirical data. Elqortobi et al. [87] and Lafi et al. [165], for example, analyzed the test-generation process primarily through discussions, without providing illustrative examples.

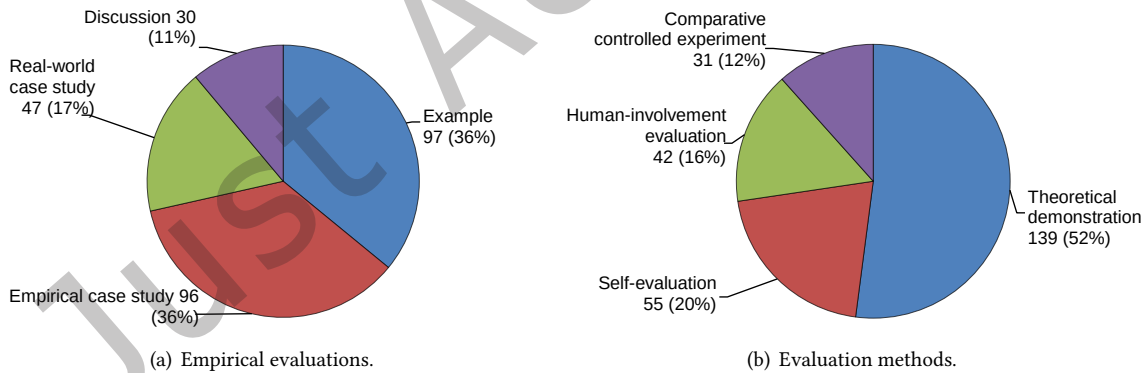


Fig. 10. Overall distribution of evaluation methods.

The 267 studies were classified based on their primary evaluation methods. Figure 10(a) shows that examples and empirical case studies were the largest proportion, each accounting for 36% of the total. In contrast, only 17% of the studies used real-world case studies [253], and 11% were categorized as discussions.

We categorized the reviewed studies according to their validation methods into four distinct types: *theoretical demonstration*, *self-evaluation*, *human-involvement evaluation*, and *comparative controlled experiment*.

- **Theoretical Demonstration.** This category included studies that primarily introduce and illustrate methods through theoretical explanations and examples, without any formal evaluation. Chen et al. [60], Kaur and Singh [147], and Reuys et al. [226] presented their test-generation process without conducting any evaluation.
- **Self-Evaluation.** This category included research that performs an initial assessment of the proposed approach using relevant data, typically focusing on internal validation without comparison to other methods. Gupta et al. [111] and Thanakorncharuwit et al. [285] evaluated their generated test cases based on some types of test coverage, while Wang et al. [308] involved mutation testing in the evaluation.
- **Human-Involvement Evaluation.** This category involved studies incorporating human participation, either through manual assessment of results or by comparing automated outputs with expert-generated ones. Arora et al. [27], Shah et al. [257], and Fernandez-Sanz and Misra [95] had their proposed approach critically evaluated by experts. Ekici and Tuglular [81] compared their automatically generated test sequences with those created manually.
- **Comparative Controlled Experiment.** The final category included research that had controlled experiments comparing the proposed method against existing approaches. This category provided rigorous validation of the proposed methods' effectiveness and advantages. Agrawal et al. [3] conducted experiments with three other algorithms, using corresponding parameter values for comparison. Elghondakly et al. [86] and Boghdady et al. [37] computed cyclomatic complexity values to measure the number of logical paths in a module and compared these values with those obtained from other methods.

Figure 10(b) shows that theoretical demonstration was the most common validation method, accounting for 52% of the studies. Self-evaluation, human-involvement evaluation, and comparative controlled experiments all had similar representation, with 20%, 16%, and 12% of the studies, respectively. Notably, this suggests that over half of the reviewed studies lack formal evaluation.

In summary, the dominance of examples and empirical case studies indicates that many proposed approaches lack sufficient practical validation: Only 17% of the studies employed case studies based on real-world systems, with most relying on theoretical demonstrations. Only 12% of the studies conducted comparative controlled experiments. The limited use of formal validation methods highlights a significant gap between theoretical research and practical applicability, making it difficult for practitioners to rely solely on existing research for decision-making.

9.2 Evaluation Methods and Metrics in RBTG

Various methods have been applied to evaluate proposed RBTG approaches. A range of metrics has been used to measure the efficiency and effectiveness of the generated test cases. This section provides a comprehensive overview of these evaluation methods, including the key metrics used in the reviewed studies.

Figure 11 summarizes the metrics adopted in the surveyed literature, categorized into three groups: *Efficiency*, *Effectiveness*, and *Other*. Similar metrics, such as fault detection, code coverage, and model coverage, were grouped within the categories. A single paper may have used multiple metrics in the evaluations.

The *effectiveness* metrics were categorized into four types: *fault detection*, *model coverage*, *code coverage*, and *precision, recall, and F-measure*.

- **Fault detection.** This metrics include *mutation score* [145, 176, 313] and *detected faults* [81, 164]. The mutation score, calculated as the ratio of killed mutants to the total number of mutants, is a widely used measure in mutation testing [176]. The detected faults metric quantifies the number of defects identified by the generated test cases, providing a direct assessment of their fault-detection capability.
- **Model coverage.** This category includes metrics such as *transition coverage* [111], *cyclomatic complexity* [68, 318], *activity coverage* [111], *action coverage* [9], and *fitness value* [166]. It also includes other coverage

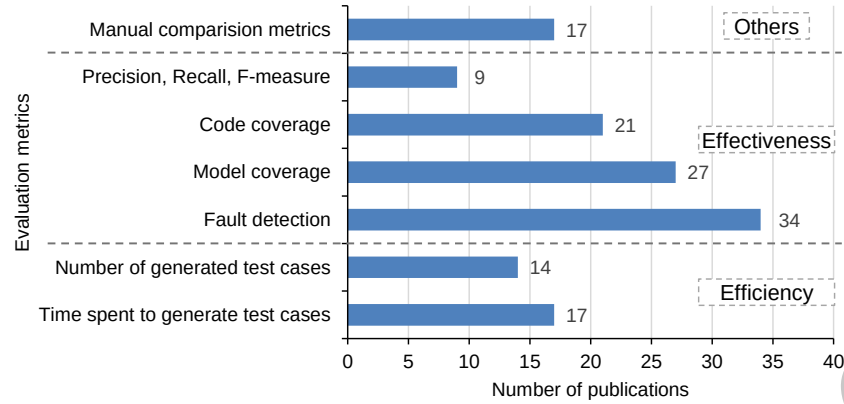


Fig. 11. Distribution of evaluation metrics.

criteria, such as *state coverage* [220] and *key path coverage* [64]. Coverage measurement, which evaluates whether or not the model can reach a certain state or transition, is a commonly used evaluation method.

- Code coverage. This category includes metrics such as *path coverage* [111], *MC/DC coverage* [223, 305, 322], *branch coverage* [304], and *row coverage* [3]. It also includes other coverage criteria, like *predicate coverage* [281]. Code coverage measurements have also been frequently used.
- Precision, recall, F-measure. The *F-measure*, also known as the F1 Score, combines precision and recall and is used to measure the accuracy of automatic test-case generation [291]. The formula for the F-measure is as follows:

$$Precision = \frac{TP}{TP + FP} \quad (1)$$

$$Recall = \frac{TP}{TP + FN} \quad (2)$$

$$F - measure = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (3)$$

(where TP is the True Positive, FP is the False Positive, and FN is the False Negative).

The evaluation of RBTG testing *efficiency* primarily used two key metrics: *time spent to generate test cases* [9], and *number of generated test cases* [61]. These metrics are commonly used to assess the reduction in manual effort needed for effective test-case generation.

Manual comparison metrics [27] involve human evaluation based on criteria such as relevance, coverage, correctness, understandability, and feasibility. This can provide qualitative insights into the effectiveness of the generated test cases, complementing quantitative evaluation approaches.

Other studies also adopted specialized evaluation methods, such as comparisons with manual evaluation (cross-validated by two evaluators) [170] and proof-of-concept evaluation [74], to assess the feasibility and effectiveness of the proposed approaches.

10 ANSWER TO RQ7: IN WHAT DOMAINS AND INDUSTRIAL APPLICATIONS HAS RBTG BEEN APPLIED?

The SUT application domain strongly influences RBTG, impacting the formulation and representation of requirements, and the selection of test-generation approaches and tools. Test cases can differ significantly between

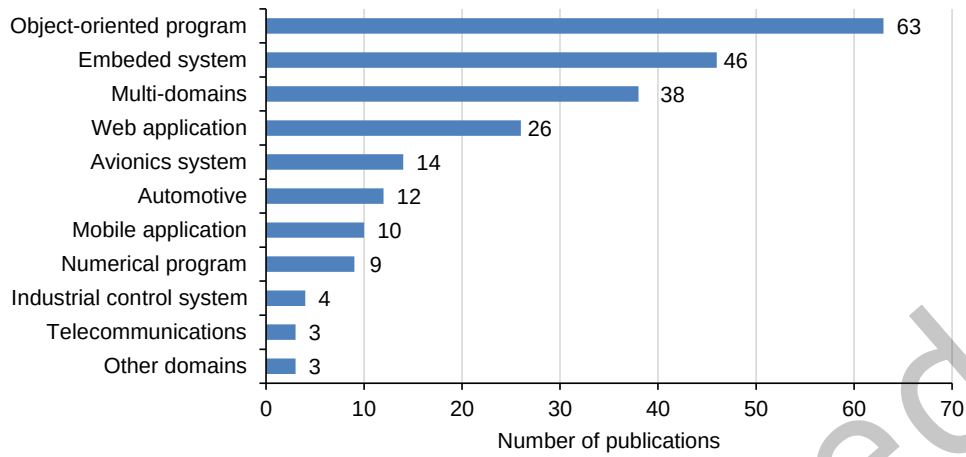


Fig. 12. Distribution of RBTG application domains.

domains in terms of number, style, detail, and granularity. This section identifies the key application domains that have been explored for RBTG.

228 of the 267 reviewed papers explicitly referenced specific application domains: These were incorporated into the analysis. For papers lacking explicit domain specifications, their case studies' domains were used. The resulting distribution of application domains is presented in Figure 12. The most extensively studied domain was object-oriented programs, in 63 papers. Other notable domains were embedded systems (46 papers), multi-domains (38 papers), and web applications (26 papers). The term multi-domain refers to approaches applied across multiple system types, such as both object-oriented programs and embedded systems [38, 40]. This highlights the significance of object-oriented programs in RBTG research, with an emphasis on model-based approaches that typically employ UML diagrams to describe object-oriented systems.

In object-oriented programs, objects interact to execute specific behaviors [146]. Within the context of RBTG research, a broad range of applications have been modeled as object-oriented programs, including: library management [45, 136, 137], online conference reviewing [106], kidney-failure diagnosis (KFD) [136], university education [315], and course registration [163]. This diversity in domains highlights the flexibility and adaptability of object-oriented modeling for RBTG.

Embedded systems, which typically operate with continuous digital input and output signals, have also been extensively explored in RBTG surveys. In such systems, input data are generally collected from sensors as the primary data source, and output data are transmitted to system actuators to perform control actions [266]. Examples of embedded systems studied in RBTG research include systems for elevator control [60], urban railway interlocking [141], BodySense (a safety-critical automotive software) [304], railroad crossings [162] and medical equipment [283]. These examples highlight the key role of embedded systems across safety-critical and control-oriented applications.

Web applications have also been a focus of RBTG research due to their widespread use in real-world systems. Example web applications include: the automatic exam-paper evaluation platform AutoEval [188]; the online trading platform Online Broker [247]; and the hotel-management system Hotel Ambassador [115]. These studies highlight the applicability of RBTG to software systems with the kind of dynamic and user-driven functionality typical of web-based applications.

Avionics and automotive domains have also drawn considerable attention in RBTG research, with 14 and 12 papers, respectively. Some examples in these domains include the Avionics Launch Abort System [305], Aircraft Engine Sensor [193], and Adaptive Cruise Control System [20].

Several other application domains have also been explored: Mobile applications, such as the Uber app [9], are linked to 10 studies. Numerical programs, including the *greatest common divisor* (GCD) [50] and Mod process [308], were examined in nine papers. Industrial control systems [71] and telecommunications [158] have been explored in four and three papers, respectively. Other less explored domains, such as healthcare [177], desktop applications [107], and logistics [27] were each covered in a single paper. These numbers highlight the applicability of RBTG across a wide range of industries and use cases.

Overall, while object-oriented programs, embedded systems, and web applications dominate the RBTG research, the diversity of applications highlights the adaptability of RBTG approaches across different domains. However, the limited exploration of some domains suggests opportunities for further research to enhance test-case generation techniques in these underrepresented areas.

11 ANSWER TO RQ8: WHAT ARE THE REMAINING CHALLENGES AND FUTURE WORK FOR RBTG?

Following the analyses in the previous sections, this section summarizes remaining challenges and potential future work for RBTG.

11.1 Challenges

This survey paper has identified that RBTG has attracted considerable research interest and has made notable progress in recent years. However, compared to some fields (like automated code generation), its advances remain limited, and the level of automation achieved is still lacking. Many challenges and open questions remain to be addressed.

11.1.1 Challenge 1: Constructing High-Quality Requirements. Requirements serve as the foundation of RBTG, directly influencing the strategies for test-case generation and determining the quality of the resulting test cases. In industrial practice, most requirements are expressed in NL, which can face lexical ambiguities, contextual inconsistencies, and structural incompleteness [195]: Test cases thus generated are often unreliable, significantly impacting the overall quality of the software testing and development. In traditional development processes, ensuring the precision, completeness, clarity, and consistency of the requirements often involves iterative reviews, conducted manually by engineers during the early project stages. The requirements can then be continuously refined throughout the development and testing phases to maintain their quality and adaptability.

The reviewed RBTG research revealed that efforts have been made to mitigate NL-related issues through translation into semi-formal [24] or formal representations, or through mapping tables for NLP [165]. However, these approaches often require substantial manual effort [107], risk losing important information (such as non-functional requirements), and do not guarantee that the requirements will be complete and unambiguous.

These limitations have hindered adoption in industry, especially in agile contexts, where requirements evolve incrementally and change frequently. In such situations, manual modeling is often perceived as costly and inflexible, providing limited effort reduction or practical benefits. Consequently, RBTG lags behind source code-based approaches in terms of practical adoption.

Recent advances in NLP and LLMs have opened new possibilities to address these issues. LLMs offer the potential to automatically detect, at scale, ambiguity, incompleteness, and inconsistency in NL requirements [182]. If integrated into the development process, this could significantly enhance the quality of requirements before test generation, helping to bridge the gap between requirements engineering and test automation. Ultimately, this

may result in high-quality, machine-readable requirements artifacts that serve as reliable inputs for automated test-case generation.

11.1.2 Challenge 2: From Abstract to Executable Test Cases. An effective testing process should start early, refine requirements with sufficient detail, support automated test generation, and enable seamless execution and feedback: This would reduce manual effort, and minimize the impact of human error.

RBTG has significant potential for improving automation and test quality. However, a key challenge is that requirements and architectural specifications are typically more abstract than the implementation, often omitting critical execution-level details. To produce executable test cases, these gaps must be filled — usually through manual effort [47]. Many current approaches either only generate ATCs, or require non-trivial manual steps to make them executable.

The creation of concrete test data and oracles involves random generation or manual crafting. Fernandes and Maciel [94] proposed AutomTest, which enables users to generate automated tests from functional requirements: It accepts data within parameter ranges and produces JUnit test cases. However, its applicability is constrained by limited variable support and the need for manual intervention. Pradhan et al. [220] used TestNG to automatically generate test cases based on events and state conditions. Moitra et al. [193] developed executable test scripts tailored for safety-critical systems: These tests can be run using *Safety-critical application development environment* (SCADE) coverage tools. However, these approaches are often based on formal requirements, and are constrained to specific domains.

The reviewed studies indicate that most RBTG approaches produce ATCs. Developing general, automated methods to transform ATCs into executable CTCs remains an open challenge — one that is critical to realizing the full potential of RBTG.

11.1.3 Challenge 3: Lack of Standard Benchmarks. Benchmarks are essential for evaluating and comparing implementations of abstract concepts and ideas. In RBTG, they enable robust analyses and fair comparisons of automated test-generation approaches. Effective benchmarks support independent replication, promote transparency, and foster reuse and extension of prior work [99]. The ATM system [233] is a widely used benchmark. However, we found that most surveyed studies rely on self-defined case studies, with nearly half of the reviewed work not involving any case-study validation. This lack of standardized, reusable benchmarks limits the reproducibility of results, hinders objective comparisons among approaches, and complicates the assessment of practical applicability.

In spite of their importance, RBTG benchmarks face several challenges. First, the development and maintenance of high-quality requirements benchmarks is time-consuming. Second, there are mismatched expectations and communication gaps among benchmark creators, users, and reviewers, leading to inconsistent documentation, incomplete artifacts, and limited usability [286]. Third, given the diversity of requirements' formats (NL, model-based, etc.), designing benchmark systems that comprehensively cover all relevant forms remains a non-trivial task.

To address this gap, we advocate for the construction of well-documented, high-quality benchmarking datasets that cover diverse requirement formats and application domains (embedded systems, enterprise applications, safety-critical systems, etc.). These benchmarks should be co-designed with the community, include realistic artifacts, and be supported by clearly defined evaluation tasks. Such efforts are crucial to promote reproducibility, enable fair comparisons, and foster measurable progress in RBTG.

11.1.4 Challenge 4: Rigorous and Efficient Evaluation. The quality of generated test cases significantly affects testing outcomes. However, in software testing, assessing test-case quality remains a challenge. Practitioners use fault-detection capability, usability, maintainability, reliability, and coverage as key indicators of high-quality test cases [289]: However, these attributes are inherently difficult to measure consistently and objectively.

In the context of RBTG, the selection of suitable approaches for automated test-case generation is critical. As illustrated in Figure 10(b), only 12% of the reviewed studies conducted controlled comparative experiments to evaluate their methods against existing baselines. Another 16% relied on expert judgment or comparisons with manually crafted test cases, introducing subjectivity into the evaluation. Although many studies report evaluation metrics (such as code coverage, model coverage, or fault-detection rate), these provide only a partial view. Furthermore, most studies adopt limited evaluation criteria, often focusing on only one or two features. This narrow scope restricts the ability to comprehensively assess the full effectiveness of the generation techniques.

The lack of a comprehensive evaluation framework hinders efforts to determine whether or not generated test cases adequately capture the intended SUT behaviors. To advance RBTG research, rigorous evaluation methodologies are required. These methodologies should extend beyond traditional metrics, incorporating both functional and non-functional dimensions, to enable a more comprehensive assessment of test-case generation techniques.

11.1.5 Challenge 5: Real-World Industrial Implementation. Despite advances in RBTG, its adoption in industrial settings remains limited. As discussed in Section 9, only 17% of the reviewed papers used real-world case studies to evaluate RBTG, and these often spanned different domains. Arora et al. [27] applied their approach to two projects at Austrian Post, where domain experts confirmed the coherence, feasibility, and comprehensiveness of the generated test scenarios. However, the necessity for human-expert involvement to confirm correctness was also noted.

Deploying RBTG approaches in industrial settings, especially those involving complex software systems, poses significant challenges related to integration, adaptability, and evaluation [30]. A primary barrier is the lack of rigorous and efficient mechanisms to evaluate generated test cases. In many situations, these test cases still require manual refinement, limiting the reduction in effort or tangible practical benefits. In addition, industrial requirements are seldom stable, especially in agile environments, where they evolve incrementally and change frequently. This dynamic nature means that test cases need to be continuously updated, and that the impact of requirements' changes be analyzed promptly and accurately. Current RBTG techniques generally lack sufficient support for such adaptability. Another obstacle is the difficulty of integrating RBTG tools into existing development and testing workflows. Most solutions are not designed to align with standard toolchains, making it difficult to embed them in *continuous integration* (CI) or DevOps pipelines without significant customization. Addressing these challenges will require close industry and academia collaboration, and a clear demonstration of cost-effectiveness in real-world environments.

11.2 Future work

The challenges identified above should be addressed through more effective and efficient approaches, which should be considered as future RBTG work. This section lists some potential future work for RBTG.

11.2.1 Future Work 1: Exploring LLM-Driven RBTG. A promising future direction will be the integration of LLMs into RBTG. While LLMs have been becoming increasingly popular, their effective application to complex software-testing workflows remains underexplored. RBTG techniques often struggle with ambiguous, incomplete, or inconsistent NL requirements, resulting in low-quality or incomplete test cases. The advanced NL understanding capabilities of LLMs offer opportunities to address these limitations by automatically detecting issues, such as ambiguity or incompleteness [182], and generating more consistent and verifiable test cases. However, challenges remain, including that LLMs frequently mishandle boundary conditions, or are not consistent with input values and test descriptions [160]. Fine-tuned LLMs can improve accuracy, but their performance is often domain-specific, and is heavily dependent on the quality of training data [122].

Future research should focus not only on improving LLM output quality, but also on integrating LLMs into traditional RBTG frameworks. This would include combining LLMs with structured requirement modeling (e.g., multi-modal LLMs), leveraging retrieval-augmented generation (RAG) with domain-specific requirements' knowledge bases, and embedding them into industrial development and testing pipelines.

11.2.2 Future Work 2: Optimizing Module-Centric Test Prioritization. Test-case prioritization (TCP) aims to optimize the execution order of test cases to enhance fault-detection efficiency. Various TCP methodologies have emerged, each leveraging different criteria and techniques to improve the process. In CI environments, for example, reinforcement-learning-based learn-to-rank models have been proposed to dynamically adapt to codebase changes, thereby reducing testing overhead while preserving fault-detection effectiveness [210]. In service-oriented workflow applications, TCP has been improved by analyzing activity dependencies and quantifying the impact of modifications, enabling more precise test scheduling [306].

Requirements-based TCP [235] offers another effective avenue, particularly for regression testing. By aligning test cases with high-priority or high-risk requirements, teams can focus on validating the most critical system behaviors first. For instance, Kavitha et al. [148] introduced a model that ranks test cases based on three key factors: customer priority, requirement volatility, and implementation complexity. Test cases are mapped to their corresponding requirements and assigned scores to reflect their relative importance.

Identifying error-prone modules early — during requirements analysis or test-case design — can provide valuable insights to guide TCP decisions and enhance testing effectiveness. Since 80% of defects are reported to be concentrated in 20% of the modules [36], allocating more testing effort to these critical modules is both intuitive and necessary. The use of LLMs may open new opportunities for identifying critical modules from NL requirements, enabling the generation of more targeted test cases, and supporting earlier fault detection and more efficient resource allocation. Therefore, a promising future direction will be to develop strategies for estimating module criticality from requirements, and using this information to guide requirements-based TCP and test-case allocation.

11.2.3 Future Work 3: Developing Effective RBTG Tools. An effective test-case generation tool should offer automation, flexibility, and adaptability. It should provide seamless integration with existing development workflows, support for complex GUI components, and have its source code openly available, to facilitate use and extension [221]. This would not only streamline the testing process, but would also help to ensure comprehensive coverage and consistent quality assurance.

Although many RBTG tools have been proposed, their adoption in industry remains limited. Several reasons have been identified for this: Limited evidence of their cost-effectiveness compared with traditional methods; difficulty in ensuring full coverage and reliability; poor integration with real-world development pipelines; and their applicability is often confined to narrow domains.

Advancing the development of practical, robust, and user-friendly RBTG tools is a critical direction for future research. Such efforts are essential to closing the gap between academic innovation and industrial adoption, ultimately delivering tools that meet the practical needs of software engineering teams.

11.2.4 Future Work 4: Exploring Non-Functional RBTG. Non-functional requirements specify quality attributes and constraints that define how well the system performs its intended functions. They relate to things such as performance, security, reliability, and usability [69]. While both functional and non-functional requirements are essential for complete software specification, non-functional requirements often receive less attention during development, and are typically validated only after functional testing [249]. This is largely due to time and budget constraints, and their frequent underspecification by stakeholders [51].

In the reviewed studies, RBTG was predominantly used for functional testing, with limited applications in performance [9], security [61], and other non-functional requirements. The primary reason for this is that

functional requirements constitute the majority of documented requirements. Additionally, performance testing and security testing often require more specialized models that detect and analyze specific types of data [307].

Despite the availability of performance tools like LoadRunner [189] and JMeter [26], there remains a need for manual script design. Future work should integrate these tools to automate non-functional testing, reducing manual effort and improving efficiency.

Although non-functional RBTG has received limited attention in recent years, the increasing adoption of LLMs offers opportunities to automatically extract performance, security, and usability-related requirements from NL, and to generate corresponding test cases. By fine-tuning pre-trained models with targeted prompts and optimizing them through reinforcement learning, LLMs can be adapted to produce realistic and effective non-functional test cases [254]. These advances may renew interest in this underexplored, but critical area.

11.2.5 Future Work 5: Extensive Empirical Studies to Compare RBTG Approaches. Future work should also include comprehensive and reproducible empirical studies that enable fair comparisons across categories (NLP-based, model-based, etc.) and within each category. Although no standardized guidelines currently exist, several comparison dimensions identified in this survey may serve as a foundation for such studies, including: The complexity of the input requirements; the learning effort required for the generation techniques; the coverage achieved by the generated test cases; the number of test cases and time required for generation; and practical aspects such as tool support and integration effort. These dimensions can serve as a basis for developing evaluation frameworks and benchmarking protocols. Establishing such foundations will enable meaningful comparisons across RBTG approaches, and provide researchers and practitioners with actionable guidance for selecting techniques that best suit their specific needs.

12 CONCLUSION

This paper has provided a comprehensive survey of 267 *requirements-based test generation* (RBTG) studies published up to 2024. We traced the distribution of RBTG topics, and analyzed the entire process of generating test cases based on requirements. We classified input requirements' types and approaches; identified different types of test cases; and conducted a statistical review of existing tools and evaluation methods. We have also highlighted the diverse applications of RBTG across multiple domains, and discussed the challenges and potential opportunities in the field.

Our review revealed that early RBTG methods primarily used formal or semi-formal requirements' representations to generate test cases, with model-based approaches emerging as the primary strategy. However, recent advancements in *natural language processing* (NLP) have shifted the focus towards NL requirements. A significant amount of previous research focused on model-based testing, which faces challenges for complex systems. There was also a notable lack of methods for transforming *abstract test cases* (ATCs) into *concrete test cases* (CTCs), and a lack of effective evaluation methods.

Several RBTG challenges require future investigation. First, input requirements are often not fully integrated with the domain knowledge, the prototype, and other information. Secondly, most test cases generated by RBTG are ATCs, with few universal methods for converting ATCs into CTCs. Finally, current evaluation processes remain insufficient. There is also a lack of general-purpose automated testing tools that can substantially reduce manual effort. We have also identified some potential opportunities, such as the application of *large language models* (LLMs) to RBTG, potentially enhancing test generation for critical modules.

This paper provides a comprehensive overview of the current state, gaps, and future research directions for RBTG. With automated code generation already widely adopted in industry, the demand for automated testing is more urgent than ever. The insights provided in this paper will benefit both researchers and practitioners, improving testing methodologies and ultimately enhancing the quality and reliability of software systems.

ACKNOWLEDGMENT

This work in this paper was partly supported by the Science and Technology Development Fund of Macau, Macau SAR (Grant No. 0021/2023/RIA1), and the General Research Project of Zhejiang Provincial Department of Education (Grant No. Y202558065).

REFERENCES

- [1] Fredrik Abbors, Dragos Truscan, and Johan Lilius. 2009. Tracing requirements in a model-based testing approach. In *Proceedings of the 1st International Conference on Advances in System Testing and Validation Lifecycle*. 123–128.
- [2] Oluwatolani Achimugu, Philip Achimugu, Chinonyelum Nwufoh, Ssegguja Hussein, Ridwan Kolapo, and Tolulope Olufemi. 2021. An improved approach for generating test cases during model-based testing using tree traversal algorithm. *Journal of Software Engineering and Applications* 14, 6 (2021), 257–265.
- [3] Supriya Agrawal, R Venkatesh, Ulka Shrotri, Amey Zare, and Sagar Verma. 2020. Scaling test case generation for expressive decision tables. In *Proceedings of the IEEE 13th International Conference on Software Testing, Validation and Verification*. 364–374.
- [4] Tanwir Ahmad, Junaid Iqbal, Adnan Ashraf, Dragos Truscan, and Ivan Porres. 2019. Model-based testing using UML activity diagrams: A systematic mapping study. *Computer Science Review* 33 (2019), 98–112.
- [5] Imran Ahsan, Wasi Haider Butt, Mudassar Adeel Ahmed, and Muhammad Waseem Anwar. 2017. A comprehensive investigation of natural language processing techniques and tools to generate automated test cases. In *Proceedings of the Second International Conference on Internet of things, Data and Cloud Computing*. 1–10.
- [6] Bernhard K Aichernig, Klaus Hörmaier, Florian Lorber, Dejan Nickovic, Rupert Schlick, Didier Simoneau, and Stefan Tiran. 2014. Integration of requirements engineering and test-case generation via OSLC. In *Proceedings of the 14th International Conference on Quality Software*. 117–126.
- [7] Bernhard K Aichernig, Dejan Ničković, and Stefan Tiran. 2015. Scalable incremental test-case generation from large behavior models. In *Proceedings of the 2015 International Conference on Tests and Proofs*. 1–18.
- [8] Mounifah Alenazi, Nan Niu, and Juha Savolainen. 2019. SysML modeling mistakes and their impacts on requirements. In *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)*. 14–23.
- [9] Amira Ali, Huda Amin Maghawry, and Nagwa Badr. 2021. Model-based test case generation approach for mobile applications load testing using OCL enhanced activity diagrams. In *Proceedings of the 10th International Conference on Intelligent Computing and Information Systems*. 493–499.
- [10] Md Azaharuddin Ali, Khasim Shaik, and Shreyansh Kumar. 2014. Test case generation using UML state diagram and OCL expression. *International Journal of Computer Applications* 95, 12 (2014), 7–11.
- [11] Sai Chaithra Allala, Juan P Sotomayor, Dionny Santiago, Tariq M King, and Peter J Clarke. 2019. Towards transforming user requirements to test cases using MDE and NLP. In *Proceedings of the IEEE 43rd Annual Computer Software and Applications Conference*, Vol. 2. 350–355.
- [12] Sai Chaithra Allala, Juan P Sotomayor, Dionny Santiago, Tariq M King, and Peter J Clarke. 2022. Generating abstract test cases from user requirements using MDSE and NLP. In *Proceedings of the IEEE 22nd International Conference on Software Quality, Reliability and Security*. 744–753.
- [13] Mateus Andrade Almeida, Juliana de Melo Bezerra, and Celso Massaki Hirata. 2013. Automatic generation of test cases for critical systems based on MC/DC criteria. In *Proceedings of the IEEE/AIAA 32nd Digital Avionics Systems Conference*. 7C5–1.
- [14] Ali Almohammad, Joao F Ferreira, Alexandra Mendes, and Phil White. 2017. Reqcap: Hierarchical requirements modeling and test generation for industrial control systems. In *Proceedings of the IEEE 25th International Requirements Engineering Conference Workshops*. 351–358.
- [15] Thamer A Alrawashed, Ammar Almomani, Ahmad Althunibat, and Abdelfatah Tamimi. 2019. An automated approach to generate test cases from use case description model. *Computer Modeling in Engineering & Sciences* 119, 3 (2019), 409–425.
- [16] Silfi Nur Amalia, Sri Widowati, and Donni Richasdy. 2022. Application of the equivalent partitioning method in testing for automatic test case generation on the Digi-OTA system. *Jurnal Media Informatika Budidarma* 6, 1 (2022), 35–40.
- [17] Paul Ammann and Jeff Offutt. 2017. *Introduction to software testing*. Cambridge University Press.
- [18] Daniel Amyot, Jean-François Roy, and Michael Weiss. 2005. UCM-driven testing of web applications. In *Proceedings of the 12th International SDL Forum on SDL 2005: Model-Driven Engineering for Smart Cities*. 247–264.
- [19] R Anbunathan and Anirban Basu. 2016. Automatic test generation from UML sequence diagrams for Android mobiles. *International Journal of Applied Engineering Research* 11, 7 (2016), 4961–4979.
- [20] Adina Aniculaesei, Falk Howar, Peer Denecke, and Andreas Rausch. 2018. Automated generation of requirements-based test cases for an adaptive cruise control system. In *2018 IEEE Workshop on Validation, Analysis and Evolution of Software Tests*. 11–15.
- [21] Adina Aniculaesei, Andreas Vorwald, and Andreas Rausch. 2019. Automated generation of requirements-based test cases for an automotive function using the SCADE toolchain. In *Proceedings of the 11th International Conference on Adaptive and Self-Adaptive*

- Systems and Applications*. 69–74.
- [22] Adina Aniculaesei, Andreas Vorwald, and Andreas Rausch. 2019. Using the SCADE toolchain to generate requirements-based test cases for an adaptive cruise control system. In *Proceedings of the ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion*. 503–513.
 - [23] Ahlam Ansari, Mirza Baig Shagufta, Ansari Sadaf Fatima, and Shaikh Tehreem. 2017. Constructing test cases using natural language processing. In *Proceedings of the 3rd International Conference on Advances in Electrical, Electronics, Information, Communication and Bio-Informatics*. 95–99.
 - [24] Yusuke Aoyama, Takeru Kuroiwa, and Noriyuki Kushiro. 2020. Test case generation algorithms and tools for specifications in natural language. In *Proceedings of the 2020 IEEE International Conference on Consumer Electronics*. 1–6.
 - [25] Yusuke Aoyama, Takeru Kuroiwa, and Noriyuki Kushiro. 2021. Executable test case generation from specifications written in natural language and test execution environment. In *Proceedings of the IEEE 18th Annual Consumer Communications & Networking Conference*. 1–6.
 - [26] Apache Software Foundation. 2024. Apache JMeter. <https://jmeter.apache.org/>.
 - [27] Chetan Arora, Tomas Herda, and Verena Himm. 2024. Generating test scenarios from NL requirements using retrieval-augmented LLMs: An industrial study. In *Proceedings of the IEEE 32nd International Requirements Engineering Conference*. 240–251.
 - [28] Shreya Banerjee, Narayan C Debnath, and Anirban Sarkar. 2021. An ontology-based approach to automated test case generation. *SN Computer Science* 2, 35 (2021), 1–12.
 - [29] Priti Bansal and Sangeeta Sabharwal. 2013. A model based approach to test case generation for testing the navigation behavior of dynamic web applications. In *Proceedings of the 6th International Conference on Contemporary Computing*. 213–218.
 - [30] Mohammad Baqar and Rajat Khanda. 2025. The future of software testing: AI-powered test case generation and validation. In *Intelligent Computing-Proceedings of the Computing Conference*. 276–300.
 - [31] Devesh Bhatt, Hao Ren, Anitha Murugesan, Jason Biatek, Srivatsan Varadarajan, and Natarajan Shankar. 2022. Requirements-driven model checking and test generation for comprehensive verification. In *Proceedings of the 14th International Symposium NASA Formal Methods*. 576–596.
 - [32] Zhu Bin and Wang Anbao. 2012. Functional and user interface model for generating test cases. In *Proceedings of the 11th International Conference on Computer and Information Science*. 605–610.
 - [33] Robert Binder. 2000. *Testing object-oriented systems: models, patterns, and tools*. Addison-Wesley Professional.
 - [34] International Software Testing Qualifications Board. 2014. Standard glossary of terms used in software testing.
 - [35] Ravishankar Boddu, Lan Guo, Supratik Mukhopadhyay, and Bojan Cukic. 2004. RETNA: from requirements to testing in a natural way. In *Proceedings of the 12th IEEE International Requirements Engineering Conference*. 262–271.
 - [36] Barry Boehm and Victor R Basili. 2007. Software defect reduction top 10 list. *Software engineering: Barry W. Boehm's lifetime contributions to software development, management, and research* 34, 1 (2007), 75.
 - [37] PN Boghdady, NL Badr, MA Hashem, and MF Tolba. 2012. Automatic generation of multi-testing types test cases using requirements-based testing. In *Proceedings of the 7th International Conference on Computer Engineering & Systems*. 249–254.
 - [38] P Boghdady, NL Badr, MA Hashem, and MF Tolba. 2012. An enhanced technique for generating hybrid coverage test cases using activity diagrams. In *Proceedings of the 8th International Conference on Informatics and Systems*. 20–28.
 - [39] Pakinam N Boghdady, Nagwa L Badr, Mohamed Hashem, and Mohamed F Tolba. 2011. A proposed test case generation technique based on activity diagrams. *International Journal of Engineering & Technology IJET-IJENS* 11, 03 (2011), 1–21.
 - [40] Pakinam N Boghdady, Nagwa L Badr, Mohamed A Hashim, and Mohamed F Tolba. 2011. An enhanced test case generation technique based on activity diagrams. In *Proceedings of the 2011 International Conference on Computer Engineering & Systems*. 289–294.
 - [41] Mathieu Boucher and Gunter Mussbacher. 2017. Transforming workflow models into automated end-to-end acceptance test cases. In *Proceedings of the IEEE/ACM 9th International Workshop on Modelling in Software Engineering*. 68–74.
 - [42] Mohamed Boukhilif, Mohamed Hanine, Nassim Kharmoum, Atenea Ruigómez Noriega, David García Obeso, and Imran Ashraf. 2024. Natural language processing-based software testing: A systematic literature review. *IEEE Access* 12 (2024), 79383–79400.
 - [43] Patrice Boulet, Daniel Amyot, and Bernard Stepien. 2015. Towards the generation of tests in the test description language from use case map models. In *Proceedings of the 17th International SDL Forum on SDL 2015: Model-Driven Engineering for Smart Cities*. 193–201.
 - [44] Fabrice Bouquet, Eddie Jaffuel, Bruno Legeard, Fabien Peureux, and Mark Utting. 2005. Requirements traceability in automated test generation: Application to smart card software validation. *ACM SIGSOFT Software Engineering Notes* 30, 4 (2005), 1–7.
 - [45] Lionel Briand and Yvan Labiche. 2002. A UML-based approach to system testing. *Software and Systems Modeling* 1 (2002), 10–42.
 - [46] Jean-Michel Bruel, Sophie Ebersold, Florian Galinier, Manuel Mazzara, Alexandr Naumchev, and Bertrand Meyer. 2021. The role of formalism in system requirements. *Comput. Surveys* 54, 5 (2021), 1–36.
 - [47] Khusbu Bubna and Sujit Chakrabarti. 2016. End to End Specification based Test Generation of Web Applications.. In *Proceedings of the 11th International Conference on Evaluation of Novel Software Approaches to Software Engineering*. 296–302.
 - [48] Khusbu Bubna and Sujit Kumar Chakrabarti. 2016. ACT (abstract to concrete tests)-A tool for generating Concrete test cases from formal specification of web applications. In *Proceedings of the 2nd Modelling Symposium*. 16–22.

- [49] Gustavo Cabral and Tetsuo Tamai. 2008. Requirement-based testing through formal methods. *Proceedings of TESTCOM-FATES* (2008).
- [50] Román Jaramillo Cajica, Raúl Ernesto González Torres, and Pedro Mejía Álvarez. 2021. Automatic generation of test cases from formal specifications using mutation testing. In *Proceedings of the 18th International Conference on Electrical Engineering, Computing Science and Automatic Control*. 1–6.
- [51] Cristina Rosa Camacho, Sabrina Marczak, and Daniela S Cruzes. 2016. Agile team members perceptions on non-functional testing: influencing factors from an empirical study. In *2016 11th international conference on availability, reliability and security*. 582–589.
- [52] Emanuela G Cartaxo, Francisco GO Neto, and Patricia DL Machado. 2007. Test case generation by means of UML sequence diagrams and labeled transition systems. In *Proceedings of the 2007 IEEE International Conference on Systems, Man and Cybernetics*. 1292–1297.
- [53] Gustavo Carvalho, Flávia Barros, Ana Carvalho, Ana Cavalcanti, Alexandre Mota, and Augusto Sampaio. 2015. NAT2TEST tool: From natural language requirements to test cases based on CSP. In *Software Engineering and Formal Methods*. 283–290.
- [54] Gustavo Carvalho, Flávia Barros, Florian Lapschies, Uwe Schulze, and Jan Peleska. 2014. Model-based testing from controlled natural language requirements. In *Formal Techniques for Safety-Critical Systems*. 19–35.
- [55] Gustavo Carvalho, Diogo Falcão, Flávia Barros, Augusto Sampaio, Alexandre Mota, Leonardo Motta, and Mark Blackburn. 2013. Test case generation from natural language requirements based on SCR specifications. In *Proceedings of the 28th Annual ACM Symposium on Applied Computing*. 1217–1222.
- [56] Gustavo Carvalho, Diogo Falcão, Flávia Barros, Augusto Sampaio, Alexandre Mota, Leonardo Motta, and Mark Blackburn. 2014. NAT2TESTSCR: Test case generation from natural language requirements based on SCR specifications. *Science of Computer Programming* 95 (2014), 275–297.
- [57] Kai H Chang, Shih-Sung Liao, Stephen B Seidman, and Richard Chapman. 1998. Testing object-oriented programs: From formal specification to test scenario generation. *Journal of Systems and Software* 42, 2 (1998), 141–151.
- [58] Suraiya Charoenreh and Adisak Intana. 2019. Enhancing software testing with ontology engineering approach. In *Proceedings of the 23rd International Computer Science and Engineering Conference*. 186–191.
- [59] Ram Chatterjee and Kalpana Johari. 2010. A prolific approach for automated generation of test cases from informal requirements. *ACM SIGSOFT Software Engineering Notes* 35, 5 (2010), 1–11.
- [60] FeiFei Chen, YaRu Zhao, and Zhen Li. 2023. Research on software test case generation based on SysML. In *Proceedings of the 2023 International Symposium on Intelligent Robotics and Systems*. 173–177.
- [61] JinFu Chen, Yemin Yin, Saihua Cai, Ye Geng, and Longxia Huang. 2022. An improved test case generation method based on test requirements for testing software component. In *Proceedings of the IEEE 22nd International Conference on Software Quality, Reliability, and Security Companion*. 209–218.
- [62] Lianping Chen and Muhammad Ali Babar. 2011. A systematic review of evaluation of variability management approaches in software product lines. *Information and Software Technology* 53, 4 (2011), 344–362.
- [63] Lizhe Chen and Qiang Li. 2010. Automated test case generation from use case: A model based approach. In *Proceedings of the 3rd International Conference on Computer Science and Information Technology*, Vol. 1. 372–377.
- [64] Mingsong Chen, Prabhat Mishra, and Dhruvajyoti Kalita. 2010. Efficient test case generation for validation of UML activity diagrams. *Design Automation for Embedded Systems* 14, 2 (2010), 105–130.
- [65] Yuan Chen, Anbang Wang, Junjie Wang, Luo Liu, Yuanzhang Song, and Qinghua Ha. 2018. Automatic test transition paths generation approach from EFSM using state tree. In *Proceedings of the 2018 IEEE International Conference on Software Quality, Reliability and Security Companion*. 87–93.
- [66] Philippe Chevalley and Pascale Thevenod-Fosse. 2001. Automated generation of statistical test cases from UML state diagrams. In *Proceedings of the 25th Annual International Computer Software and Applications Conference*. 205–214.
- [67] Arunkumar Chinnaswamy, BA Sabarish, and R Deepak Menan. 2024. User story based automated test case generation using NLP. In *Proceedings of the 2024 International Conference on Computational Intelligence in Data Science*. 156–166.
- [68] Chanda Chouhan, Vivek Shrivastava, and Parminder S Sodhi. 2012. Test case generation based on activity diagram for mobile application. *International Journal of Computer Applications* 57, 23 (2012), 4–9.
- [69] Lawrence Chung, Brian A Nixon, Eric Yu, and John Mylopoulos. 2012. *Non-functional requirements in software engineering*. Springer Science & Business Media.
- [70] Diego Clerissi, Maurizio Leotta, Gianna Reggio, and Filippo Ricca. 2017. Towards the generation of end-to-end web test scripts from requirements specifications. In *Proceedings of the IEEE 25th International Requirements Engineering Conference Workshops*. 343–350.
- [71] Andrew W Crapo and Abha Moitra. 2019. Using OWL ontologies as a domain-specific language for capturing requirements for formal analysis and test case generation. In *Proceedings of the IEEE 13th International Conference on Semantic Computing*. 361–366.
- [72] Steven J Cunneing and JW Rozenbitt. 1999. Test scenario generation from a structured requirements specification. In *Proceedings of the IEEE Conference and Workshop on Engineering of Computer-Based Systems*. 166–172.
- [73] Swagatika Dalai, Arup Abhinna Acharya, and Durga Prasad Mohapatra. 2012. Test case generation for concurrent object-oriented systems using combinational UML models. *International Journal of Advanced Computer Science and Applications* 3, 5 (2012), 97–102.

- [74] Flemström Daniel, Enoiu Eduard, Azal Wasif, Sundmark Daniel, Gustafsson Thomas, and Kobetski Avenir. 2018. From natural language requirements to passive test cases using guarded assertions. In *2018 IEEE International Conference on Software Quality, Reliability and Security*. 470–481.
- [75] André LL de Figueiredo, Wilkerson L Andrade, and Patrícia DL Machado. 2006. Generating interaction test cases for mobile phone systems from use case specifications. *ACM SIGSOFT Software Engineering Notes* 31, 6 (2006), 1–10.
- [76] Valdivino Alexandre de Santiago Júnior and Felipe Elias Costa da Silva. 2017. From statecharts into model checking: A hierarchy-based translation and specification patterns properties to generate test cases. In *Proceedings of the 2nd Brazilian Symposium on Systematic and Automated Software Testing*. 1–10.
- [77] Xavier Devroey, Gilles Perrouin, and Pierre-Yves Schobbens. 2014. Abstract test case generation for behavioural testing of software product lines. In *Proceedings of the 18th International Software Product Line Conference: Companion Volume for Workshops, Demonstrations and Tools-Volume 2*. 86–93.
- [78] M Dhineshkumar et al. 2014. An approach to generate test cases from sequence diagram. In *Proceedings of the 2014 International Conference on Intelligent Computing Applications*. 345–349.
- [79] Jeremy Dick and Alain Faivre. 1993. Automating the generation and sequencing of test cases from model-based specifications. In *International Symposium of Formal Methods Europe*. 268–284.
- [80] Anurag Dwarakanath and Shubhashis Sengupta. 2012. Litmus: Generation of test cases from functional requirements in natural language. In *Proceedings of the 17th International Conference on Applications of Natural Language Processing and Information Systems*. 58–69.
- [81] Nazım Umut Ekici and Tugkan Tuglular. 2023. Automatic test sequence generation and functional coverage measurement from UML sequence diagrams. *International Journal of Information System Modeling and Design* 14, 1 (2023), 1–21.
- [82] Mohamed El-Attar and James Miller. 2010. Developing comprehensive acceptance tests from use cases and robustness diagrams. *Requirements Engineering* 15, 3 (2010), 285–306.
- [83] Mounia El Qortobi, Amine Rahj, Jamal Bentahar, and Rachida Dssouli. 2020. Test generation tool for modified condition/decision coverage: model based Testing. In *Proceedings of the 13th International Conference on Intelligent Systems: Theories and Applications*. 1–6.
- [84] Meryem Elallaoui, Khalid Nafil, and Raja Touahni. 2016. Automatic generation of TestNG tests cases from UML sequence diagrams in Scrum process. In *Proceedings of the 4th IEEE International Colloquium on Information Science and Technology*. 65–70.
- [85] Roaa Elghondakly, Sherin Moussa, and Nagwa Badr. 2015. Waterfall and agile requirements-based model for automated test cases generation. In *Proceedings of the IEEE 7th International Conference on Intelligent Computing and Information Systems*. 607–612.
- [86] Roaa Elghondakly, Sherin Moussa, and Nagwa Badr. 2016. An optimized approach for automated test case generation and validation for UML diagrams. *Asian J Inf Technol* 15, 21 (2016), 4276–4290.
- [87] Mounia Elqortobi, Amine Rahj, and Jamal Bentahar. 2023. Granular traceability between requirements and test cases for safety-critical software systems. In *Proceedings of the 19th International Conference on Mobile Web and Intelligent Information Systems*. 202–220.
- [88] María José Escalona, Javier J Gutierrez, Manuel Mejías, Gustavo Aragón, Isabel Ramos, Jesús Torres, and Francisco José Domínguez. 2011. An overview on test generation from functional requirements. *Journal of Systems and Software* 84, 8 (2011), 1379–1393.
- [89] Imane Essebaa, Salima Chantit, and Mohammed Ramdani. 2019. Model-based testing from model driven architecture: A novel approach for automatic test cases generation. In *Proceedings of the 2019 International Conference on Smart Information & Communication Technologies*. 600–609.
- [90] Xin Fan, Jian Shu, LinLan Liu, and Qijun Liang. 2009. Test case generation from UML subactivity and activity diagram. In *Proceedings of the 2nd International Symposium on Electronic Commerce and Security*. 244–248.
- [91] MS Farooq and Tayyaba Tahreem. 2022. Requirement-based automated test case generation: Systematic literature review. *VFAST Transactions on Software Engineering* 10, 2 (2022), 133–142.
- [92] Mattia Fazzini, Martin Prammer, Marcelo d’Amorim, and Alessandro Orso. 2018. Automatically translating bug reports into test cases for mobile apps. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 141–152.
- [93] Stefan Feldmann, Susanne Rösch, Christoph Legat, and Birgit Vogel-Heuser. 2014. Keeping requirements and test cases consistent: Towards an ontology-based approach. In *Proceedings of the 12th IEEE International Conference on Industrial Informatics*. 726–732.
- [94] Daniel David Fernandes and Rita Suzana Pitangueira Maciel. 2020. Towards a test case generation tool based on functional requirements. In *Proceedings of the 19th Brazilian Symposium on Software Quality*. 1–6.
- [95] Luis Fernandez-Sanz and Sanjay Misra. 2012. Practical application of UML activity diagrams for the generation of test cases. *Proceedings of the Romanian Academy, Series A* 13, 3 (2012), 251–260.
- [96] Jannik Fischbach, Julian Frattini, Andreas Vogelsang, Daniel Mendez, Michael Unterkalmsteiner, Andreas Wehrle, Pablo Restrepo Henao, Parisa Yousefi, Tedi Juricic, Jeannette Radduenz, et al. 2023. Automatic creation of acceptance tests by extracting conditionals from requirements: NLP approach and case study. *Journal of Systems and Software* 197 (2023), 111549.
- [97] Jannik Fischbach, Maximilian Junker, Andreas Vogelsang, and Dietmar Freudenstein. 2019. Automated generation of test models from semi-structured requirements. In *Proceedings of the IEEE 27th International Requirements Engineering Conference Workshops*. 263–269.

- [98] Jannik Fischbach, Andreas Vogelsang, Dominik Spies, Andreas Wehrle, Maximilian Junker, and Dietmar Freudenstein. 2020. SPECMATE: Automated creation of test cases from acceptance criteria. In *Proceedings of the IEEE 13th International Conference on Software Testing, Validation and Verification*. 321–331.
- [99] Afonso Fontes and Gregory Gay. 2023. The integration of machine learning into automated test generation: A systematic mapping study. *Software Testing, Verification and Reliability* 33, 4 (2023), e1845.
- [100] Peter Fröhlich and Johannes Link. 2000. Automated test case generation from dynamic models. In *Proceedings of the European Conference on Object-Oriented Programming*. 472–491.
- [101] Yuto Fujita and Kiyoshi Ueda. 2024. A method for selecting training data using Doc2Vec for automatic test Cases generation. In *Proceedings of the 2024 IEEE International Conference on Consumer Electronics*. 1–6.
- [102] Angelo Gargantini and Constance Heitmeyer. 1999. Using model checking to generate tests from requirements specifications. *ACM SIGSOFT Software Engineering Notes* 24, 6 (1999), 146–162.
- [103] Vahid Garousi, Sara Bauer, and Michael Felderer. 2020. NLP-assisted software testing: A systematic mapping of the literature. *Information and Software Technology* 126 (2020), 106321.
- [104] Chuyan Ge, TianTian Wang, XiaoTian Yang, and Christoph Treude. 2025. Cross-level requirements tracing based on large language models. *IEEE Transactions on Software Engineering* 9 (2025), 1–5.
- [105] Stefania Gnesi, Diego Latella, and Mieke Massink. 2004. Formal test-case generation for UML statecharts. In *Proceedings of the 9th IEEE International Conference on Engineering of Complex Computer Systems*. 75–84.
- [106] Maria Fernanda Granda, Nelly Condori-Fernández, Tanja EJ Vos, and Oscar Pastor. 2014. Towards the automated generation of abstract test cases from requirements models. In *Proceedings of the IEEE 1st International Workshop on Requirements Engineering and Testing*. 39–46.
- [107] Maria Fernanda Granda, Otto Parra, and Bryan Alba-Sarango. 2021. Towards a model-driven testing framework for GUI test cases generation from user stories. In *Proceedings of the 16th International Conference on Evaluation of Novel Approaches to Software Engineering*. 453–460.
- [108] Robin Gröpler, Viju Sudhi, Emilio José Calleja García, and Andre Bergmann. 2021. NLP-based requirements formalization for automatic test case generation. In *Proceedings of the 29th International Workshop on Concurrency, Specification and Programming*, Vol. 21. 18–30.
- [109] Atulya Gupta and Rajendra Prasad Mahapatra. 2023. Test case generation and history data analysis during optimization in regression testing: An NLP study. *Cogent Engineering* 10, 2 (2023), 2276495.
- [110] Nishant Gupta, Vibhash Yadav, and Mayank Singh. 2018. Automated regression test case generation for web application: A survey. *Comput. Surveys* 51, 4 (2018), 1–25.
- [111] Nishant Gupta, Vibhash Yadav, and Mayank Singh. 2023. Decision tree based test case generation using story board and natural language processing. In *Proceedings of the 2023 International Conference on Advances in Computing and Data Sciences*. 581–591.
- [112] P Gupta, SI Cunning, and Jerzy W Rozenblit. 2001. Synthesis of high-level requirements models for automatic test generation. In *Proceedings of the 8th Annual IEEE International Conference and Workshop on the Engineering of Computer-Based Systems*. 76–82.
- [113] Priya Gupta and Prafullakumar Surve. 2011. Model based approach to assist test case creation, execution, and maintenance for test automation. In *Proceedings of the 1st International Workshop on End-to-End Test Script Engineering*. 1–7.
- [114] Javier Gutiérrez, Gustavo Aragón, Manuel Mejías, Francisco Jose Dominguez Mayo, and Carmen M Ruiz Cutilla. 2012. Automatic test case generation from functional requirements in NDT. In *Proceedings of the 12th International Conference on Web Engineering*. 176–185.
- [115] JJ Gutiérrez, MJ Escalona, and M Mejías. 2015. A model-driven approach for functional test case generation. *Journal of Systems and Software* 109 (2015), 214–228.
- [116] Javier J Gutierrez, Maria J Escalona, Manuel Mejias, Jesus Torres, and Arturo H Centeno. 2008. A case study for generating test cases from use cases. In *Proceedings of the 2nd International Conference on Research Challenges in Information Science*. 209–214.
- [117] Peter Hamberger, Claus Klammer, Thomas Luger, Michael Moser, Michael Pfeiffer, and Christina Piereder. 2023. Specification-based test case generation for C++ engineering software. In *Proceedings of the 2023 IEEE International Conference on Software Maintenance and Evolution*. 519–529.
- [118] Reinhard Hametner, Benjamin Kormann, Birgit Vogel-Heuser, Dietmar Winkler, and Alois Zoitl. 2011. Test case generation approach for industrial automation systems. In *Proceedings of the 5th International Conference on Automation, Robotics and Applications*. 57–62.
- [119] Zahra Abdulkarim Hamza and Mustafa Hammad. 2019. Generating test sequences from UML use-case diagrams. In *Proceedings of the 2019 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies*. 1–6.
- [120] Zahra Abdulkarim Hamza and Mustafa Hammad. 2021. Analyzing UML use cases to generate test sequences. *International Journal of Computing and Digital Systems* 10, 1 (2021), 125–134.
- [121] Jean Hartmann, Claudio Imoberdorf, and Michael Meisinger. 2000. UML-based integration testing. In *Proceedings of the 2000 ACM SIGSOFT International Symposium on Software Testing and Analysis*. 60–70.
- [122] Navid Bin Hasan, Md Ashraf Islam, Junaed Younus Khan, Sanjida Senjik, and Anindya Iqbal. 2025. Automatic high-level test case generation using large language models. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories*. 674–685.

- [123] Bill Hasling, Helmut Goetz, and Klaus Beetz. 2008. Model based testing of system requirements using UML use case models. In *Proceedings of the 1st International Conference on Software Testing, Verification, and Validation*. 367–376.
- [124] Steffen Helke, Thomas Neustupny, and Thomas Santen. 1997. Automating test case generation from Z specifications with Isabelle. In *Proceedings of 10th the International Conference of Z Users*. 52–71.
- [125] Abdelkamel Hettab, Allaoua Chaoui, and Ahmad Aldahoud. 2013. Automatic test cases generation from UML activity diagrams using graph transformation. In *Proceedings of the 6th International Conference on Information Technology*. 1–12.
- [126] Abdelkamel Hettab, Elhillali Kerkouche, and Allaoua Chaoui. 2015. A graph transformation approach for automatic test cases generation from UML activity diagrams. In *Proceedings of the 8th International C* Conference on Computer Science & Software Engineering*. 88–97.
- [127] Robert M Hierons. 1997. Testing from a Z specification. *Software Testing, Verification and Reliability* 7, 1 (1997), 19–33.
- [128] Hyoung Seok Hong, Insup Lee, Oleg Sokolsky, and Sung Deok Cha. 2001. Automatic test generation from statecharts using model checking. In *Workshop on Formal Approaches to Testing of Software*. 15–30.
- [129] John E Hopcroft, Rajeev Motwani, and Jeffrey D Ullman. 2001. Introduction to automata theory, languages, and computation. *ACM SIGACT News* 32, 1 (2001), 60–65.
- [130] Chu Thi Minh Hue, Dang Duc Hanh, and Nguyen Ngoc Binh. 2018. A transformation-based method for test case automatic generation from use cases. In *Proceedings of the 10th International Conference on Knowledge and Systems Engineering*. 252–257.
- [131] Chu Thi Minh Hue, Dang Duc Hanh, Nguyen Ngoc Binh, and Truong Anh Hoang. 2019. USLTG: test case automatic generation by transforming use cases. *International Journal of Software Engineering and Knowledge Engineering* 29, 09 (2019), 1313–1345.
- [132] Aliya Hussain, Saurabh Tiwari, Jagadish Suryadevara, and Eduard Enoiu. 2018. From modeling to test case generation in the industrial embedded system domain. In *Proceedings of the 2018 International Conferences on Software Technologies: Applications and Foundations*. 499–505.
- [133] Alfredo Ibias, Pablo Vazquez-Gomis, and Miguel Benito-Parejo. 2021. Coverage-based grammar-guided genetic programming generation of test suites. In *Proceedings of the 2021 IEEE Congress on Evolutionary Computation*. 2411–2418.
- [134] Rosziati Ibrahim, Mohd Zainura Saringat, Noraini Ibrahim, and Noraida Ismail. 2007. An automatic tool for generating test cases from the system’s requirements. In *Proceedings of the 7th IEEE International Conference on Computer and Information Technology*. 861–866.
- [135] Adisak Intana and Thiwatip Sriraksa. 2019. Impact analysis framework of test cases based on changes of use case based requirements. In *Proceedings of the 23rd International Computer Science and Engineering Conference*. 230–235.
- [136] Adisak Intana, Kuljaree Tantayakul, Kanjana Laosen, and Suraiya Charoenreh. 2023. An approach of test case generation with software requirement ontology. *International Journal of Advanced Computer Science and Applications* 14, 8 (2023), 1005–1014.
- [137] Adisak Intana, Monchanok Thongthep, Phatcharee Thepnimit, Phaplak Saethapan, and Tanawat Monpipat. 2020. SYNTTest: Prototype of syntax test case generation tool. In *Proceedings of the 5th International Conference on Information Technology*. 259–264.
- [138] Ajay Kumar Jena, Santosh Kumar Swain, and Durga Prasad Mohapatra. 2014. A novel approach for test case generation from UML activity diagram. In *Proceedings of the 2014 International Conference on Issues and Challenges in Intelligent Computing Techniques*. 621–629.
- [139] Mingyue Jiang and Zuohua Ding. 2011. Automation of test case generation from textual use cases. In *Proceedings of the 4th International Conference on Interaction Sciences*. 102–107.
- [140] Kunxiang Jin and Kevin Lano. 2021. Generation of test cases from UML diagrams - A systematic literature review. In *Proceedings of the 14th Innovations in Software Engineering Conference*. 1–10.
- [141] Lokanna Kadakolmath and Umesh D Ramu. 2022. Model-checking-based automated test case generation for Z formal specification of an urban railway interlocking system. In *Proceedings of the 4th International Conference on Emerging Research in Electronics, Computer Science and Technology*. 1–8.
- [142] Akram Kalae and Vahid Rafe. 2016. An optimal solution for test case generation using ROBDD graph and PSO algorithm. *Quality and Reliability Engineering International* 32, 7 (2016), 2263–2279.
- [143] Suwatchai Kamonsantiroj, Luepol Pipanmaekaporn, and Siriluck Lorpunmanee. 2019. A memorization approach for test case generation in concurrent UML activity diagram. In *Proceedings of the 2nd International Conference on Geoinformatics and Data Analysis*. 20–25.
- [144] Supaporn Kansomkeat and Wanchai Rivepiboon. 2003. Automated-generating test case using UML statechart diagrams. In *Proceedings of the 2003 Annual Research Conference of the South African Institute of Computer Scientists and Information Technologists on Enablement through Technology*. 296–300.
- [145] Supaporn Kansomkeat, Phachayane Thiket, and Jeff Offutt. 2010. Generating test cases from UML activity diagrams using the Condition-Classification Tree Method. In *Proceedings of the 2nd International conference on software technology and engineering*. 62–66.
- [146] Arvinder Kaur and Vidhi Vig. 2018. Automatic test case generation through collaboration diagram: A case study. *International Journal of System Assurance Engineering and Management* 9, 2 (2018), 362–376.
- [147] Manpreet Kaur and Rupinder Singh. 2014. Generation of Test Cases from Sliced Sequence Diagram. *International Journal of Computer Applications* 97, 5 (2014), 29–34.
- [148] RVRK Kavitha, VR Kavitha, and N Suresh Kumar. 2010. Requirement based test case prioritization. In *2010 International Conference on Communication Control and Computing Technologies*. 826–829.

- [149] Nader Kesserwan, Jameela Al-Jaroodi, Nader Mohamed, and Imad Jawhar. 2023. Transforming software requirements into test cases via model transformation. *International Journal of Software Engineering & Applications* 14, 4 (2023).
- [150] Nader Kesserwan, Rachida Dssouli, Jamal Bentahar, Bernard Stepien, and Pierre Labrèche. 2019. From use case maps to executable test procedures: A scenario-based approach. *Software & Systems Modeling* 18, 2 (2019), 1543–1570.
- [151] Sarah Khader and Rana Yousef. 2016. Utilizing business process models to generate software test cases. *International Journal of Computer Science Issues* 13, 2 (2016), 1–7.
- [152] Eglal M Khalifa, D Jawawi, and HA Jamil. 2019. An efficient method to generate test cases from UML-use case diagram. *International Journal of Engineering Research and Technology* 12, 7 (2019), 1138–1145.
- [153] Usman Habib Khan, Muhammad Naeem Ahmed Khan, Aamir Mehmood Mirza, Muhammad Akram, Shariqa Fakhar, Shumaila Hussain, Irfan Ahmed Magsi, and Raja Asif Wagan. 2022. Automating test case generation for android applications using model-based testing. *EMITTER International Journal of Engineering Technology* (2022), 63–82.
- [154] Monalisha Khandai, Arup Abhinna Acharya, and Durga Prasad Mohapatra. 2011. A novel approach of test case generation for concurrent systems using UML sequence diagram. In *Proceedings of the 3rd International Conference on Electronics Computer Technology*. 157–161.
- [155] Monalisha Khandai, Arup Abhinna Acharya, and Durga Prasad Mohapatra. 2011. Test case generation for concurrent system using UML combinational diagram. *International Journal of Computer Science and Information Technologies* 2, 3 (2011), 1172–1181.
- [156] Namita Khurana, Rajender Singh Chhillar, and Usha Chhillar. 2016. A Novel Technique for Generation and Optimization of Test Cases Using Use Case, Sequence, Activity Diagram and Genetic Algorithm. *Journal of Software* 11, 3 (2016), 242–250.
- [157] Namita Khurana and RS Chhillar. 2015. Test case generation and optimization using UML models and genetic algorithm. *Procedia Computer Science* 57 (2015), 996–1004.
- [158] Kazuhiro Kikuma, Takeshi Yamada, Kiyoshi Ueda, and Akira Fukuda. 2018. Automatic test case generation method for large scale communication node software. In *Proceedings of the 2018 International Conference on Emerging Internetworking, Data & Web Technologies*. 492–503.
- [159] Hyungchoul Kim, Sungwon Kang, Jongmoon Baik, and Inyoung Ko. 2007. Test cases generation from UML activity diagrams. In *Proceedings of the 8th ACIS international conference on software engineering, artificial intelligence, networking, and parallel/distributed computing*. 556–561.
- [160] Hiroyuki Kirinuki and Haruto Tanno. 2024. Chatgpt and human synergy in black-box testing: A comparative analysis. *arXiv preprint arXiv:2401.13924* (2024).
- [161] Barbara Kitchenham, O Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen Linkman. 2009. Systematic literature reviews in software engineering—A systematic literature review. *Information and Software Technology* 51, 1 (2009), 7–15.
- [162] Ehlimana Krupalija, Emir Cogo, Šeila Bećirović, Irfan Prazina, Kerim Hodžić, and Ingmar Bešić. 2022. Forward-propagation approach for generating feasible and minimum test case suites from cause-effect graph specifications. *IEEE Access* 10 (2022), 124545–124562.
- [163] U Senthil Kumaran, S Arun Kumar, and K Vijaya Kumar. 2011. An approach to automatic generation of test cases based on use cases in the requirements phase. *International Journal on Computer Science and Engineering* 3, 1 (2011), 102–113.
- [164] Debasish Kundu and Debasis Samanta. 2009. A novel approach to generate test cases from UML activity diagrams. *Journal of Object Technology* 8, 3 (2009), 65–83.
- [165] Mohammed Lafi, Thamer Alrawashed, and Ahmad Munir Hammad. 2021. Automated test cases generation from requirements specification. In *Proceedings of the 2021 International Conference on Information Technology*. 852–857.
- [166] P Lakshminarayana and TV SureshKumar. 2020. Automatic generation and optimization of test case using hybrid cuckoo search and bee colony algorithm. *Journal of Intelligent Systems* 30, 1 (2020), 59–72.
- [167] Bruno Legeard and Fabien Peureux. 2001. Generation of functional test sequences from B formal specifications presentation and industrial case study. In *Proceedings 16th Annual International Conference on Automated Software Engineering*. 377–381.
- [168] Liping Li, Xingsen Li, Tao He, and Jie Xiong. 2013. Extenics-based test case generation for UML activity diagram. *Procedia Computer Science* 17 (2013), 1186–1193.
- [169] Yihao Li, Pan Liu, Haiyang Wang, Jie Chu, and W Eric Wong. 2025. Evaluating large language models for software testing. *Computer Standards & Interfaces* 93 (2025), 103942.
- [170] Jin Wei Lim, Thiam Kian Chiew, Moon Ting Su, Simying Ong, Hema Subramaniam, Mumtaz Begum Mustafa, and Yin Kia Chiam. 2024. Test case information extraction from requirements specifications using NLP-based unified boilerplate approach. *Journal of Systems and Software* 211 (2024), 112005.
- [171] Peter A Lindsay, Sentot Kromodimoeljo, Paul A Strooper, and Mohamed Almorsy. 2015. Automation of test case generation from behavior tree requirements models. In *Proceedings of the 24th Australasian Software Engineering Conference*. 118–127.
- [172] Peter Linz and Susan H Rodger. 2022. *An introduction to formal languages and automata*. Jones & Bartlett Learning.
- [173] Wang Linzhang, Yuan Jiesong, Yu Xiaofeng, Hu Jun, Li Xuandong, and Zheng Guoliang. 2004. Generating test cases from UML activity diagram based on gray-box method. In *Proceedings of the 11th Asia-Pacific Software Engineering Conference*. 284–291.

- [174] Pan Liu and Huaikou Miao. 2010. A new approach to generating high quality test cases. In *Proceedings of the 19th IEEE Asian Test Symposium*. 71–76.
- [175] Shaoying Liu and Shin Nakajima. 2011. A “Vibration” method for automatically generating test cases based on formal specifications. In *Proceedings of the 18th Asia-Pacific Software Engineering Conference*. 73–80.
- [176] Shaoying Liu and Shin Nakajima. 2020. Automatic test case and test oracle generation based on functional scenarios in formal specifications for conformance testing. *IEEE Transactions on Software Engineering* 48, 2 (2020), 691–712.
- [177] Renate Löffler, Matthias Meyer, and Matthias Gottschalk. 2010. Formal scenario-based requirements specification and test case generation in healthcare applications. In *Proceedings of the 2010 ICSE Workshop on Software Engineering in Health Care*. 57–67.
- [178] Peng Lu. 1994. Test case generation for specification-based software testing. In *Proceedings of the 1994 conference of the Centre for Advanced Studies on Collaborative research*. 41.
- [179] David Lugato, Frédéric Maraux, Yves Le Traon, Clémentine Nebut, Véronique Normand, Hubert Dubois, J-Y Jean-Yves Pierron, and J-P Jean-Pierre Gallois. 2004. Automated functional test case synthesis from Thales industrial requirements. In *Proceedings of the 10th IEEE Real-Time and Embedded Technology and Applications Symposium*. 104–111.
- [180] Daniel Maciel, Ana CR Paiva, and Alberto Rodrigues Da Silva. 2019. From requirements to automated acceptance tests of interactive apps: An integrated model-based testing approach. In *Proceedings of the 14th International Conference on Evaluation of Novel Approaches to Software Engineering*. 265–272.
- [181] Guruvayur Mahalakshmi, Vani Vijayan, and Betina Antony. 2018. Named entity recognition for automated test case generation. *The International Arab Journal of Information Technology* 15, 1 (2018), 112–120.
- [182] Taslim Mahbub, Dana Dghaym, Aadith Shankarnarayanan, Taufiq Syed, Salsabeel Shapsough, and Imran Zuolkernan. 2024. Can GPT-4 aid in detecting ambiguities, inconsistencies, and incompleteness in requirements analysis? A comprehensive case study. *IEEE Access* 12 (2024), 171972–171992.
- [183] Qaisar Ahmad Malik, Johan Lilius, and Linas Laibinis. 2009. Scenario-based test case generation using event-B models. In *Proceedings of the 1st International Conference on Advances in System Testing and Validation Lifecycle*. 31–37.
- [184] Brian Marick. 1994. *The craft of software testing: subsystem testing including object-based and object-oriented testing*. Prentice-Hall, Inc.
- [185] M Masud, Muhammad Iqbal, MU Khan, and Farooque Azam. 2017. Automated user story driven approach for web based functional testing. *International Journal of Computer and Information Sciences* 11, 1 (2017), 91–98.
- [186] Satoshi Masuda, Tohru Matsuodani, and Kazuhiko Tsuda. 2016. Syntactic rules of extracting test cases from software requirements. In *Proceedings of the 8th International Conference on Information Management and Engineering*. 12–17.
- [187] Satoshi Masuda, Tohru Matsuodani, and Kazuhiko Tsuda. 2021. Syntax-tree similarity for test-case derivability in software requirements. In *Proceedings of the 2021 IEEE International Conference on Software Testing, Verification and Validation Workshops*. 162–172.
- [188] Alok Mathur, Shreyaan Pradhan, Prasoon Soni, Dhruvil Patel, and Rajeshkannan Regunathan. 2023. Automated test case generation using T5 and GPT-3. In *Proceedings of the 9th International Conference on Advanced Computing and Communication Systems*, Vol. 1. 1986–1992.
- [189] Micro Focus. 2024. LoadRunner Professional. <https://www.opentext.com/products/core-performance-engineering>.
- [190] Chen Mingsong, Qiu Xiaokang, and Li Xuandong. 2006. Automatic test case generation for UML activity diagrams. In *Proceedings of the 2006 International Workshop on Automation of Software Test*. 2–8.
- [191] João Miranda, Ana CR Paiva, and Alberto Rodrigues da Silva. 2020. Preliminary experiences in requirements-based security testing. In *Proceedings of the 2020 International Conference on the Quality of Information and Communications Technology*. 412–425.
- [192] Aamir Mehmood Mirza and Muhammad Naeem Ahmed Khan. 2018. An automated functional testing framework for context-aware applications. *IEEE Access* 6, 1 (2018), 46568–46583.
- [193] Abha Moitra, Kit Siu, Andrew W Crapo, Michael Durling, Meng Li, Panagiotis Manolios, Michael Meiners, and Craig McMillan. 2019. Automating requirements analysis and test case generation. *Requirements Engineering* 24, 3 (2019), 341–364.
- [194] Nor Aiza Mokhtar, Massila Kamalrudin, Safiah Sidek, Mark Robinson, and John Grundy. 2016. TestMEReq: generating abstract tests for requirements validation. In *Proceedings of the 3rd International Workshop on Software Engineering Research and Industrial Practice*. 39–45.
- [195] Ahmad Mustafa, Wan MN Wan-Kadir, Noraini Ibrahim, Muhammad Arif Shah, Muhammad Younas, Atif Khan, Mahdi Zareei, and Faisal Alanazi. 2021. Automated test case generation from requirements: A systematic literature review. *Computers, Materials and Continua* 67, 2 (2021), 1819–1833.
- [196] Alexandr Naumchev. 2019. Seamless object-oriented requirements. In *Proceedings of the 2019 International Multi-Conference on Engineering, Computer and Information Sciences*. 0743–0748.
- [197] Anmol Nayak, Vaibhav Kesri, and Rahul Kumar Dubey. 2020. Knowledge graph based automated generation of test cases in software engineering. In *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*. 289–295.
- [198] Ashalatha Nayak and Debasis Samanta. 2009. Model-based test cases synthesis using UML interaction diagrams. *ACM SIGSOFT Software Engineering Notes* 34, 2 (2009), 1–10.

- [199] Ashalatha Nayak and Debasis Samanta. 2011. Synthesis of test scenarios using UML activity diagrams. *Software & Systems Modeling* 10, 1 (2011), 63–89.
- [200] Clémentine Nebut, Franck Fleurey, Yves Le Traon, and Jean-Marc Jézéquel. 2003. A requirement-based approach to test product families. In *Proceedings of the 2003 International Workshop on Software Product-Family Engineering*. 198–210.
- [201] Clémentine Nebut, Franck Fleurey, Yves Le Traon, and J-M Jézéquel. 2003. Requirements by contracts allow automated system testing. In *Proceedings of the 14th International Symposium on Software Reliability Engineering*. 85–96.
- [202] Clementine Nebut, Franck Fleurey, Yves Le Traon, and J-M Jezequel. 2006. Automatic test generation: A use case driven approach. *IEEE Transactions on Software Engineering* 32, 3 (2006), 140–155.
- [203] Sidney Nogueira, Hugo Araujo, Renata Araujo, Juliano Iyoda, and Augusto Sampaio. 2019. Test case generation, selection and coverage from natural language. *Science of Computer Programming* 181 (2019), 84–110.
- [204] Sidney Nogueira, Hugo LS Araujo, Renata BS Araujo, Juliano Iyoda, and Augusto Sampaio. 2016. Automatic generation of test cases and test purposes from natural language. In *Formal Methods: Foundations and Applications*. 145–161.
- [205] Sidney Nogueira, Augusto Sampaio, and Alexandre Mota. 2014. Test generation from state based use case models. *Formal Aspects of Computing* 26 (2014), 441–490.
- [206] Siripol Noikajana and Taratip Suwannasart. 2008. Web service test case generation based on decision table (short paper). In *Proceedings of the 8th International Conference on Quality Software*. 321–326.
- [207] Jeff Offutt and Aynur Abdurazik. 1999. Generating tests from UML specifications. In *Proceedings of the 1999 International Conference on the Unified Modeling Language*. 416–429.
- [208] Oyindamola Olajubu, Suraj Ajit, Mark Johnson, Scott Thomson, Mark Edwards, and Scott Turner. 2017. Automated test case generation from high-level logic requirements using model transformation techniques. In *Proceedings of the 9th Computer Science and Electronic Engineering*. 178–182.
- [209] Oluwatolani Oluwagbemi and Hishammuddin Asmuni. 2015. Automatic generation of test cases from activity diagrams for UML based testing (UBT). *Jurnal Teknologi (Sciences & Engineering)* 77, 13 (2015), 37–48.
- [210] Safa Omri and Carsten Sinz. 2022. Learning to rank for test case prioritization. In *Proceedings of the 15th workshop on search-based software testing*. 16–24.
- [211] Ana CR Paiva, Daniel Maciel, and Alberto Rodrigues da Silva. 2019. From requirements to automated acceptance tests with the RSL language. In *Proceedings of the 2019 International Conference on Evaluation of Novel Approaches to Software Engineering*. 39–57.
- [212] Ana CR Paiva, Daniel Maciel, and Alberto Rodrigues da Silva. 2020. From requirements to automated acceptance tests with the RSL language. In *Proceedings of the 14th International Conference on Evaluation of Novel Approaches to Software Engineering*. 39–57.
- [213] Satya Sobhan Panigrahi, Shreshtha Shaurya, Pramita Das, Anil Kumar Swain, and Ajay Kumar Jena. 2018. Test scenarios generation using UML sequence diagram. In *Proceedings of the 2018 International Conference on Information Technology*. 50–56.
- [214] Amit Paradkar, Kuo-Chung Tai, and Mladen A. Vouk. 1997. Specification-based testing using cause-effect graphs. *Annals of Software Engineering* 4 (1997), 133–157.
- [215] Kanjane Pechtanun and Supaporn Kansomkeat. 2012. Generation test case from UML activity diagram based on AC grammar. In *Proceedings of the 2012 International Conference on Computer & Information Science*, Vol. 2. 895–899.
- [216] Constanza Pérez and Beatriz Marín. 2018. Automatic generation of test cases from UML models. *CLEI Electronic Journal* 21, 1 (2018), 1–20.
- [217] Christian Pfaller. 2008. Requirements-based test case specification by using information from model construction. In *Proceedings of the 3rd International Workshop on Automation of Software Test*. 7–16.
- [218] Laura Plein, Wendküni C Ouédraogo, Jacques Klein, and Tegawendé F Bissyandé. 2024. Automatic generation of test cases based on bug reports: A feasibility study with large language models. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 360–361.
- [219] Tomas Potuzak and Richard Lipka. 2023. Current trends in automated test case generation. In *2023 18th Conference on Computer Science and Intelligence Systems*. 627–636.
- [220] Sonali Pradhan, Mitrabinda Ray, and Santosh Kumar Swain. 2022. Transition coverage based test case generation from state chart diagram. *Journal of King Saud University-Computer and Information Sciences* 34, 3 (2022), 993–1002.
- [221] Rudolf Ramlér, Claus Klammer, and Georg Buchgeher. 2018. Applying automated test case generation in industry: a retrospective. In *2018 IEEE International Conference on Software Testing, Verification and Validation Workshops*. 364–369.
- [222] Sana Rao, Hosney Jahan, and Dongmei Liu. 2016. A search-based approach for test suite generation from extended finite state machines. In *Proceedings of the 2016 International Conference on Progress in Informatics and Computing*. 82–87.
- [223] Sanjai Rayadurgam and Mats Per Erik Heimdahl. 2001. Test-sequence generation from formal requirement models. In *Proceedings of the 6th IEEE International Symposium on High Assurance Systems Engineering*. 23–31.
- [224] Vaclav Rechtberger, Miroslav Bures, Bestoun S Ahmed, Youcef Belkhir, Jiri Nema, and Hyněk Schvach. 2022. Prioritized Variable-length Test Cases Generation for Finite State Machines. In *Proceedings of the 2022 IEEE International Conference on Software Testing, Verification and Validation Workshops*. 11–20.

- [225] Andreas Reuys, Erik Kamsties, Klaus Pohl, and Sacha Reis. 2005. Model-based system testing of software product families. In *Advanced Information Systems Engineering*. 519–534.
- [226] Andreas Reuys, Sacha Reis, Erik Kamsties, and Klaus Pohl. 2003. Derivation of domain test scenarios from activity diagrams. In *Proceedings of the International Workshop on Product Line Engineering*. 35–42.
- [227] Hassan Reza and Scott D Kerlin. 2011. A model-based testing using scenarios and constraints-based modular petri nets. In *Proceedings of the 8th International Conference on Information Technology: New Generations*. 568–573.
- [228] Matthias Riebisch, Ilka Philippow, and Marco Götze. 2002. UML-based statistical test case generation. In *Proceedings of the 2002 International Conference on Objects, Components, Architectures, Services, and Applications for a Networked World*. 394–411.
- [229] Mauricio Rocha, Adenilso Simão, and Thiago Sousa. 2021. Model-based test case generation from UML sequence diagrams using extended finite state machines. *Software Quality Journal* 29, 3 (2021), 597–627.
- [230] Alessandro Rodrigues, Jéssyka Vilela, and Carla Silva. 2024. A systematic mapping study on techniques for generating test cases from requirements. In *Proceedings of the 9th International Conference on Internet of Things, Big Data and Security*. 141–148.
- [231] Johannes Ryser and Martin Glinz. 1999. A scenario-based approach to validating and testing software systems using statecharts. *12th International Conference on Software and Systems Engineering and their Applications* (1999), 1–7.
- [232] Arash Sabbaghi. 2021. A framework for model-based testing. *Future Generation of Communication and Internet of Things* 1, 1 (2021), 28–38.
- [233] Rajesh Kumar Sahoo, Morched Derbali, Houssemer Jerbi, Doan Van Thang, P Pavan Kumar, and Sipra Sahoo. 2021. Test case generation from UML-diagrams using genetic algorithm. *Computers, Materials and Continua* 67, 2 (2021), 2321–2336.
- [234] Kenya Saiki, Shaoying Liu, Hiroyuki Okamura, and Tadashi Dohi. 2021. A tool to support vibration testing method for automatic test case generation and test result analysis. In *Proceedings of the IEEE 21st International Conference on Software Quality, Reliability and Security*. 149–156.
- [235] Yasmine Ibrahim Salem and Riham Hassan. 2010. Requirement-based test case generation and prioritization. In *Proceedings of the 2010 International Computer Engineering Conference*. 152–157.
- [236] Philip Samuel and Anju Teresa Joseph. 2008. Test sequence generation from UML sequence diagrams. In *Proceedings of the 9th ACIS International Conference on Software Engineering, Artificial Intelligence, Networking, and Parallel/Distributed Computing*. 879–887.
- [237] Philip Samuel and Rajib Mall. 2008. A novel test case design technique using dynamic slicing of UML sequence diagrams. *e-Informatica Software Engineering Journal* 2, 1 (2008), 71–92.
- [238] Philip Samuel and Rajib Mall. 2009. Slicing-based test case generation from UML activity diagrams. *ACM SIGSOFT Software Engineering Notes* 34, 6 (2009), 1–14.
- [239] Philip Samuel, Rajib Mall, and Ajay Kumar Bothra. 2008. Automatic test case generation using unified modeling language (UML) state diagrams. *The Institution of Engineering and Technology* 2, 2 (2008), 79–93.
- [240] Philip Samuel, Rajib Mall, and Pratyush Kanth. 2007. Automatic test case generation from UML communication diagrams. *Information and software technology* 49, 2 (2007), 158–171.
- [241] Valdivino Santiago, Ana Silvia Martins Do Amaral, Nandamudi L Vijaykumar, Maria de Fatima Mattiello-Francisco, Eliane Martins, and Odnei Cuesta Lopes. 2006. A practical approach for automated test case generation using statecharts. In *Proceedings of the 30th Annual International Computer Software and Applications Conference*, Vol. 2. 183–188.
- [242] Valdivino Alexandre de Santiago Júnior and Nandamudi Lankalapalli Vijaykumar. 2012. Generating model-based test cases from natural language requirements for space application software. *Software Quality Journal* 20, 1 (2012), 77–143.
- [243] Monalisa Sarma, Debasish Kundu, and Rajib Mall. 2007. Automatic test case generation from UML sequence diagram. In *Proceedings of the 15th International Conference on Advanced Computing and Communications*. 60–67.
- [244] Monalisa Sarma and Rajib Mall. 2007. Automatic test case generation from UML models. In *Proceedings of the 10th International Conference on Information Technology*. 196–201.
- [245] Monalisa Sarma and Rajib Mall. 2009. Automatic generation of test specifications for coverage of system state transitions. *Information and Software Technology* 51, 2 (2009), 418–432.
- [246] Edgar Sarmiento, Julio Cesar Sampaio do Prado Leite, and Eduardo Almentero. 2014. C&L: Generating model based test cases from natural language requirements descriptions. In *Proceedings of the IEEE 1st International Workshop on Requirements Engineering and Testing*. 32–38.
- [247] Edgar Sarmiento, Julio CSP Leite, Eduardo Almentero, and Guina Sotomayor Alzamora. 2016. Test scenario generation from natural language requirements descriptions based on Petri-Nets. *Electronic Notes in Theoretical Computer Science* 329 (2016), 123–148.
- [248] Edgar Sarmiento, Julio CSP Leite, Noemi Rodriguez, and Arndt von Staa. 2014. An automated approach of test case generation for concurrent systems from requirements descriptions. In *International Conference on Enterprise Information Systems*, Vol. 2. 339–347.
- [249] S Saroja and S Haseena. 2023. Functional and non-functional requirements in agile software development. *Agile software development: trends, Challenges and Applications* (2023), 71–86.
- [250] Yuji Sato. 2020. Specification-based test case generation with constrained genetic programming. In *Proceedings of the 20th International Conference on Software Quality, Reliability and Security Companion*. 98–103.

- [251] Yuji Sato and Taku Sugihara. 2015. Automatic generation of specification-based test cases by applying genetic algorithms in reinforcement learning. In *International Workshop on Structured Object-Oriented Formal Language and Method*. 59–71.
- [252] Manoranjan Satpathy, Qaisar A Malik, and Johan Lilius. 2006. Synthesis of scenario based test cases from B models. In *Proceedings of the 2006 International Workshop on Formal Approaches to Software Testing*. 133–147.
- [253] Matthias Schnelte. 2009. Generating test cases for timed systems from controlled natural language specifications. In *Proceedings of the 3rd IEEE International Conference on Secure Software Integration and Reliability Improvement*. 348–353.
- [254] Denis Schwachhofer, Peter Domanski, Steffen Becker, Stefan Wagner, Matthias Sauer, Dirk Pflüger, and Ilia Polian. 2024. Training large language models for system-level test program generation targeting non-functional properties. In *2024 IEEE European Test Symposium*. 1–4.
- [255] Christian Schwarzl and Bernhard Peischl. 2010. Test sequence generation from communicating UML state charts: An industrial application of symbolic transition systems. In *Proceedings of the 10th International Conference on Quality Software*. 122–131.
- [256] Irwandhi Septian, Ricky Setiawan Alianto, Ford Lumban Gaol, et al. 2017. Automated test case generation from UML activity diagram and sequence diagram using depth first search algorithm. *Procedia Computer Science* 116 (2017), 629–637.
- [257] Syed Asad Ali Shah, Syed Shafique Ali Bukhari, Mamoon Humayun, NZ Jhanjhi, and Syed Fakhar Abbas. 2019. Test case generation using unified modeling language. In *Proceedings of the International Conference on Computer and Information Sciences*. 1–6.
- [258] Saima Shaheen, Hasna Arshad, Momeena Mir, and Muhammad Tufail Hashmi. 2024. A Case Study: To Discuss the Construction of the Finite State Machine and Markov-Chain OP Models for Test Case Generation. In *Proceedings of the 5th International Conference on Advancements in Computational Sciences*. 1–9.
- [259] S Shakthi, Pratibha Srivastava, Ravi Kumar, and SG Prasad. 2024. Automated test case generation for satellite FRD using NLP and large language model. In *Proceedings of the 4th International Conference on Electrical, Computer, Communications and Mechatronics Engineering*. 1–8.
- [260] Richa Sharma and K. K. Biswas. 2014. Automated generation of test cases from logical specification of software requirements. In *Proceedings of the 9th International Conference on Evaluation of Novel Approaches to Software Engineering*. 1–8.
- [261] Richa Sharma and Kanad K Biswas. 2014. Natural language generation approach for automated generation of test cases from logical specification of requirements. In *Proceedings of the 2014 International Conference on Evaluation of Novel Approaches to Software Engineering*. 125–139.
- [262] Richa Sharma and Kanad K Biswas. 2015. Natural language generation approach for automated generation of test cases from logical specification of requirements. In *Proceedings of the 9th International Conference on Evaluation of Novel Approaches to Software Engineering*. 125–139.
- [263] Mahesh Shirole and Rajeev Kumar. 2010. A hybrid genetic algorithm based test case generation using sequence diagrams. In *Proceedings of the 3rd International Conference on Contemporary Computing*. 53–63.
- [264] Mahesh Shirole, Amit Suthar, and Rajeev Kumar. 2011. Generation of improved test cases from UML state diagram using genetic algorithm. In *Proceedings of the 4th India Software Engineering Conference*. 125–134.
- [265] Bruno Cesar F Silva, Gustavo Carvalho, and Augusto Sampaio. 2015. Test case generation from natural language requirements using CPN simulation. In *Proceedings of the 2015 Brazilian Symposium on Formal Methods*. 178–193.
- [266] Bruno Cesar F Silva, Gustavo Carvalho, and Augusto Sampaio. 2019. CPN simulation-based test case generation from controlled natural-language requirements. *Science of Computer Programming* 181 (2019), 111–139.
- [267] Arjinder Singh and Er Sumit Sharma. 2015. Functional test cases generation based on automated generated use case diagram. *International Journal of Innovative Research in Advanced Engineering* 2, 8 (2015), 105–110.
- [268] Kapil Singi, Dipin Era, and Vikrant Kaulgud. 2015. Model-based approach for automated test case generation from visual requirement specifications. In *Proceedings of the IEEE 8th International Conference on Software Testing, Verification and Validation Workshops*. 1–6.
- [269] Roopak Sinha, Cheng Pang, Gerardo Santillán Martínez, and Valeriy Vyatkin. 2016. Automatic test case generation from requirements for industrial cyber-physical systems. *at-Automatisierungstechnik* 64, 3 (2016), 216–230.
- [270] Harry M Sneed. 2018. Requirement-based testing-extracting logical test cases from requirement documents. In *Software Quality: Methods and Tools for Better Software and Systems*. 60–79.
- [271] Dehla Sokenou. 2006. Generating test sequences from UML sequence diagrams and state diagrams. In *INFORMATIK 2006–Informatik für Menschen–Band 2, Beiträge der 36. Jahrestagung der Gesellschaft für Informatik eV*. 236–240.
- [272] Stephane S Some and Xu Cheng. 2008. An approach for supporting system-level test scenarios generation from textual use cases. In *Proceedings of the 2008 ACM Symposium on Applied computing*. 724–729.
- [273] Hyun Seung Son, Young B Park, and R Young Chul Kim. 2019. MCCFG: An MOF-based multiple condition control flow graph for automatic test case generation. *Cluster Computing* 22, 1 (2019), 2461–2470.
- [274] Phil Stocks and David Carrington. 1996. A framework for specification-based testing. *IEEE Transactions on Software Engineering* 22, 11 (1996), 777–793.
- [275] Chang-ai Sun. 2008. A transformation-based approach to generating scenario-oriented test cases from UML activity diagrams for concurrent applications. In *Proceedings of the 32nd Annual IEEE International Computer Software and Applications Conference*. 160–167.

- [276] Chang-ai Sun, Baobao Zhang, and Jin Li. 2009. TSGen: A UML activity diagram-based test scenario generation tool. In *Proceedings of the 2009 International Conference on Computational Science and Engineering*, Vol. 2. 853–858.
- [277] Chang-ai Sun, Yan Zhao, Lin Pan, Xiao He, and Dave Towey. 2016. A transformation-based approach to testing concurrent programs using UML activity diagrams. *Software: Practice and Experience* 46, 4 (2016), 551–576.
- [278] Ranjita Kumari Swain, Vikas Panthi, and Prafulla Kumar Behera. 2013. Generation of test cases using activity diagram. *International Journal of Computer Science and Informatics* 3, 2 (2013), 1–10.
- [279] Santosh Kumar Swain and Durga Prasad Mohapatra. 2010. Test case generation from Behavioral UML Models. *International Journal of Computer Applications* 6, 8 (2010), 5–11.
- [280] Santosh Kumar Swain, Durga Prasad Mohapatra, and Rajib Mall. 2010. Test case generation based on state and activity models. *Journal of Object Technology* 9, 5 (2010), 1–27.
- [281] Santosh Kumar Swain, Durga Prasad Mohapatra, and Rajib Mall. 2010. Test case generation based on use case and sequence diagram. *International Journal of Software Engineering* 3, 2 (2010), 21–52.
- [282] Dassault Systems. 2016. CATIA Reqtify. <https://www.3ds.com/products-services/catia/products/reqtify>.
- [283] Luay Ho Tahat, Boris Vaysburg, Bogdan Korel, and Atef J Bader. 2001. Requirement-based automated black-box test generation. In *Proceedings of the 25th annual international computer software and applications conference*. 489–495.
- [284] Vladimir Tarasov, He Tan, Muhammad Ismail, Anders Adlemo, and Mats Johansson. 2017. Application of inference rules to a software requirements ontology to generate software test cases. In *Proceedings of the 13th International Workshop on Experiences and Directions and 5th International Workshop on Reasoner Evaluation*. 82–94.
- [285] Walaitip Thanakorncharuwit, Suwathai Kamonsantiroj, and Luepol Pipanmaekaporn. 2016. Generating test cases from UML activity diagram based on business flow constraints. In *Proceedings of the 5th International Conference on Network, Communication and Computing*. 155–160.
- [286] Christopher S. Timperley, Lauren Herckis, Claire Le Goues, and Michael Hilton. 2021. Understanding and improving artifact sharing in software engineering research. *Empirical Software Engineering* 26, 67 (2021).
- [287] Saurabh Tiwari and Atul Gupta. 2013. An approach to generate safety validation test cases from UML activity diagram. In *Proceedings of the 20th Asia-Pacific Software Engineering Conference*. 189–198.
- [288] Saurabh Tiwari and Atul Gupta. 2015. An approach of generating test requirements for agile software development. In *Proceedings of the 8th India Software Engineering Conference*. 186–195.
- [289] Huynh Khanh Vi Tran, Nauman bin Ali, Michael Unterkalmsteiner, Jürgen Börstler, and Panagiota Chatzipetrou. 2025. Quality attributes of test cases and test suites—importance & challenges from practitioners’ perspectives. *Software quality journal* 33, 1 (2025), 9.
- [290] W. T. Tsai, L. Yu, XX Liu, A. Saimi, and Y. Xiao. 2003. Scenario-based test case generation for state-based embedded systems. In *Proceedings of the 2003 IEEE International Performance, Computing, and Communications Conference*. 335–342.
- [291] Kiyoshi Ueda, Riku Shikama, and Yuki Shimizu. 2024. Automatic test cases generation with selection of training data for various system specifications. In *Proceedings of the 6th International Conference on Computer Communication and the Internet*. 1–5.
- [292] Kiyoshi Ueda and Hikaru Tsukada. 2021. Accuracy improvement by training data selection in automatic test cases generation method. In *Proceedings of the 9th International Conference on Information and Education Technology*. 438–442.
- [293] Sami Ul Haq and Usman Qamar. 2019. Ontology based test case generation for black box testing. In *Proceedings of the 8th International Conference on Educational and Information Technology*. 236–241.
- [294] Muhammad Aminu Umar and Kevin Lano. 2024. Advances in automated support for requirements engineering: A systematic literature review. *Requirements Engineering* 29, 2 (2024), 1–31.
- [295] Burak Uzun and Bedir Tekinerdogan. 2018. Model-driven architecture based testing: A systematic literature review. *Information and Software technology* 102 (2018), 30–48.
- [296] Axel Van Lamsweerde. 2001. Goal-oriented requirements engineering: A guided tour. In *Proceedings the 5th IEEE International Symposium on Requirements Engineering*. 249–262.
- [297] Boris Vaysburg, Luay H Tahat, and Bogdan Korel. 2002. Dependence analysis in reduction of requirement based test suites. In *Proceedings of the 2002 ACM SIGSOFT International Symposium on Software Testing and Analysis*. 107–111.
- [298] R Venkatesh, Ulka Shrotri, Amey Zare, and Supriya Agrawal. 2015. Cost-effective functional testing of reactive software. In *Proceedings of the 9th International Conference on Evaluation of Novel Approaches to Software Engineering*. 67–77.
- [299] R Venkatesh, Ulka Shrotri, Amey Zare, and Supriya Agrawal. 2015. On generating test cases from EDT specifications. In *International Conference on Evaluation of Novel Approaches to Software Engineering*. 1–20.
- [300] Abhishek Singh Verma, Ankur Choudhary, and Shailesh Tiwari. 2023. Software test case generation tools and techniques: A review. *International Journal of Mathematical, Engineering and Management Sciences* 8, 2 (2023), 293.
- [301] Ravi Prakash Verma and Md Rizwan Beg. 2013. Generation of test cases from software requirements using natural language processing. In *Proceedings of the 6th International Conference on Emerging Trends in Engineering and Technology*. 140–147.

- [302] Kritchawat Waitchasarn, Arthit Thongtak, and Taratip Suwannasart. 2023. Generating robot framework test scripts from user stories and scenarios for web application testing. In *Proceedings of the 8th International Conference on Computer and Communication Systems*. 795–801.
- [303] Chunhui Wang, Fabrizio Pastore, Arda Goknil, Lionel Briand, and Zohaib Iqbal. 2015. Automatic generation of system test cases from use case specifications. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*. 385–396.
- [304] Chunhui Wang, Fabrizio Pastore, Arda Goknil, and Lionel C Briand. 2022. Automatic generation of acceptance test cases from use case specifications: An NLP-based approach. *IEEE Transactions on Software Engineering* 48, 2 (2022), 585–616.
- [305] Cong Wang, Haiying Sun, Hui Dou, Hongtao Chen, and Jing Liu. 2022. MC/DC test case automatic generation for safety-critical systems. In *Proceedings of the IEEE 22nd International Conference on Software Quality, Reliability and Security*. 732–743.
- [306] Hongda Wang, Man Yang, Lihua Jiang, Jianchun Xing, Qiliang Yang, and Fuyong Yan. 2020. Test case prioritization for service-oriented workflow applications: A perspective of modification impact analysis. *IEEE Access* 8 (2020), 101260–101273.
- [307] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936.
- [308] Rong Wang, Yuji Sato, and Shaoying Liu. 2019. Specification-based test case generation with genetic algorithm. In *Proceedings of the 2019 IEEE Congress on Evolutionary Computation*. 1382–1389.
- [309] WenXuan Wang, Jun Hu, JianChen Hu, JieXiang Kang, Hui Wang, and ZhongJie Gao. 2020. Automatic test case generation from formal requirement model for avionics software. In *Proceedings of the 6th International Symposium on System and Software Reliability*. 12–20.
- [310] Michael W Whalen, Ajitha Rajan, Mats PE Heimdahl, and Steven P Miller. 2006. Coverage metrics for requirements-based testing. In *Proceedings of the 2006 International Symposium on Software Testing and Analysis*. 25–36.
- [311] Claes Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*. 1–10.
- [312] Dong Xu, Huaizhong Li, and Chiou Peng Lam. 2005. Using adaptive agents to automatically generate test scenarios from the UML activity diagrams. In *Proceedings of the 12th Asia-Pacific Software Engineering Conference*. 385–392.
- [313] Dianxiang Xu, Weifeng Xu, Michael Kent, Lijo Thomas, and Linzhang Wang. 2014. An automated test generation technique for software quality assurance. *IEEE Transactions on Reliability* 64, 1 (2014), 247–268.
- [314] Xinyue Yang, Jian Zhang, Shan Zhou, Bin Wang, and Ruixue Wang. 2021. Generating test scenarios using SysML activity diagram. In *Proceedings of the 8th International Conference on Dependable Systems and Their Applications*. 257–264.
- [315] Arezoo Yazdani Sequerloo, Mohammad Javad Amiri, Saeed Parsa, and Mahnaz Koupae. 2019. Automatic test cases generation from business process models. *Requirements Engineering* 24, 1 (2019), 119–132.
- [316] Hang Yin, Hamza Mohammed, and Sai Boyapati. 2024. Leveraging pre-trained large language models (LLMs) for on-premises comprehensive automated test case generation: An empirical study. In *Proceedings of the 9th International Conference on Intelligent Informatics and Biomedical Sciences*, Vol. 9. 597–607.
- [317] Yufei Yin, Yiqun Xu, Weikai Miao, and Yixiang Chen. 2017. An automated test case generation approach based on activity diagrams of SysML. *International Journal of Performability Engineering* 13, 6 (2017), 922–936.
- [318] Arpriansah Yonathan, Muhammad Johan Alibasa, and Rosa Reska Riskiana. 2024. Generating automated test case from sequence diagram using Pre-order Traversal. *Procedia Computer Science* 234, 1 (2024), 1730–1737.
- [319] Y. T. Yu, S. P. Ng, and Eric Y. K. Chan. 2003. Generating, selecting and prioritizing test cases from specifications with tool support. In *Proceedings of the 3rd International Conference on Quality Software (QSIC'03)*. 83–90.
- [320] Arvin Zakeriyan, Ramtin Khosravi, Hadi Safari, and Ehsan Khamespanah. 2021. Towards automatic test case generation for industrial software systems based on functional specifications. In *Proceedings of the 2021 International Conference on Fundamentals of Software Engineering*. 199–214.
- [321] Man Zhang, Tao Yue, Shaikat Ali, Huihui Zhang, and Ji Wu. 2014. A systematic approach to automatically derive test cases from use cases specified in restricted natural languages. In *Proceedings of the 2014 International Conference on System Analysis and Modeling*. 142–157.
- [322] Hanyue Zheng, Jincan Feng, Weikai Miao, and Geguang Pu. 2021. Generating test cases from requirements: A case study in railway control system domain. In *Proceedings of the 2021 International Symposium on Theoretical Aspects of Software Engineering*. 183–190.
- [323] Qiu Zhipeng, Wang Lisong, Kang Jiexiang, Gao Zhongjie, Hui Wang, Wei Yin, and Shen Xiangyu. 2021. A method of test case generation based on VRM model. In *Proceedings of the IEEE 6th International Conference on Computer and Communication Systems*. 1099–1107.
- [324] Shaofan Zhu, Peipei Xing, and Hao Wang. 2021. Model-based automated testing method for airborne system. In *Proceedings of the 5th International Conference on Aerospace System Science and Engineering*. 597–614.

A THE FINALIZED LIST OF SURVEYED STUDIES

Table A.I presents the finalized list of 267 research studies surveyed in this work.

Table A.I. List of Surveyed Studies.

No.	Title	Publication Year	Requirement Categories	Approach Categories	Test Cases Type	Domain
1	Test case generation for specification-based software testing	1994	Hybrid category	Specific approach	Concrete test cases	Not specified
2	A framework for specification-based testing	1996	Formal mathematical	Test template framework	Abstract test cases	Not specified
3	Testing from a Z Specification	1997	Formal mathematical	Category-partition method	Abstract test cases	Not specified
4	Specification-based testing using cause-effect graphs	1997	Model-based	CEG-BOR strategy	Abstract test cases	Industrial control systems
5	Testing object-oriented programs: from formal specification to test scenario generation	1998	Formal mathematical	Model-based approach	Test scenarios	Object-oriented programs
6	Test scenario generation from a structured requirements specification	1999	Semi-formal	Model-based approach	Test scenarios	Embedded systems
7	Using model checking to generate tests from requirements specifications	1999	Formal mathematical	Model-based approach	Concrete test cases	Not specified
8	A scenario-based approach to validating and testing software systems using statecharts	1999	Natural language	Model-based approach	Concrete test cases	Embedded systems
9	Automated test case generation from dynamic models	2000	Natural language	Model-based approach	Abstract test cases	Not specified
10	UML-Based integration testing	2000	Model-based	Category-partition method	Concrete test cases	Telecommunications domain
11	Automatic test generation from statecharts using model checking	2001	Model-based	Model checking approach	Abstract test cases	Embedded systems
12	Synthesis of high-level requirements models for Automatic test generation	2001	Formal mathematical	Specific approach	Test scenarios	Embedded systems
13	Test-sequence generation from formal requirement models	2001	Model-based	Specific approach	Abstract test cases	Embedded systems
14	Automated generation of statistical test cases from UML state diagrams	2001	Model-based	Statistical testing approach	Concrete test cases	Object-oriented programs
15	Requirement-based automated black-box test generation	2001	Model-based	Model-based approach	Abstract test cases	Embedded systems
16	Generation of functional test sequences from B formal specifications presentation and industrial case-study	2001	Model-based	Model-based approach	Abstract test cases	Embedded systems
17	Dependence analysis in reduction of requirement based test suites	2002	Model-based	Model-based approach	Abstract test cases	ATM system
18	A UML-based approach to system testing	2002	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
19	Derivation of domain Test scenarios from activity diagrams	2003	Model-based	Model-based approach	Test scenarios	Not specified
20	A requirement-based approach to test product families	2003	Model-based	Model-based approach	Concrete test cases	Web services and applications
21	UML-based statistical test case generation	2003	Model-based	Statistical testing approach	Abstract test cases	Object-oriented programs
22	Generating tests from UML specifications	2003	Model-based	Model-based approach	Concrete test cases	Multi-disciplinary domains
23	Scenario-based test case generation for state-based embedded systems	2003	Model-based	scenario-based state/event tree approach	Concrete test cases	Embedded systems
24	Automated-generating test case using UML statechart diagrams	2003	Model-based	Model-based approach	Abstract test cases	Not specified
25	Requirements by contracts allow automated system testing	2003	Model-based	Requirement-by-contract approach	Abstract test cases	Web services and applications
26	Generating, selecting and prioritizing test cases from specifications with tool support	2003	Semi-formal	Classification-tree method	Abstract test cases	Numerical programs

Table A.I. (Continued).

No.	Title	Publication Year	Requirement Categories	Approach Categories	Test Cases Type	Domain
27	Formal test-case generation for UML statecharts	2004	Model-based	Specific approach	Abstract test cases	Multi-disciplinary domains
28	Automated functional test case synthesis from THALES industrial requirements	2004	Natural language	symbolic execution techniques	Abstract test cases	Avionics system
29	RETNA: from requirements to testing in a natural way	2004	Natural language	Model-based approach	Abstract test cases	Multi-disciplinary domains
30	Generating test cases from UML activity diagram based on Gray-box method	2004	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
31	Model-based system testing of software product families	2005	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
32	UCM-driven testing of web applications	2005	Model-based	Model-based approach	Abstract test cases	Web applications
33	Automating test case generation from Z specifications with Isabelle	2005	Formal mathematical	Disjunctive normal form approach	Abstract test cases	Industrial control systems
34	Automating the generation and sequencing of test cases from model-based specifications	2005	Formal mathematical	Model-based approach	Concrete test cases	Numerical programs
35	Requirements traceability in automated test generation: application to smart card software validation	2005	Model-based	Model-based approach	Concrete test cases	Embedded systems
36	Using adaptive agents to Automatically generate Test scenarios from the UML activity diagrams	2005	Model-based	Adaptive agent approach	Test scenarios	Object-oriented programs
37	Synthesis of scenario based test cases from B models	2006	Model-based	Model-based approach	Abstract test cases	Numerical programs
38	Generating test sequences from UML sequence diagrams and state diagrams	2006	Model-based	Specific approach	Abstract test cases	Object-oriented programs
39	Automatic test generation: a use case driven approach	2006	Model-based	Model-based approach	Concrete test cases	Embedded systems
40	Coverage metrics for requirements-based testing	2006	Formal mathematical	Specific approach	Abstract test cases	Avionics system
41	A practical approach for automated test case generation using statecharts	2006	Model-based	Specific approach	Abstract test cases	Embedded systems
42	Generating interaction test cases for mobile phone systems from use case specifications	2006	Natural language	Specific approach	Abstract test cases	Android applications
43	Automatic test case generation from UML communication diagrams	2007	Model-based	Model-based approach	Concrete test cases	Object-oriented programs
44	Test cases generation from UML activity diagrams	2007	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
45	Test case generation by means of UML sequence diagrams and labeled transition systems	2007	Model-based	Model-based approach	Abstract test cases	Android applications
46	An Automatic tool for generating test cases from the system's requirements	2007	Model-based	Specific approach	Abstract test cases	Object-oriented programs
47	Automatic test case generation from UML sequence diagram	2007	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
48	Automatic test case generation from UML models	2007	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
49	Requirement-based testing through formal methods	2008	Semi-formal	Model-based approach	Abstract test cases	Not specified
50	A novel test case design technique using dynamic slicing of UML sequence diagrams	2008	Model-based	Edge marking dynamic slicing method	Concrete test cases	Object-oriented programs
51	Requirements-based test case specification by using information from model construction	2008	Model-based	Model-based approach	Abstract test cases	Embedded systems

Table A.I. (Continued).

No.	Title	Publication Year	Requirement Categories	Approach Categories	Test Cases Type	Domain
52	An approach for supporting system-level Test scenarios generation from textual use cases	2008	Natural language	Model-based approach	Test scenarios	Web services and applications
53	Automatic test case generation using unified modeling language (UML) state diagrams	2008	Model-based	Model-based approach	Concrete test cases	Object-oriented programs
54	Model based testing of system requirements using UML use case models	2008	Model-based	Model-based approach	Concrete test cases	Multi-disciplinary domains
55	A case study for generating test cases from use cases	2008	Natural language	Model-based approach	Abstract test cases	Web services and applications
56	A transformation-based approach to generating scenario-oriented test cases from UML activity diagrams for concurrent applications	2008	Model-based	Tree traversal algorithm	Abstract test cases	Object-Oriented Programs
57	Test Sequence Generation from UML Sequence Diagrams	2008	Model-based	Model-based approach	Abstract test cases	Not specified
58	Web service test case generation based on decision table (short paper)	2008	Semi-formal	Specific approach	Abstract test cases	Not specified
59	A novel approach to generate test cases from UML activity diagrams	2009	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
60	Automatic generation of test specifications for coverage of system state transitions	2009	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
61	Model-based test cases synthesis using UML interaction diagrams	2009	Model-based	Model-based approach	Concrete test cases	Not specified
62	Test case generation from UML subactivity and activity diagram	2009	Model-based	Specific approach	Abstract test cases	Object-oriented programs
63	Generating test cases for timed systems from controlled natural language specifications	2009	Natural language	Partial order causal link planning algorithm	Abstract test cases	Automotive domain
64	TSGen: A UML activity diagram-based test scenario generation tool	2009	Model-based	Model-based approach	Test scenarios	Object-oriented programs
65	Scenario-based test case generation using event-B models	2009	Model-based	scenario-based approach	Concrete test cases	Web services and applications
66	Tracing requirements in a model-based testing approach	2009	Semi-formal	Model-based approach	Concrete test cases	telecommunications domain
67	Developing comprehensive acceptance tests from use cases and robustness diagrams	2009	Model-based	UC-driven approach	Concrete test cases	Object-oriented programs
68	Synthesis of Test scenarios using UML activity diagrams	2009	Model-based	Model-based approach	Test scenarios	Object-oriented programs
69	Slicing-based test case generation from UML activity diagrams	2009	Model-based	Edge marking dynamic slicing method	Concrete test cases	Object-oriented programs
70	A hybrid genetic algorithm based test case generation using sequence diagrams	2010	Model-based	Heuristic algorithms	Concrete test cases	Object-oriented programs
71	Test case generation based on state and activity models	2010	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
72	Test case generation based on use case and sequence diagram	2010	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
73	Formal scenario-based requirements specification and test case generation in healthcare applications	2010	Model-based	Model-based approach	Abstract test cases	Healthcare Applications
74	Efficient test case generation for validation of UML activity diagrams	2010	Model-based	Model-based approach	Abstract test cases	Embedded systems
75	Test sequence generation from communicating UML state charts: An industrial application of symbolic transition systems	2010	Model-based	Model-based approach	Abstract test cases	Embedded systems

Table A.I. (Continued).

No.	Title	Publication Year	Requirement Categories	Approach Categories	Test Cases Type	Domain
76	Automated test case generation from use case: A model based approach	2010	Model-based	Model-based approach	Abstract test cases	Not specified
77	Test case generation from Behavioral UML models	2010	Model-based	Category-partition method	Abstract test cases	Object-oriented programs
78	Generating test cases from UML activity diagrams using the Condition-Classification Tree Method	2010	Model-based	Condition-classification free method	Abstract test cases	Object-oriented programs
79	A prolific approach for automated generation of test cases from informal requirements	2010	Natural language	Hybrid Approach	Abstract test cases	ATM System
80	A new approach to generating high quality test cases	2010	Model-based	Regular expression-based approach	Abstract test cases	Web services and applications
81	Requirement-based test case generation and prioritization	2010	Model-based	Heuristic algorithms	Abstract test cases	Embedded systems
82	An approach to Automatic generation of test cases based on use cases in the requirements phase	2011	Model-based	Specific approach	Abstract test cases	Not specified
83	Test Case Generation for Concurrent System using UML Combinational Diagram	2011	Model-based	Model-based approach	Abstract test cases	ATM System
84	A proposed test case generation technique based on activity diagrams	2011	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
85	Model based approach to assist test case creation, execution, and maintenance for test automation	2011	Model-based	Model-based approach	Concrete test cases	Multi-disciplinary domains
86	Generation of improved test cases from UML state diagram using genetic algorithm	2011	Model-based	Heuristic algorithms	Abstract test cases	Object-oriented programs
87	A model-based testing using scenarios and constraints-based modular petri nets	2011	Natural language	Model-based approach	Abstract test cases	Numerical programs
88	A novel approach of test case generation for concurrent systems using UML Sequence Diagram	2011	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
89	Generating model-based test cases from natural language requirements for space application software	2011	Natural language	Model-based approach	Abstract test cases	Embedded systems
90	Automation of test case generation from textual use cases	2011	Natural language	Model-based approach	Abstract test cases	Object-oriented programs
91	An enhanced test case generation technique based on activity diagrams	2011	Model-based	Model-based approach	Abstract test cases	Multi-disciplinary domains
92	Test case generation approach for industrial automation systems	2011	Model-based	Model-based approach	Concrete test cases	Automotive domain
93	A "Vibration" method for Automatically generating test cases based on formal specifications	2011	Formal mathematical	Vibration-method	Concrete test cases	Embedded systems
94	Automatic test case generation from functional requirements in NDT	2012	Model-based	Model-based approach	Abstract test cases	Web services and applications
95	Test case generation for concurrent object-oriented systems using combinational UML models	2012	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
96	Litmus: Generation of test cases from functional requirements in natural language	2012	Natural language	Specific approach	Abstract test cases	Not specified
97	An enhanced technique for generating hybrid coverage test cases using activity diagrams	2012	Model-based	Model-based approach	Abstract test cases	Multi-disciplinary domains

Table A.I. (Continued).

No.	Title	Publication Year	Requirement Categories	Approach Categories	Test Cases Type	Domain
98	Functional and user interface model for generating test cases	2012	Hybrid category	Model-based approach	Abstract test cases	Web services and applications
99	Generation test case from UML activity diagram based on AC grammar	2012	Model-based	Specific approach	Abstract test cases	Object-oriented programs
100	Practical Application of UML activity diagrams for the generation of test cases	2012	Model-based	Model-based approach	Concrete test cases	Object-oriented programs
101	Test generation from state based use case models	2012	Natural language	Model-based approach	Abstract test cases	Android applications
102	Automatic generation of multi-testing types test cases using requirements-based testing	2012	Model-based	Model-based approach	Abstract test cases	Not specified
103	Test case generation based on activity diagram for mobile application	2012	Model-based	Model-based approach	Abstract test cases	Android applications
104	Automatic test cases generation from UML activity diagrams using graph transformation	2013	Model-based	Model-based approach	Abstract test cases	Not specified
105	Test case generation from natural language requirements based on SCR specifications	2013	Natural language	Model-based approach	Abstract test cases	Avionics system
106	Extensics-based test case generation for UML activity diagram	2013	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
107	A model based approach to test case generation for testing the navigation behavior of dynamic web applications	2013	Model-based	Model-based approach	Abstract test cases	Web applications
108	Automatic generation of test cases for critical systems based on MC/DC criteria	2013	Semi-formal	Specific approach	Abstract test cases	Avionics system
109	An approach to generate safety validation test cases from UML activity diagram	2013	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
110	Generation of test cases from software requirements using natural language processing	2013	Natural language	Specific approach	Concrete test cases	Numerical programs
111	An automated approach of test case generation for concurrent systems from requirements descriptions	2014	Natural language	Model-based approach	Abstract test cases	ATM System
112	NAT2TESTSCR: Test case generation from natural language requirements based on SCR specifications	2014	Natural language	NAT2TEST	Concrete test cases	Multi-disciplinary domains
113	A systematic approach to Automatically derive test cases from use cases specified in restricted natural languages	2014	Natural language	Transformation-based approach	Concrete test cases	Multi-disciplinary domains
114	Model-based testing from controlled natural language requirements	2014	Natural language	Model-based approach	Concrete test cases	Multi-disciplinary domains
115	A novel approach for test case generation from UML activity diagram	2014	Model-based	Heuristic algorithms	Concrete test cases	Object-oriented programs
116	An approach to generate test cases from sequence diagram	2014	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
117	Automated generation of test cases from logical specification of software requirements	2014	Semi-formal	Semantic-head-driven generation algorithm	Abstract test cases	Multi-disciplinary domains
118	Test case generation using UML state diagram and OCL expression	2014	Hybrid category	Model-based approach	Abstract test cases	Object-oriented programs
119	Generation of test cases using activity diagram	2014	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
120	Keeping requirements and test cases consistent: Towards an ontology-based approach	2014	Semi-formal	Ontology and semantic-based methods	Abstract test cases	Embedded systems

Table A.I. (Continued).

No.	Title	Publication Year	Requirement Categories	Approach Categories	Test Cases Type	Domain
121	Generation of test cases from sliced sequence diagram	2014	Model-based	Specific approach	Concrete test cases	Not specified
122	Towards the automated generation of Abstract test cases from requirements models	2014	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
123	C&L: Generating model based test cases from natural language requirements descriptions	2014	Natural language	Model-based approach	Abstract test cases	Web services and applications
124	Abstract test case generation for behavioural testing of software product lines	2014	Formal mathematical	Model-based approach	Abstract test cases	Multi-disciplinary domains
125	Integration of requirements engineering and test-case generation via OSLC	2014	Natural language	Specific approach	Concrete test cases	Automotive domain
126	Cost-effective functional testing of reactive software	2015	Semi-formal	Row-guided random algorithm with fuzzing	Abstract test cases	Automotive domain
127	On generating test cases from EDT specifications	2015	Semi-formal	Row-guided random algorithm with fuzzing	Abstract test cases	Automotive domain
128	Functional test cases generation based on automated generated use case diagram	2015	Model-based	Model-based approach	Abstract test cases	ATM system
129	A model-driven approach for functional test case generation	2015	Model-based	Model-based approach	Abstract test cases	Web services and applications
130	Test case generation and optimization using UML models and genetic algorithm	2015	Model-based	Heuristic algorithms	Abstract test cases	Object-oriented programs
131	A graph transformation approach for Automatic test cases generation from UML activity diagrams	2015	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
132	Automatic generation of system test cases from use case specifications	2015	Natural language	Model-based approach	Concrete test cases	Automotive domain
133	An approach of generating test requirements for agile software development	2015	Natural language	Model-based approach	Abstract test cases	Object-oriented programs
134	Natural language generation approach for automated generation of test cases from logical specification of requirements	2015	Semi-formal	Semantic-head-driven approach	Abstract test cases	Multi-disciplinary domains
135	Scalable incremental test-case generation from large behavior models	2015	Natural language	Specific approach	Abstract test cases	Multi-disciplinary domains
136	NAT2TEST tool: From natural language requirements to test cases based on CSP	2015	Natural language	Model-based approach	Abstract test cases	Embedded systems
137	An automated test generation technique for software quality assurance	2015	Model-based	Model-based approach	Concrete test cases	Web services and applications
138	Model-based approach for automated test case generation from visual requirement specifications	2015	Hybrid category	Model-based approach	Abstract test cases	Web services and applications
139	Automation of test case generation from behavior tree requirements models	2015	Model-based	Model-based approach	Abstract test cases	Multi-disciplinary domains
140	Towards the generation of tests in the test description language from use case map models	2015	Model-based	Model-based approach	Test scenarios	Not specified
141	Waterfall and agile requirements-based model for automated test cases generation	2015	Natural language	Hybrid Approach	Abstract test cases	Multi-disciplinary domains
142	Automatic generation of test cases from activity diagrams for UML based testing (UBT)	2015	Model-based	Model-based approach	Abstract test cases	Object-oriented programs

Table A.I. (Continued).

No.	Title	Publication Year	Requirement Categories	Approach Categories	Test Cases Type	Domain
143	ACT (Abstract to Concrete Tests) - A tool for generating Concrete test cases from formal specification of web applications	2016	Model-based	Model-based approach	Concrete test cases	Web services and applications
144	A novel technique for generation and optimization of test cases using use case, sequence, activity diagram and genetic algorithm	2016	Model-based	Heuristic algorithms	Abstract test cases	Object-oriented programs
145	An optimized approach for automated test case generation and validation for UML diagrams	2016	Model-based	Hybrid Approach	Abstract test cases	Multi-disciplinary domains
146	A transformation-based approach to testing concurrent programs using UML activity diagrams	2016	Model-based	Transformation-based approach	Concrete test cases	Object-oriented programs
147	Automatic test generation from UML sequence diagrams for Android mobiles	2016	Model-based	Model-based approach	Concrete test cases	Android applications
148	Automatic generation of specification-based test cases by applying genetic algorithms in reinforcement learning	2016	Formal mathematical	Heuristic algorithms	Abstract test cases	ATM system
149	An optimal solution for test case generation using ROBDD graph and PSO algorithm	2016	Model-based	Heuristic algorithms	Abstract test cases	Avionics system
150	Test scenario generation from natural language requirements descriptions based on Petri-Nets	2016	Natural language	Model-based approach	Test scenarios	Web services and applications
151	Generating test cases from UML activity diagram based on business flow constraints	2016	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
152	Syntactic rules of extracting test cases from software requirements	2016	Natural language	NLP-based methods	Abstract test cases	Multi-disciplinary domains
153	Test case generation from natural language requirements using CPN simulation	2016	Natural language	NAT2TEST	Abstract test cases	Embedded systems
154	Automatic generation of test cases and test purposes from natural language	2016	Natural language	Model-based approach	Abstract test cases	Android applications
155	Automatic test case generation from requirements for industrial cyber-physical systems	2016	Semi-formal	Ontology and semantic-based methods	Abstract test cases	Embedded systems
156	Utilizing business process models to generate software test cases	2016	Model-based	Category-partition method	Abstract test cases	Object-oriented programs
157	TestMEReq: generating abstract tests for requirements validation	2016	Natural language	Specific approach	Abstract test cases	Not specified
158	Automatic generation of TestNG tests cases from UML sequence diagrams in Scrum process	2016	Model-based	Model-based approach	Concrete test cases	Not specified
159	A search-based approach for test suite generation from extended finite state machines	2016	Model-based	Search-based approach	Abstract test cases	ATM machine
160	Application of inference rules to a software requirements ontology to generate software test cases	2017	Semi-formal	Ontology and semantic-based methods	Abstract test cases	Avionics system
161	Automated user story driven approach for web based functional testing	2017	Natural language	Specific approach	Concrete test cases	Web services and applications
162	From statecharts into model checking: A hierarchy-based translation and specification patterns properties to generate test cases	2017	Model-based	Model checking approach	Abstract test cases	Multi-disciplinary domains
163	Generating test cases and procedures from use cases in dynamic software product lines	2017	Natural language	Specific approach	Concrete test cases	Multi-disciplinary domains

Table A.I. (Continued).

No.	Title	Publication Year	Requirement Categories	Approach Categories	Test Cases Type	Domain
164	Automated test case generation from UML activity diagram and sequence diagram using depth first search algorithm	2017	Model-based	Model-based approach	Abstract test cases	Not specified
165	Constructing test cases using natural language processing	2017	Natural language	NLP-based methods	Abstract test cases	Not specified
166	Transforming workflow models into automated end-to-end acceptance test cases	2017	Model-based	Transformation-based approach	Concrete test cases	Not specified
167	Automated test case generation from high-level logic requirements using model transformation techniques	2017	Semi-formal	Model-based approach	Abstract test cases	Avionics system
168	Towards the generation of end-to-end web test scripts from requirements specifications	2017	Hybrid category	Transformation-based approach	Concrete test cases	Web services and applications
169	Reqcap: Hierarchical requirements modeling and test generation for industrial control systems	2017	Hybrid category	Model-based approach	Abstract test cases	Industrial Control Systems
170	An automated test case generation approach based on activity diagrams of SysML	2017	Semi-formal	Model-based approach	Abstract test cases	Embedded systems
171	Automatic test case generation through collaboration diagram: a case study	2017	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
172	Requirement-based testing-extracting logical test cases from requirement documents	2017	Natural language	Specific approach	Abstract test cases	Multi-disciplinary domains
173	An automated functional testing framework for context-aware applications	2018	Model-based	Model-based approach	Concrete test cases	Android applications
174	Named entity recognition for automated test case generation	2018	Natural language	Machine learning-based methods	Abstract test cases	Not specified
175	Automatic test case generation method for large scale communication node software	2018	Natural language	Machine learning-based methods	Abstract test cases	Telecommunications Domain
176	Automated generation of requirements-based test cases for an adaptive cruise control system	2018	Formal mathematical	Model-based approach	Concrete test cases	Automotive domain
177	Automatic generation of test cases from UML models	2018	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
178	Automatic test transition paths generation approach from EFSM using state tree	2018	Model-based	Model-based approach	Abstract test cases	Multi-disciplinary domains
179	From natural language requirements to passive test cases using guarded assertions	2018	Natural language	SAGA approach	Concrete test cases	Automotive domain
180	Automatic test cases generation from business process models	2018	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
181	A transformation-based method for test case Automatic generation from use cases	2018	Model-based	Transformation-based approach	Concrete test cases	Object-oriented programs
182	Test scenarios generation using UML sequence diagram	2018	Model-based	Model-based approach	Test scenarios	Multi-disciplinary domains
183	From modeling to test case generation in the industrial embedded system domain	2018	Semi-formal	Model-based approach	Concrete test cases	Embedded systems
184	Automated generation of requirements-based test cases for an automotive function using the SCADE toolchain	2019	Formal mathematical	Model-based approach	Concrete test cases	Automotive domain
185	An efficient method to generate test cases from UML-use case diagram	2019	Model-based	Heuristic algorithms	Abstract test cases	Multi-disciplinary domains

Table A.I. (Continued).

No.	Title	Publication Year	Requirement Categories	Approach Categories	Test Cases Type	Domain
186	From requirements to automated acceptance tests of interactive apps: An integrated model-based testing approach	2019	Natural language	Model-based approach	Concrete test cases	Web services and applications
187	CPN simulation-based test case generation from controlled natural-language requirements	2019	Natural language	NAT2TEST	Abstract test cases	Multi-disciplinary domains
188	A memorization approach for test case generation in concurrent UML activity diagram	2019	Model-based	Model-based approach	Abstract test cases	Not specified
189	Ontology based test case generation for black box testing	2019	Semi-formal	Ontology and semantic-based methods	Concrete test cases	Not specified
190	From use case maps to executable test procedures: a scenario-based approach	2019	Model-based	Model-based approach	Concrete test cases	Embedded systems
191	Using OWL ontologies as a domain-specific language for capturing requirements for formal analysis and test case generation	2019	Semi-formal	Hybrid Approach	Abstract test cases	Industrial Control Systems
192	MCCFG: an MOF-based multiple condition control flow graph for Automatic test case generation	2019	Model-based	Model-based approach	Abstract test cases	Not specified
193	Test case generation using unified modeling language	2019	Model-based	Model-based approach	Concrete test cases	Not specified
194	An automated approach to generate test cases from use case description model	2019	Natural language	Model-based approach	Abstract test cases	Embedded systems
195	Specification-based test case generation with genetic algorithm	2019	Formal mathematical	Heuristic algorithms	Concrete test cases	Numerical programs
196	Automating requirements analysis and test case generation	2019	Semi-formal	Hybrid Approach	Concrete test cases	Avionics system
197	Towards transforming user requirements to test cases using MDE and NLP	2019	Natural language	Transformation-based approach	Abstract test cases	Not specified
198	Test case generation, selection and coverage from natural language	2019	Natural language	Model-based approach	Abstract test cases	Android applications
199	Using the SCADE toolchain to generate requirements-based test cases for an adaptive cruise control system	2019	Formal mathematical	Model-based approach	Concrete test cases	Automotive domain
200	USLTG: Test Case Automatic Generation by Transforming Use Cases	2019	Model-based	Transformation-based approach	Concrete test cases	Object-oriented programs
201	Requirements-based automated test generation for safety critical software	2019	Formal mathematical	Hybrid Approach	Concrete test cases	Avionics system
202	Generating test sequences from UML use-case diagrams	2019	Model-based	Model-based approach	Abstract test cases	Multi-disciplinary domains
203	Enhancing software testing with ontology engineering approach	2019	Natural language	Classification-tree method	Concrete test cases	Object-oriented programs
204	Impact analysis framework of test cases based on changes of use case based requirements	2019	Model-based	Hybrid Approach	Abstract test cases	Object-oriented programs
205	Knowledge graph based automated generation of test cases in software engineering	2020	Natural language	Constituency parse tree based path finding algorithm	Abstract test cases	Automotive domain
206	Test case generation algorithms and tools for specifications in natural language	2020	Natural language	Specific approach	Abstract test cases	Not specified
207	From requirements to automated acceptance tests with the RSL language	2020	Natural language	Model-based approach	Concrete test cases	Web services and applications
208	Automatic generation and optimization of test case using hybrid cuckoo search and bee colony algorithm	2020	Model-based	Hybrid Approach	Abstract test cases	ATM system

Table A.I. (Continued).

No.	Title	Publication Year	Requirement Categories	Approach Categories	Test Cases Type	Domain
209	Model-based testing from model driven architecture: A novel approach for Automatic test cases generation	2020	Hybrid category	Model-based approach	Test scenarios	Object-oriented programs
210	Preliminary experiences in requirements-based security testing	2020	Natural language	Specific approach	Concrete test cases	Web services and applications
211	Scaling test case generation for expressive decision tables	2020	Model-based	Heuristic algorithms	Abstract test cases	Multi-disciplinary domains
212	SPECMATE: Automated creation of test cases from acceptance criteria	2020	Natural language	Basic path sensitization technique	Abstract test cases	Multi-disciplinary domains
213	SYNTTest: Prototype of syntax test case generation tool	2020	Natural language	NLP-based methods	Concrete test cases	Object-oriented programs
214	Automatic test case generation from formal requirement model for avionics software	2020	Model-based	Tree traversal algorithm	Concrete test cases	Avionics system
215	Test generation tool for modified condition/decision coverage: model based Testing	2020	Model-based	Model-based approach	Concrete test cases	Not specified
216	Specification-based test case generation with constrained genetic programming	2020	Formal mathematical	Heuristic algorithms	Concrete test cases	Not specified
217	NLP-based requirements formalization for Automatic test case generation	2021	Natural language	Model-based approach	Abstract test cases	Automotive domain
218	Towards a test case generation tool based on functional requirements	2021	Natural language	Hybrid Approach	Concrete test cases	Not specified
219	Test case generation from UML-diagrams using genetic algorithm	2021	Model-based	Heuristic algorithms	Abstract test cases	Multi-disciplinary domains
220	Towards a model-driven testing framework for GUI test cases generation from user stories	2021	Natural language	Model-based approach	Concrete test cases	Desktop applications
221	Analyzing UML use cases to generate test sequences	2021	Model-based	Model-based approach	Abstract test cases	Multi-disciplinary domains
222	Executable test case generation from specifications written in natural language and test execution environment	2021	Natural language	Specific approach	Concrete test cases	Embedded systems
223	An ontology-based approach to automated test case generation	2021	Semi-formal	Specific approach	Abstract test cases	Multi-disciplinary domains
224	Accuracy improvement by training data selection in Automatic test cases generation method	2021	Natural language	Machine learning-based methods	Abstract test cases	Not specified
225	Model-based test case generation from UML sequence diagrams using extended finite state machines	2021	Model-based	Model-based approach	Concrete test cases	Object-oriented programs
226	A method of test case generation based on VRM model	2021	Model-based	Model-based approach	Concrete test cases	Embedded systems
227	Syntax-tree similarity for test-case derivability in software requirements	2021	Natural language	Syntax-tree similarity technique	Abstract test cases	Multi-disciplinary domains
228	Coverage-based grammar-guided genetic programming generation of test suites	2021	Model-based	Heuristic algorithms	Concrete test cases	Not specified
229	An improved approach for generating test cases during model-based testing using tree traversal algorithm	2021	Model-based	Tree traversal algorithm	Abstract test cases	Not specified
230	Automated test cases generation from requirements specification	2021	Natural language	Model-based approach	Abstract test cases	Not specified

Table A.I. (Continued).

No.	Title	Publication Year	Requirement Categories	Approach Categories	Test Cases Type	Domain
231	Generating Test scenarios using SysML activity diagram	2021	Model-based	Model-based approach	Test scenarios	Embedded systems
232	Generating test cases from requirements: A case study in railway control system domain	2021	Natural language	Specific approach	Concrete test cases	Embedded systems
233	Automatic generation of test cases from formal specifications using mutation testing	2021	Formal mathematical	Particle swarm optimization algorithm	Concrete test cases	Numerical programs
234	Model-based test case generation approach for mobile applications load testing using OCL enhanced activity diagrams	2021	Model-based	Model-based approach	Abstract test cases	Android applications
235	A tool to support vibration testing method for Automatic test case generation and test result analysis	2021	Formal mathematical	Vibration-method	Concrete test cases	Numerical programs
236	Forward-propagation approach for generating feasible and minimum test case suites from cause-effect graph specifications	2022	Model-based	Forward-propagation approach	Abstract test cases	Embedded systems
237	Application of the equivalent partitioning method in testing for Automatic test case generation on the Digi-OTA system	2022	Model-based	Model-based approach	Concrete test cases	Web services and applications
238	Automatic generation of acceptance test cases from use case specifications: An NLP-based approach	2022	Natural language	Model-based approach	Concrete test cases	Embedded systems
239	Automatic test case and test oracle generation based on functional scenarios in formal specifications for conformance testing	2022	Formal mathematical	Heuristic algorithms	Concrete test cases	Embedded systems
240	Transition coverage based test case generation from state chart diagram	2022	Model-based	Model-based approach	Concrete test cases	Embedded systems
241	Automating test case generation for Android applications using model-based testing	2022	Model-based	Model-based approach	Concrete test cases	Android applications
242	Prioritized variable-length test cases generation for finite state machines	2022	Model-based	Hybrid approach	Test scenarios	Embedded systems
243	Requirements-driven model checking and test generation for comprehensive verification	2022	Formal mathematical	Model-checking-based method	Concrete test cases	Embedded systems
244	Model-based automated testing method for airborne system	2022	Semi-formal	Model-based approach	Concrete test cases	Avionics system
245	MC/DC test case Automatic generation for safety-critical systems	2022	Model-based	Model-based approach	Concrete test cases	Avionics system
246	An improved test case generation method based on test requirements for testing software component	2022	Model-based	Model-based approach	Abstract test cases	Object-oriented programs
247	Generating Abstract test cases from user requirements using MDSE and NLP	2022	Natural language	Transformation-based approach	Abstract test cases	Object-oriented programs
248	Model-checking-based automated test case generation for Z formal specification of an urban railway interlocking system	2022	Formal mathematical	Model-based approach	Abstract test cases	Embedded systems
249	Automatic creation of acceptance tests by extracting conditionals from requirements: NLP approach and case study	2023	Natural language	Specific approach	Abstract test cases	Multi-disciplinary domains
250	Automated test case generation using T5 and GPT-3	2023	Natural language	NLP-based methods	Abstract test cases	Web services and applications
251	Generating robot framework test scripts from user stories and scenarios for web application testing	2023	Natural language	Specific approach	Concrete test cases	Web services and applications

Table A.I. (Continued).

No.	Title	Publication Year	Requirement Categories	Approach Categories	Test Cases Type	Domain
252	Research on software test case generation based on SysML	2023	Semi-formal	Model-based approach	Abstract test cases	Embedded systems
253	Decision tree based test case generation using story board and natural language processing	2023	Natural language	Decision tree approach	Abstract test cases	Multi-disciplinary domains
254	Transforming software requirements into test cases via model transformation	2023	Natural language	Model-based approach	Concrete test cases	Avionics system
255	An approach of test case generation with software requirement ontology	2023	Natural language	Classification-tree method	Concrete test cases	Object-oriented programs
256	Granular traceability between requirements and test cases for safety-critical software systems	2023	Model-based	Model-based approach	Concrete test cases	Avionics system
257	Automatic test sequence generation and functional coverage measurement from UML sequence diagrams	2023	Model-based	Model-based approach	Concrete test cases	Multi-disciplinary domains
258	Test case generation and history data analysis during optimization in regression testing: An NLP study	2023	Natural language	NLP-based methods	Concrete test cases	Numerical programs
259	Generating automated test case from sequence diagram using Pre-order Traversal	2024	Model-based	Model-based approach	Abstract test cases	Web Applications
260	Test case information extraction from requirements specifications using NLP-based unified boilerplate approach	2024	Natural language	NLP-based methods	Abstract test cases	Multi-disciplinary domains
261	A method for selecting training data using Doc2Vec for Automatic test Cases generation	2024	Natural language	Machine learning-based methods	Abstract test cases	Multi-disciplinary domains
262	A case study: To discuss the construction of the finite state machine and Markov-chain OP models for test case generation	2024	Model-based	Specific approach	Abstract test cases	Web Applications
263	Generating Test scenarios from NL requirements using retrieval-augmented LLMs: An industrial study	2024	Natural language	NLP-based methods	Test scenarios	Other domains
264	Automatic test cases generation with selection of training data for various system specifications	2024	Natural language	Machine learning-based methods	Abstract test cases	Multi-disciplinary domains
265	User story based automated test case generation using NLP	2024	Natural language	NLP-based methods	Abstract test cases	Not specified
266	Automated test case generation for satellite FRD using NLP and large language model	2024	Natural language	NLP-based methods	Concrete test cases	Embedded systems
267	Leveraging pre-trained large language models (LLMs) for on-premises comprehensive automated test case generation: An empirical study	2024	Natural language	NLP-based methods	Abstract test cases	Not specified

Received 4 May 2025; revised 29 August 2025; accepted 6 October 2025